

Foundations of Autonomous Robotics

Mathematics, Physics, Algorithms, and Engineering

Table of contents

1	Foundations of Autonomous Robotics	7
1.1	How to use this book	7
2	Unified Notation and Conventions	8
2.1	Core conventions	8
2.2	Phase 1 notation	8
2.2.1	General mathematics and time	8
2.2.2	Frames, geometry, and kinematics	9
2.2.3	Differential-drive motion and mechanics	9
2.2.4	Dynamics, control, limits, and verification	10
I	Part I - Motion, Mechanics, and Control	11
3	Linear Algebra Foundations	12
3.1	Why linear algebra matters	12
3.2	Prerequisites and assumptions	12
3.2.1	Prerequisites	12
3.2.2	Assumptions	12
3.3	Dependency map	13
3.4	1. Vectors: quantities with magnitude and direction	13
3.4.1	1.1 Geometric vector versus coordinate column	13
3.4.2	1.2 Addition and scalar multiplication	14
3.4.3	1.3 Length and unit direction	14
3.4.4	1.4 Dot product and angle	14
3.4.5	1.5 Linear combinations, span, and basis	17
3.5	Quick test A: vectors and bases	17
3.5.1	Answers and hints	18
3.6	2. Matrices: linear maps and basis changes	18
3.6.1	2.1 Matrix dimensions	18
3.6.2	2.2 Identity, transpose, null space, rank, and inverse	19
3.6.3	2.3 Multiplication order	21
3.7	Quick test B: matrices, rank, and inverses	21
3.7.1	Answers and hints	22
3.8	3. Determinants and signed volume	22
3.8.1	3.1 What the determinant measures	22
3.8.2	3.2 Algebraic definition	24
3.8.3	3.3 Why determinants multiply	27
3.9	Quick test C: determinants	31
3.9.1	Answers and hints	32
3.10	4. Orthogonal matrices	32

3.11	Quick test D: orthogonal matrices	33
3.11.1	Answers and hints	33
3.12	Cumulative chapter test	33
3.12.1	Cumulative answers and hints	34
3.13	References	34
4	Geometry and Coordinate Frames	35
4.1	Why coordinate frames matter	35
4.2	Prerequisites and assumptions	35
4.2.1	Prerequisites	35
4.2.2	Assumptions	36
4.3	Dependency map	36
4.4	1. Cross product and rigid-body velocity	36
4.4.1	1.1 Example: velocity of a point on a rotating rigid body	37
4.5	Quick test A: cross products and rigid-body velocity	39
4.5.1	Answers and hints	40
4.6	2. Coordinate frames	40
4.6.1	2.1 What defines a frame?	40
4.6.2	2.2 Orientation convention	40
4.6.3	2.3 Points require translation	44
4.7	3. Worked frame-change example	44
4.7.1	Interactive frame-rotation graph	46
4.8	4. Differential-drive interpretation	48
4.9	5. Failure modes and engineering checks	48
4.9.1	Missing frame labels	48
4.9.2	Reversed transformation	48
4.9.3	Mixed units	48
4.9.4	Point-vector confusion	48
4.9.5	Invalid numerical rotation	49
4.10	Quick test B: coordinate frames and rotations	49
4.10.1	Answers and hints	49
4.11	References	50
5	Differential-Drive Kinematics and Numerical Integration: Model and Derivation	51
5.1	Why this chapter matters	51
5.2	Prerequisites and assumptions	51
5.3	1. Planar pose and its derivative	52
5.3.1	1.1 Deriving the planar model	53
5.3.2	1.2 The no-sideways-motion constraint	54
5.3.3	Quick test A: planar kinematics	55
5.4	2. Differential-drive wheel kinematics	56
5.4.1	2.1 Relating wheel speeds to body motion	57
5.4.2	2.2 Characteristic motions	59
5.4.3	Quick test B: wheel-to-body mapping	61
5.5	3. From velocity to a finite pose change	62
5.5.1	3.1 Exact straight motion	62
5.5.2	3.2 Exact constant-curvature motion	63
5.5.3	Quick test C: exact finite motion	64

5.6	4. Numerical integration with Forward Euler	65
5.6.1	4.1 Derivation from a Taylor expansion	65
5.6.2	4.2 Local and accumulated error	66
5.6.3	4.3 Worked comparison with the exact update	66
5.6.4	4.4 Choosing and checking an integration method	67
5.6.5	Quick test D: Forward Euler	67
5.7	5. Engineering implications and failure modes	68
5.7.1	Frame inconsistency	68
5.7.2	Sign-convention mismatch	68
5.7.3	Unit mismatch	68
5.7.4	Incorrect or variable sampling time	68
5.7.5	Angle representation	68
5.7.6	Model error versus integration error	68
5.7.7	Encoder integration and drift	68
5.8	Cumulative chapter test	69
5.8.1	Cumulative answers and hints	69
5.9	References	69
6	Differential-Drive Kinematics and Numerical Integration: Implementation	70
6.1	Purpose and scope	70
6.2	1. Wheel-to-body velocity mapping	70
6.2.1	Software contract	70
6.2.2	Simplified reference snippet	71
6.2.3	Invariants	71
6.3	2. Body-to-wheel velocity mapping	72
6.3.1	Software contract	72
6.3.2	Implementation mapping	72
6.3.3	Invariants and required tests	73
6.4	3. Exact constant-input pose integration	73
6.4.1	Software contract	74
6.4.2	Numerically stable implementation mapping	74
6.4.3	Invariants and required tests	75
6.5	4. Forward Euler pose integration	75
6.5.1	Software contract	76
6.5.2	Implementation mapping	76
6.5.3	Exact and approximate cases	76
6.5.4	Invariants and required tests	76
7	ROS 2 Package Development Workflow	77
7.1	Goal	77
7.2	Step 1 — Source the ROS 2 underlay	77
7.3	Step 2 — Create a workspace	78
7.4	Step 3 — Create a Python package	78
7.5	Step 4 — Inspect the generated package	79
7.6	Step 5 — Complete the package metadata	79
7.7	Step 6 — Resolve dependencies and run fast tests	81
7.8	Step 7 — Build and install the package	81
7.8.1	What <code>--symlink-install</code> changes	82
7.9	Step 8 — Source the overlay and run the package	82

7.10	Step 9 — Run tests through <code>colcon</code>	83
7.11	Step 10 — Repeat the daily development loop	83
7.12	Common failure checks	84
7.13	References	84
8	Planar Rigid-Body Motion: $SE(2)$ and Twists	85
8.1	Why this chapter matters	85
8.2	Prerequisites and assumptions	85
8.3	1. Pose coordinates and rigid transformations	85
8.3.1	1.1 A pose is a relative geometric relationship	85
8.3.2	1.2 Planar rotations and $SO(2)$	86
8.3.3	1.3 Transforming a point by rotation and translation	87
8.3.4	1.4 Homogeneous transformations and $SE(2)$	87
8.3.5	1.5 Composing transformations	88
8.3.6	1.6 Inverting a transformation	89
8.3.7	1.7 Pose coordinates versus the pose itself	90
8.3.8	1.8 Worked example	90
8.4	Quick test A: pose and $SE(2)$	91
8.4.1	Answers and hints	91
8.5	2. Passive coordinate changes and active rigid motions	92
8.5.1	2.1 Passive coordinate change	92
8.5.2	2.2 Active rigid-body motion	93
8.5.3	2.3 One matrix form, three interpretations	93
8.5.4	2.4 Composition order	94
8.5.5	2.5 Why composition in $SE(2)$ is not commutative	94
8.5.6	2.6 Interpretation failures and engineering checks	95
8.6	Quick test B: interpretation and composition	95
8.6.1	Answers and hints	96
8.7	3. Time-varying rotations and rigid-body point velocity	96
8.7.1	3.1 Translational and angular velocity	96
8.7.2	3.2 Deriving the rotation-matrix derivative	97
8.7.3	3.3 Why $\mathbf{R}_{wb}$ is not a rotation matrix	98
8.7.4	3.4 Velocity of a point fixed to the body	100
8.7.5	3.5 Worked example	102
8.7.6	3.6 Assumptions, failure modes, and checks	103
8.8	Quick test C: rotation derivatives and point velocity	103
8.8.1	Answers and hints	103
8.9	4. Planar twists and $\mathfrak{se}(2)$	103
8.9.1	4.1 Twist coordinates	104
8.9.2	4.2 The hat operator and $\mathfrak{se}(2)$	104
8.9.3	4.3 Body twist	105
8.9.4	4.4 Spatial twist expressed in the world frame	106
8.9.5	4.5 Relating body and spatial twists	107
8.9.6	4.6 Differential-drive body twist	107
8.9.7	4.7 Worked example: the two twist representations	108
8.9.8	4.8 Assumptions, failure modes, and checks	109
8.10	Quick test D: planar twists	109
8.10.1	Answers and hints	109
8.11	References	110

9	Dynamics, Forces, Torque, Friction, and Actuator Limits	111
9.1	Why this chapter matters	111
9.2	Prerequisites and assumptions	111
9.3	1. From kinematics to dynamics	111
9.3.1	1.1 Position, velocity, and acceleration	111
9.3.2	1.2 Net external force	112
9.3.3	1.3 Newton’s second law and the forward-dynamics equation	112
9.3.4	1.4 Constant-force motion over one interval	113
9.3.5	1.5 Worked example	114
9.3.6	Quick test A	114
9.3.7	Quick test A answers and hints	114
9.4	2. From motor torque to wheel traction	115
9.4.1	2.1 Torque and its units	115
9.4.2	2.2 The wheel rotational balance	117
9.4.3	2.3 Combining the two drive wheels	118
9.4.4	2.4 Ideal gearing	119
9.4.5	2.5 Worked example	120
9.4.6	Quick test B	120
9.4.7	Quick test B answers and hints	120
9.5	References	121

1 Foundations of Autonomous Robotics

An autonomous robot must connect mathematical reasoning to physical action. Geometry describes where the robot and its surroundings are; mechanics explains how forces and motion interact; control turns desired behaviour into stable action; estimation and perception interpret uncertain measurements; planning compares possible futures; and software engineering joins these ideas into a dependable system.

This book develops those foundations from first principles. Each topic begins with its physical or mathematical meaning, introduces notation when it is needed, derives the central relationships step by step, and connects the theory to autonomous-robot behaviour.

1.1 How to use this book

Read the chapters in order when a topic depends on earlier mathematics. Work through each derivation with pencil and paper, check the units at every step, and complete the retrieval questions before consulting the answers. Treat code as a tool for testing a model rather than a substitute for understanding it.

The notation and conventions chapter defines the shared mathematical language used throughout the book. The linear algebra foundations chapter then develops the vector and matrix tools required by geometry, kinematics, estimation, optimisation, and control. When another source uses different symbols, translate them into the project conventions before combining results.

2 Unified Notation and Conventions

This note is the canonical notation registry for the project. Chapters may introduce local symbols, but they must not redefine symbols listed here. When a textbook uses different notation, translate it into these conventions.

2.1 Core conventions

1. Scalars use italic lowercase letters, vectors use bold lowercase letters, and matrices use bold uppercase letters.
2. Vectors are column vectors unless stated otherwise.
3. SI units are used in calculations. Angles are expressed in radians.
4. Coordinate frames are right-handed. A leading left superscript identifies the frame in which coordinates are expressed.
5. Continuous time uses t ; sampled time uses the integer index k .
6. Subscripts such as ref, cmd, min, and max describe roles rather than coordinate frames.
7. A hat, as in $\hat{\mathbf{x}}$, is reserved for an estimated quantity.

2.2 Phase 1 notation

2.2.1 General mathematics and time

Symbol	Meaning	Units or convention
a, α	Scalar quantity	Depends on the quantity
$\mathbf{v} \in \mathbb{R}^n$	Column vector with n components	Depends on the quantity
$\mathbf{A} \in \mathbb{R}^{m \times n}$	Matrix with m rows and n columns	Depends on the map
$\mathcal{S}$	Set or mathematical space	—
$\mathbf{I}_n$	$n \times n$ identity matrix	Dimensionless
$\mathbf{0}_n$	Zero vector with n components	Depends on the quantity
$(\cdot)^\top$	Transpose	—
$(\cdot)^{-1}$	Inverse, when it exists	—
$\ \mathbf{v}\ _2$	Euclidean norm of $\mathbf{v}$	Same units as $\mathbf{v}$
t	Continuous time	s
t_k	Time at sample k	s
$\Delta t = t_{k+1} - t_k$	Sampling or numerical-integration interval	s

Symbol	Meaning	Units or convention
$\dot{\mathbf{x}}, \ddot{\mathbf{x}}$	First and second derivatives with respect to time	Quantity per s and per s ²
$\mathbf{x}_k$	Value of $\mathbf{x}$ at sample k	Same units as $\mathbf{x}$

2.2.2 Frames, geometry, and kinematics

Symbol	Meaning	Units or convention
$\{a\}$	Coordinate frame named a	Right-handed
O_a	Origin of frame $\{a\}$	—
$\mathbf{e}_{x,a}, \mathbf{e}_{y,a}, \mathbf{e}_{z,a}$	Unit basis vectors of frame $\{a\}$	Dimensionless
${}^a\mathbf{v}$	Coordinates of geometric vector $\mathbf{v}$ expressed in frame $\{a\}$	Depends on $\mathbf{v}$
${}^a\mathbf{p}_P$	Position of point P relative to O_a , expressed in frame $\{a\}$	m
$\mathbf{R}_{ab}$	Rotation mapping coordinates from frame $\{b\}$ to frame $\{a\}$	$\mathbf{R}_{ab} \in SO(2)$ or $SO(3)$
${}^a\mathbf{p}_b$	Position of origin O_b relative to O_a , expressed in frame $\{a\}$	m
$\mathbf{T}_{ab}$	Homogeneous transform mapping coordinates from frame $\{b\}$ to frame $\{a\}$	$\mathbf{T}_{ab} \in SE(2)$ or $SE(3)$
$[\mathbf{v}]_{\times}$	Skew-symmetric matrix satisfying $[\mathbf{v}]_{\times} \mathbf{w} = \mathbf{v} \times \mathbf{w}$	Same units as $\mathbf{v}$
$\mathbf{q}$	Robot configuration or generalised coordinates	Components may have different units
$\mathbf{J}(\mathbf{q})$	Jacobian mapping configuration rates to a specified task velocity	Depends on the mapping

2.2.3 Differential-drive motion and mechanics

Symbol	Meaning	Units or convention
$\{w\}, \{b\}$	World and robot body frames	—
$\mathbf{q} = [x, y, \theta]^T$	Planar robot pose in the world frame	m, m, rad
θ	Counter-clockwise heading of $\{b\}$ relative to $\{w\}$	rad
v, ω	Body-forward speed and yaw rate	m s ⁻¹ , rad s ⁻¹
ϕ_L, ϕ_R	Left and right wheel angles	rad
$\dot{\phi}_L, \dot{\phi}_R$	Left and right wheel angular speeds	rad s ⁻¹

Symbol	Meaning	Units or convention
r, L	Wheel radius and distance between wheel contact lines	m
m	Robot mass	kg
$\mathbf{F}$	Force	N
τ	Torque	N m
I_z	Yaw moment of inertia about the body vertical axis	kg m ²
μ	Coefficient of friction	Dimensionless

2.2.4 Dynamics, control, limits, and verification

Symbol	Meaning	Units or convention
$\mathbf{x}(t)$	System state	Declared with each model
$\mathbf{u}(t)$	Applied control input	Declared with each model
$\mathbf{d}(t)$	External disturbance or unmodelled input	Declared with each model
$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, \mathbf{d}, t)$	Continuous-time state model	Units must be consistent
$\mathbf{x}_{k+1} = \mathbf{f}_d(\mathbf{x}_k, \mathbf{u}_k, \mathbf{d}_k, \Delta t)$	Discrete-time state update	Units must be consistent
$\mathbf{x}_{\text{ref}}$	Desired or reference state	Same units as $\mathbf{x}$
$\mathbf{e} = \mathbf{x}_{\text{ref}} - \mathbf{x}$	Tracking error; angular components must be wrapped	Same units as $\mathbf{x}$
$\mathbf{u}_{\text{cmd}}$	Controller-requested input before limits	Same units as $\mathbf{u}$
$\text{sat}(\mathbf{u}_{\text{cmd}})$	Command after applying declared actuator limits	Same units as $\mathbf{u}$
$\mathbf{K}_P, \mathbf{K}_I, \mathbf{K}_D$	Proportional, integral, and derivative gain matrices	Chosen for dimensional consistency
T_d	Command, sensing, or computation delay	s
e_p, e_θ	Position and heading tracking errors	m, rad
RMSE_p	Root-mean-square position error over a declared interval	m
M_p, t_s	Peak overshoot and settling time	Declared output units, s
$E_u = \int \mathbf{u}^\top \mathbf{u} dt$	Control-effort measure	Determined by the input units
N_{viol}	Number of declared constraint violations	Count

Part I

Part I - Motion, Mechanics, and Control

3 Linear Algebra Foundations

3.1 Why linear algebra matters

Autonomous robots represent directions, velocities, forces, sensor measurements, model parameters, and control commands with vectors. Matrices transform those vectors, combine coordinate descriptions, solve systems of equations, and expose whether information has been preserved or lost. Linear algebra provides the language needed to reason about these operations before they are used in geometry, kinematics, estimation, optimisation, and control.

This chapter develops that language from first principles. It distinguishes geometric vectors from coordinate columns, explains matrices as linear maps, connects null spaces and rank to invertibility, defines determinants as signed volume scaling, and derives the properties of orthogonal matrices used later for rotations.

3.2 Prerequisites and assumptions

3.2.1 Prerequisites

The reader should be comfortable with:

- arithmetic and elementary algebra;
- Cartesian coordinates;
- solving simple equations;
- sine and cosine; and
- the project notation conventions.

3.2.2 Assumptions

- Scalars are real numbers unless stated otherwise.
- Vectors are written as columns.
- Matrix and vector dimensions are declared when symbols are introduced.
- Coordinate bases used for Euclidean lengths and angles are orthonormal.
- Physical units are carried by the represented quantity; abstract basis and transformation matrices are dimensionless unless stated otherwise.

3.3 Dependency map

```
Vectors and linear combinations
      |
      v
Bases and coordinate columns
      |
      v
Matrices as linear maps
      |
      v
Null spaces, rank, and inverses
      |
      v
Determinants and orthogonal matrices
      |
      v
Coordinate frames, rotations, kinematics, estimation, and control
```

3.4 1. Vectors: quantities with magnitude and direction

3.4.1 1.1 Geometric vector versus coordinate column

A geometric vector is independent of any coordinate frame. A coordinate column is the numerical description of that vector in a chosen basis.

Let $\{a\}$ denote a right-handed coordinate frame with orthonormal basis vectors $\mathbf{e}_{x,a}$, $\mathbf{e}_{y,a}$, and $\mathbf{e}_{z,a}$. For a geometric vector $\mathbf{v}$, the symbol ${}^a\mathbf{v}$ denotes its coordinate column expressed in frame $\{a\}$. In three-dimensional space,

$${}^a\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}.$$

The entries mean that

$$\mathbf{v} = v_x \mathbf{e}_{x,a} + v_y \mathbf{e}_{y,a} + v_z \mathbf{e}_{z,a}.$$

The vector $\mathbf{v}$ is the physical or geometric object, whereas the column ${}^a\mathbf{v}$ depends on frame $\{a\}$. Omitting the frame label is safe only when the surrounding context makes the frame unambiguous.

3.4.2 1.2 Addition and scalar multiplication

For vectors expressed in the same frame,

$${}^a\mathbf{u} + {}^a\mathbf{v} = \begin{bmatrix} u_x + v_x \\ u_y + v_y \\ u_z + v_z \end{bmatrix}.$$

For a scalar α ,

$$\alpha {}^a\mathbf{v} = \begin{bmatrix} \alpha v_x \\ \alpha v_y \\ \alpha v_z \end{bmatrix}.$$

The algebra is valid only when the quantities are physically compatible. For example, adding two positions expressed in different frames is not meaningful until one has been transformed. Adding a velocity in metres per second to a position in metres is dimensionally invalid even if both are three-element columns.

3.4.3 1.3 Length and unit direction

The Euclidean norm is

$$\|{}^a\mathbf{v}\|_2 = \sqrt{v_x^2 + v_y^2 + v_z^2}.$$

If $\mathbf{v} \neq \mathbf{0}$, let ${}^a\mathbf{e}_v$ denote its unit direction expressed in frame $\{a\}$:

$${}^a\mathbf{e}_v = \frac{{}^a\mathbf{v}}{\|{}^a\mathbf{v}\|_2}.$$

The norm has the same physical units as the vector, while the unit vector is dimensionless because the units cancel.

3.4.4 1.4 Dot product and angle

Let two vectors be expressed as coordinate columns in the same orthonormal frame:

$${}^a\mathbf{u} = \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix}, \quad {}^a\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}.$$

The superscript T denotes the **transpose** operation. It turns the column ${}^a\mathbf{u} \in \mathbb{R}^{3 \times 1}$ into the row

$$({}^a\mathbf{u})^{\mathsf{T}} = {}^a\mathbf{u}^{\mathsf{T}} = \begin{bmatrix} u_x & u_y & u_z \end{bmatrix} \in \mathbb{R}^{1 \times 3}.$$

Here, T is an operation on the coordinate column; it is not a power. Multiplying this row by the column ${}^a\mathbf{v}$ produces a scalar:

$${}^a\mathbf{u}^{\mathsf{T}} {}^a\mathbf{v} = \begin{bmatrix} u_x & u_y & u_z \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} = u_x v_x + u_y v_y + u_z v_z.$$

This scalar is the **dot product**, or Euclidean inner product, of the two vectors:

$$\mathbf{u} \cdot \mathbf{v} := {}^a\mathbf{u}^{\mathsf{T}} {}^a\mathbf{v}.$$

The norm introduced in Section 1.3 can equivalently be defined from the dot product of a vector with itself:

$$\begin{aligned} \|{}^a\mathbf{u}\|_2 &:= \sqrt{{}^a\mathbf{u}^{\mathsf{T}} {}^a\mathbf{u}} \\ &= \sqrt{u_x^2 + u_y^2 + u_z^2}. \end{aligned}$$

Equivalently, the squared norm is

$$\|{}^a\mathbf{u}\|_2^2 = {}^a\mathbf{u}^{\mathsf{T}} {}^a\mathbf{u}.$$

Thus, dotting a vector with itself gives its squared length, and taking the nonnegative square root gives its length. Because frame $\{a\}$ is orthonormal, this coordinate norm equals the geometric length $\|\mathbf{u}\|_2$.

The coordinate formula above has a geometric interpretation. To establish it, we first derive the law of cosines.

3.4.4.1 Law of cosines

Consider a triangle with two sides of lengths r and s and included angle θ . Place its vertices at

$$\mathbf{p}_0 = (0, 0), \quad \mathbf{p}_1 = (r, 0), \quad \mathbf{p}_2 = (s \cos \theta, s \sin \theta).$$

Let c be the length of the side from $\mathbf{p}_1$ to $\mathbf{p}_2$. The horizontal and vertical displacements along that side are $s \cos \theta - r$ and $s \sin \theta$. By the Pythagorean theorem,

$$\begin{aligned} c^2 &= (s \cos \theta - r)^2 + (s \sin \theta)^2 \\ &= s^2 \cos^2 \theta - 2rs \cos \theta + r^2 + s^2 \sin^2 \theta \\ &= r^2 + s^2 (\cos^2 \theta + \sin^2 \theta) - 2rs \cos \theta \\ &= r^2 + s^2 - 2rs \cos \theta. \end{aligned}$$

Therefore, the law of cosines is

$$\boxed{c^2 = r^2 + s^2 - 2rs \cos \theta}.$$

3.4.4.2 Deriving the dot-product angle formula

For nonzero vectors $\mathbf{u}$ and $\mathbf{v}$, place their tails at the same point and let θ be the angle between them. The vectors form a triangle whose side lengths are

$$r = \|\mathbf{u}\|_2, \quad s = \|\mathbf{v}\|_2, \quad c = \|\mathbf{u} - \mathbf{v}\|_2.$$

The law of cosines therefore gives

$$\|\mathbf{u} - \mathbf{v}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2 - 2\|\mathbf{u}\|_2\|\mathbf{v}\|_2 \cos \theta.$$

Because frame $\{a\}$ is orthonormal, Euclidean lengths are preserved by its coordinate representation. Expanding the same squared length using coordinate columns gives

$$\begin{aligned} \|\mathbf{u} - \mathbf{v}\|_2^2 &= ({}^a\mathbf{u} - {}^a\mathbf{v})^\top ({}^a\mathbf{u} - {}^a\mathbf{v}) \\ &= \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2 - 2 {}^a\mathbf{u}^\top {}^a\mathbf{v}. \end{aligned}$$

Equating the two expressions for $\|\mathbf{u} - \mathbf{v}\|_2^2$ and cancelling their common terms gives

$${}^a\mathbf{u}^\top {}^a\mathbf{v} = \|\mathbf{u}\|_2\|\mathbf{v}\|_2 \cos \theta,$$

which is the dot-product angle formula. Solving it for the angle gives

$$\theta = \cos^{-1} \left(\frac{{}^a\mathbf{u}^\top {}^a\mathbf{v}}{\|\mathbf{u}\|_2\|\mathbf{v}\|_2} \right),$$

provided neither vector is zero.

The dot product is useful for projection, alignment, and checking orthogonality. If

$${}^a\mathbf{u}^\top {}^a\mathbf{v} = 0,$$

then nonzero $\mathbf{u}$ and $\mathbf{v}$ are perpendicular.

3.4.5 1.5 Linear combinations, span, and basis

Let $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{R}^n$ and let $\alpha_1, \dots, \alpha_k \in \mathbb{R}$. A **linear combination** of these vectors is any vector of the form

$$\alpha_1 \mathbf{v}_1 + \dots + \alpha_k \mathbf{v}_k.$$

The **span** of $\mathbf{v}_1, \dots, \mathbf{v}_k$ is the set of every linear combination that can be formed from them:

$$\text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\} := \left\{ \sum_{j=1}^k \alpha_j \mathbf{v}_j : \alpha_1, \dots, \alpha_k \in \mathbb{R} \right\}.$$

The vectors are **linearly independent** if

$$\alpha_1 \mathbf{v}_1 + \dots + \alpha_k \mathbf{v}_k = \mathbf{0}_n$$

implies $\alpha_1 = \dots = \alpha_k = 0$, where $\mathbf{0}_n \in \mathbb{R}^n$ is the zero vector. If a nonzero set of coefficients produces $\mathbf{0}_n$, the vectors are linearly dependent.

A **basis** of $\mathbb{R}^n$ is a set of n linearly independent vectors that spans $\mathbb{R}^n$. If $\mathbf{e}_1, \dots, \mathbf{e}_n$ is a basis, every $\mathbf{x} \in \mathbb{R}^n$ has a unique expansion

$$\mathbf{x} = x_1 \mathbf{e}_1 + \dots + x_n \mathbf{e}_n.$$

The scalar coefficients $x_1, \dots, x_n \in \mathbb{R}$ form the coordinate column of $\mathbf{x}$ in that basis. Uniqueness is essential: if the basis vectors were dependent, the same vector could have more than one coefficient description.

The **standard basis** of $\mathbb{R}^n$ is $\mathbf{e}_1, \dots, \mathbf{e}_n$, where $\mathbf{e}_i$ has entry 1 in position i and entry 0 in every other position. In the standard basis,

$$\mathbf{x} = x_1 \mathbf{e}_1 + \dots + x_n \mathbf{e}_n \quad \Longleftrightarrow \quad \mathbf{x} = \begin{bmatrix} x_1 & \dots & x_n \end{bmatrix}^T.$$

3.5 Quick test A: vectors and bases

Complete this test without looking back at the section.

1. **Recall:** What is the difference between a geometric vector $\mathbf{v}$ and a coordinate column ${}^a\mathbf{v}$?
2. **Explanation:** Why must coordinate columns be associated with a basis?
3. **Derivation:** Use the law of cosines and an algebraic expansion of $\|\mathbf{u} - \mathbf{v}\|_2^2$ to derive ${}^a\mathbf{u}^T {}^a\mathbf{v} = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos \theta$.
4. **Application:** Find the norm and unit direction of $\mathbf{x} = [3 \ 4]^T$.
5. **Limitations:** Why is the angle formula undefined when either vector is zero?
6. **Basis:** Explain why two parallel nonzero vectors cannot form a basis of $\mathbb{R}^2$.

3.5.1 Answers and hints

1. $\mathbf{v}$ is the coordinate-independent geometric object; ${}^a\mathbf{v}$ is its numerical representation in basis or frame $\{a\}$.
2. The entries are coefficients multiplying particular basis vectors. Without the basis, the numbers do not determine physical or geometric directions.
3. Equate the law-of-cosines expression for $\|\mathbf{u} - \mathbf{v}\|_2^2$ with its dot-product expansion and cancel the common norm terms.
4. $\|\mathbf{x}\|_2 = 5$ and the unit direction is $[0.6 \ 0.8]^\top$.
5. The denominator $\|\mathbf{u}\|_2\|\mathbf{v}\|_2$ becomes zero, and a zero vector has no direction.
6. One vector is a scalar multiple of the other, so their span is only a line and their coefficient representation is not unique.

3.6 2. Matrices: linear maps and basis changes

3.6.1 2.1 Matrix dimensions

A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ maps an n -element coordinate vector to an m -element coordinate vector:

$$\mathbf{y} = \mathbf{A}\mathbf{x}$$

where $\mathbf{y} \in \mathbb{R}^m$ and $\mathbf{x} \in \mathbb{R}^n$.

If

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

then

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 \\ a_{21}x_1 + a_{22}x_2 \end{bmatrix}.$$

Equivalently, if $\mathbf{a}_1$ and $\mathbf{a}_2$ are the columns of $\mathbf{A}$,

$$\mathbf{A}\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2.$$

This column interpretation is central to rotation matrices: their columns are the axes of one frame expressed in another frame.

3.6.2 2.2 Identity, transpose, null space, rank, and inverse

Let $\mathbf{I}_n$ denote the $n \times n$ identity matrix. It leaves compatible coordinates unchanged:

$$\mathbf{I}_n \mathbf{x} = \mathbf{x}.$$

The transpose swaps rows and columns:

$$(\mathbf{A}^\top)_{ij} = a_{ji}.$$

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$. Its **column space**, denoted $\text{Col}(\mathbf{A})$, is the set of all linear combinations of its columns. The **rank**, denoted $\text{rank}(\mathbf{A})$, is the number of linearly independent columns, equivalently the dimension of $\text{Col}(\mathbf{A})$.

The **null space** of $\mathbf{A}$ is

$$\mathcal{N}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{0}_m\},$$

where $\mathbf{0}_m \in \mathbb{R}^m$ is the zero vector. The null space contains all input directions that the mapping sends to zero.

3.6.2.1 Injective mappings and unique recovery

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ define the linear map

$$T_{\mathbf{A}} : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad T_{\mathbf{A}}(\mathbf{x}) = \mathbf{A}\mathbf{x}.$$

The map $T_{\mathbf{A}}$ is **injective**, or **one-to-one**, if equal outputs can occur only when the inputs are equal. Formally, for every $\mathbf{x}^{(1)}, \mathbf{x}^{(2)} \in \mathbb{R}^n$, where the parenthesised superscripts (1) and (2) label two inputs and are not exponents,

$$\mathbf{A}\mathbf{x}^{(1)} = \mathbf{A}\mathbf{x}^{(2)} \implies \mathbf{x}^{(1)} = \mathbf{x}^{(2)}.$$

Equivalently, distinct inputs must produce distinct outputs:

$$\mathbf{x}^{(1)} \neq \mathbf{x}^{(2)} \implies \mathbf{A}\mathbf{x}^{(1)} \neq \mathbf{A}\mathbf{x}^{(2)}.$$

The map is **not injective** if at least one pair of distinct inputs produces the same output:

$$\exists \mathbf{x}^{(1)}, \mathbf{x}^{(2)} \in \mathbb{R}^n : \quad \mathbf{x}^{(1)} \neq \mathbf{x}^{(2)}, \quad \mathbf{A}\mathbf{x}^{(1)} = \mathbf{A}\mathbf{x}^{(2)}.$$

To connect injectivity to the null space, suppose two inputs have the same output. Subtracting the outputs gives

$$\begin{aligned}\mathbf{A}\mathbf{x}^{(1)} &= \mathbf{A}\mathbf{x}^{(2)} \\ \implies \mathbf{A}(\mathbf{x}^{(1)} - \mathbf{x}^{(2)}) &= \mathbf{0}_m.\end{aligned}$$

If $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}_n\}$, then $\mathbf{x}^{(1)} - \mathbf{x}^{(2)} = \mathbf{0}_n$, so the inputs must be equal. Conversely, if a nonzero vector $\mathbf{z} \in \mathcal{N}(\mathbf{A})$ exists, then for any $\mathbf{x} \in \mathbb{R}^n$,

$$\mathbf{A}(\mathbf{x} + \mathbf{z}) = \mathbf{A}\mathbf{x} + \mathbf{A}\mathbf{z} = \mathbf{A}\mathbf{x}.$$

Because $\mathbf{z} \neq \mathbf{0}_n$, the inputs $\mathbf{x}$ and $\mathbf{x} + \mathbf{z}$ are different but produce the same output. Therefore,

$$\boxed{T_{\mathbf{A}} \text{ is injective} \iff \mathcal{N}(\mathbf{A}) = \{\mathbf{0}_n\} \iff \mathbf{A} \text{ has full column rank}}.$$

For a square matrix, these conditions are also equivalent to nonsingularity and invertibility. If the map is injective, its output uniquely identifies its input; if it is not injective, some input information cannot be recovered from the output.

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a square matrix with columns $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^n$, and let $\mathbf{x} = [x_1 \ \dots \ x_n]^T \in \mathbb{R}^n$. Matrix-vector multiplication forms the linear combination

$$\mathbf{A}\mathbf{x} = x_1\mathbf{a}_1 + \dots + x_n\mathbf{a}_n = \sum_{j=1}^n x_j\mathbf{a}_j.$$

For every matrix, $\mathbf{x} = \mathbf{0}$ implies $\mathbf{A}\mathbf{x} = \mathbf{0}$. The square matrix $\mathbf{A}$ is **nonsingular** when the converse is also true:

$$\mathbf{A}\mathbf{x} = \mathbf{0} \iff \mathbf{x} = \mathbf{0}.$$

Equivalently,

$$\mathbf{x} \neq \mathbf{0} \implies \mathbf{A}\mathbf{x} \neq \mathbf{0}.$$

Because $\mathbf{A}\mathbf{x} = \sum_{j=1}^n x_j\mathbf{a}_j$, nonsingularity means that

$$x_1\mathbf{a}_1 + \dots + x_n\mathbf{a}_n = \mathbf{0}$$

only when $x_1 = \dots = x_n = 0$. This is precisely the definition that the columns $\mathbf{a}_1, \dots, \mathbf{a}_n$ are linearly independent.

A square matrix that is not nonsingular is called **singular**. For a singular matrix, at least one nonzero $\mathbf{x}$ satisfies $\mathbf{A}\mathbf{x} = \mathbf{0}$, so the mapping loses information about that input direction.

More generally, if $\mathbf{A} \in \mathbb{R}^{m \times n}$, then its columns are $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^m$ and $\mathbf{A}\mathbf{x} = \sum_{j=1}^n x_j\mathbf{a}_j$. The condition $\mathbf{A}\mathbf{x} = \mathbf{0}_m \Rightarrow \mathbf{x} = \mathbf{0}_n$ means that $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}_n\}$ and that all n columns are linearly independent. In that case $\text{rank}(\mathbf{A}) = n$, which is called **full column rank**. The term *nonsingular* is normally reserved for the square case $m = n$.

Only a nonsingular square matrix has an inverse $\mathbf{A}^{-1} \in \mathbb{R}^{n \times n}$. The inverse reverses the action of $\mathbf{A}$ and satisfies

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}_n.$$

3.6.3 2.3 Multiplication order

Let n be a positive integer. Let $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{R}^{n \times n}$ be three matrices, and let $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{R}^n$ be coordinate vectors. Define the intermediate vector $\mathbf{z}$ and output vector $\mathbf{y}$ by

$$\mathbf{z} = \mathbf{B}\mathbf{x} \quad \text{and} \quad \mathbf{y} = \mathbf{A}\mathbf{z}.$$

Substituting the expression for $\mathbf{z}$ into the expression for $\mathbf{y}$ gives

$$\mathbf{y} = \mathbf{A}(\mathbf{B}\mathbf{x}) = (\mathbf{A}\mathbf{B})\mathbf{x}.$$

Thus, the rightmost matrix acts first: $\mathbf{B}$ maps $\mathbf{x}$ to $\mathbf{z}$, and then $\mathbf{A}$ maps $\mathbf{z}$ to $\mathbf{y}$.

The third matrix $\mathbf{C}$ allows three transformations to be composed. Matrix multiplication is **associative**, meaning that changing the grouping does not change the product:

$$(\mathbf{A}\mathbf{B})\mathbf{C} = \mathbf{A}(\mathbf{B}\mathbf{C}).$$

Both sides are matrices in $\mathbb{R}^{n \times n}$. However, matrix multiplication is generally **not commutative**, meaning that changing the order can change the result:

$$\mathbf{A}\mathbf{B} \neq \mathbf{B}\mathbf{A}.$$

This order becomes critical when several coordinate transformations are composed.

3.7 Quick test B: matrices, rank, and inverses

1. **Recall:** For $\mathbf{A} \in \mathbb{R}^{m \times n}$, what are the dimensions of an input $\mathbf{x}$ and output $\mathbf{y} = \mathbf{A}\mathbf{x}$?
2. **Explanation:** What does column j of $\mathbf{A}$ tell us about the mapping?
3. **Derivation:** Explain why $\mathbf{A}\mathbf{x} = \mathbf{0}$ has only the zero solution exactly when the columns of $\mathbf{A}$ are linearly independent.
4. **Application:** Determine the null space and rank of $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$.
5. **Limitations:** Why can a nonsquare matrix not have a two-sided inverse of the same kind as a nonsingular square matrix?
6. **Composition:** If $\mathbf{z} = \mathbf{B}\mathbf{x}$ and $\mathbf{y} = \mathbf{A}\mathbf{z}$, which matrix acts first in $\mathbf{y} = \mathbf{A}\mathbf{B}\mathbf{x}$?
7. **Injectivity:** Define an injective matrix mapping and explain why it is equivalent to $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$.

3.7.1 Answers and hints

1. $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} \in \mathbb{R}^m$.
2. It is the output produced by applying $\mathbf{A}$ to the corresponding standard basis input; general outputs are linear combinations of the columns.
3. The equation is the linear combination $x_1 \mathbf{a}_1 + \cdots + x_n \mathbf{a}_n = \mathbf{0}$. Only zero coefficients solve it precisely when the columns are independent.
4. The second column is twice the first, so the rank is 1. Solving $x_1 + 2x_2 = 0$ gives $\mathcal{N}(\mathbf{A}) = \{t[-2 \ 1]^\top : t \in \mathbb{R}\}$.
5. A two-sided inverse would have to undo mappings in both domain and codomain, which requires equal dimensions and no lost directions.
6. $\mathbf{B}$ acts first, followed by $\mathbf{A}$.
7. A map is injective if $\mathbf{A}\mathbf{x}^{(1)} = \mathbf{A}\mathbf{x}^{(2)}$ implies $\mathbf{x}^{(1)} = \mathbf{x}^{(2)}$. If $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$, subtracting equal outputs gives $\mathbf{A}(\mathbf{x}^{(1)} - \mathbf{x}^{(2)}) = \mathbf{0}$, so $\mathbf{x}^{(1)} - \mathbf{x}^{(2)} = \mathbf{0}$. Conversely, a nonzero null vector $\mathbf{z}$ makes $\mathbf{x}$ and $\mathbf{x} + \mathbf{z}$ distinct inputs with the same output.

3.8 3. Determinants and signed volume

3.8.1 3.1 What the determinant measures

The determinant is defined for a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and returns a scalar:

$$\det : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}.$$

Geometrically, $|\det(\mathbf{A})|$ is the factor by which the linear map $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$ scales n -dimensional volume. The sign records whether the ordered basis orientation is preserved or reversed.

For $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \in \mathbb{R}^{2 \times 2}$,

$$\det(\mathbf{A}) = ad - bc.$$

Its magnitude is the area of the parallelogram formed by the columns of $\mathbf{A}$. For $\mathbf{A} = [\mathbf{a}_1 \ \mathbf{a}_2 \ \mathbf{a}_3] \in \mathbb{R}^{3 \times 3}$, the determinant is the scalar triple product

$$\det(\mathbf{A}) = \mathbf{a}_1^\top (\mathbf{a}_2 \times \mathbf{a}_3).$$

Its magnitude is the volume of the parallelepiped formed by the columns.

3.8.1.1 Determinant sign, the right-hand rule, and reflection

The columns of $\mathbf{A} = [\mathbf{a}_1 \cdots \mathbf{a}_n]$ are the images of the standard basis vectors because $\mathbf{A}\mathbf{e}_i = \mathbf{a}_i$. The determinant sign therefore compares the orientation of the ordered transformed columns with the orientation of the ordered standard basis.

In three dimensions, the standard basis is right-handed:

$$\mathbf{e}_1 \times \mathbf{e}_2 = \mathbf{e}_3.$$

For $\mathbf{A} = [\mathbf{a}_1 \ \mathbf{a}_2 \ \mathbf{a}_3] \in \mathbb{R}^{3 \times 3}$, the determinant can be written as

$$\det(\mathbf{A}) = (\mathbf{a}_1 \times \mathbf{a}_2)^\top \mathbf{a}_3.$$

The cross product $\mathbf{a}_1 \times \mathbf{a}_2$ points in the right-hand-rule normal direction determined by the first two columns. Consequently:

- If $\det(\mathbf{A}) > 0$, then $\mathbf{a}_3$ has a positive component in that right-hand-rule direction. The ordered columns $(\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3)$ are right-handed, so the transformation preserves orientation.
- If $\det(\mathbf{A}) < 0$, then $\mathbf{a}_3$ points to the opposite side of the plane spanned by $\mathbf{a}_1$ and $\mathbf{a}_2$. The ordered columns are left-handed, so the transformation reverses orientation.
- If $\det(\mathbf{A}) = 0$, then the three columns are coplanar or otherwise linearly dependent. The three-dimensional volume collapses to zero, so a full three-dimensional orientation is no longer defined.

A reflection reverses orientation. For example, reflection across the y - z plane is represented by

$$\mathbf{S} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \det(\mathbf{S}) = -1.$$

This transformation sends $\mathbf{e}_1$ to $-\mathbf{e}_1$ while leaving $\mathbf{e}_2$ and $\mathbf{e}_3$ unchanged. The transformed basis is left-handed because

$$(-\mathbf{e}_1) \times \mathbf{e}_2 = -\mathbf{e}_3,$$

whereas its third basis vector is $+\mathbf{e}_3$. Thus, the negative determinant detects the orientation reversal introduced by the reflection.

A matrix with negative determinant need not be a pure reflection: it may also stretch, scale, or shear. The negative sign means that its total action contains an orientation-reversing, or reflection-like, component. By contrast, a proper rotation preserves right-handedness and has determinant $+1$.

3.8.2 3.2 Algebraic definition

The determinant is the unique scalar-valued function of the columns of a square matrix with three defining properties:

1. **Multilinearity:** it is linear in each column while the other columns are held fixed.
2. **Alternation:** swapping two columns changes the sign; consequently, a matrix with repeated columns has determinant zero.
3. **Normalisation:** $\det(\mathbf{I}_n) = 1$.

3.8.2.1 Multilinearity in one column

Write the columns of $\mathbf{A} \in \mathbb{R}^{n \times n}$ as $\mathbf{A} = [\mathbf{a}_1 \ \cdots \ \mathbf{a}_n]$, where each $\mathbf{a}_i \in \mathbb{R}^n$. Choose a column index $j \in \{1, \dots, n\}$ and hold every column except column j fixed. For any vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and scalars $\alpha, \beta \in \mathbb{R}$, linearity in column j means

$$\begin{aligned} \det(\mathbf{a}_1, \dots, \mathbf{a}_{j-1}, \alpha \mathbf{u} + \beta \mathbf{v}, \mathbf{a}_{j+1}, \dots, \mathbf{a}_n) \\ = \alpha \det(\mathbf{a}_1, \dots, \mathbf{a}_{j-1}, \mathbf{u}, \mathbf{a}_{j+1}, \dots, \mathbf{a}_n) \\ + \beta \det(\mathbf{a}_1, \dots, \mathbf{a}_{j-1}, \mathbf{v}, \mathbf{a}_{j+1}, \dots, \mathbf{a}_n). \end{aligned}$$

Thus, a sum placed in one column can be separated into a sum of determinants, and a scalar multiplying one column can be moved outside the determinant. The word **multilinear** means that this linearity property holds separately for every column. It does not mean that the determinant is linear in the whole matrix at once; in general,

$$\det(\mathbf{A} + \mathbf{B}) \neq \det(\mathbf{A}) + \det(\mathbf{B}).$$

For a two-dimensional check, let $\mathbf{u} = [u_1 \ u_2]^\top$, $\mathbf{v} = [v_1 \ v_2]^\top$, and $\mathbf{w} = [w_1 \ w_2]^\top$. Linearity in the first column follows directly from

$$\begin{aligned} \det \begin{bmatrix} \alpha u_1 + \beta v_1 & w_1 \\ \alpha u_2 + \beta v_2 & w_2 \end{bmatrix} &= (\alpha u_1 + \beta v_1)w_2 - w_1(\alpha u_2 + \beta v_2) \\ &= \alpha(u_1 w_2 - w_1 u_2) + \beta(v_1 w_2 - w_1 v_2) \\ &= \alpha \det[\mathbf{u} \ \mathbf{w}] + \beta \det[\mathbf{v} \ \mathbf{w}]. \end{aligned}$$

3.8.2.2 Alternation and repeated columns

Let $\mathbf{A}'$ be obtained from $\mathbf{A}$ by swapping columns i and j , where $i \neq j$. Alternation means

$$\det(\mathbf{A}') = -\det(\mathbf{A}).$$

A single swap reverses the orientation of the ordered columns, so it reverses the sign of the signed volume without changing its magnitude.

Now suppose columns i and j are equal to the same vector $\mathbf{c} \in \mathbb{R}^n$. Let the determinant of this matrix be D . Swapping the two equal columns leaves the matrix numerically unchanged, so its determinant is still D . Alternation simultaneously requires the determinant to become $-D$. Therefore,

$$D = -D \implies 2D = 0 \implies D = 0.$$

Hence, every matrix with two identical columns has determinant zero. Geometrically, two identical column directions cannot span an n -dimensional parallelepiped, so its n -dimensional volume collapses to zero.

Equivalently, if $\mathbf{A} = [a_{ij}] \in \mathbb{R}^{n \times n}$, then

$$\det(\mathbf{A}) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{j=1}^n a_{\sigma(j),j}.$$

Here, S_n is the set of all permutations of the indices $1, \dots, n$. The sign $\text{sgn}(\sigma)$ equals $+1$ when the permutation can be produced by an even number of pairwise swaps and -1 when it requires an odd number.

If the columns of $\mathbf{A}$ are linearly dependent, one column can be written as a linear combination of the others. Multilinearity and alternation then force $\det(\mathbf{A}) = 0$. Conversely, $\det(\mathbf{A}) = 0$ means that the signed volume has collapsed, so the columns do not span $\mathbb{R}^n$. Therefore,

$$\boxed{\det(\mathbf{A}) \neq 0 \iff \mathbf{A} \text{ is nonsingular}}.$$

3.8.2.3 Example: a singular matrix loses one input component

Consider the linear map $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$ from $\mathbb{R}^2$ to $\mathbb{R}^2$, where

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Its determinant is

$$\det(\mathbf{A}) = 0.$$

For an input coordinate column

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2,$$

the output is

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} x_1 \\ 0 \end{bmatrix} \in \mathbb{R}^2.$$

The output retains x_1 but completely removes x_2 . For example,

$$\mathbf{A} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}, \quad \mathbf{A} \begin{bmatrix} 3 \\ 7 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}.$$

The two inputs are different, but their outputs are identical. From the output $[3 \ 0]^\top$, we can recover $x_1 = 3$, but we cannot determine whether x_2 was 1, 7, or any other real number. In fact,

$$\mathcal{N}(\mathbf{A}) = \left\{ t \begin{bmatrix} 0 \\ 1 \end{bmatrix} : t \in \mathbb{R} \right\},$$

so the entire input direction $[0 \ 1]^\top$ is lost.

Geometrically, $\mathbf{A}$ maps the entire two-dimensional plane onto the one-dimensional x -axis:

$$\mathbb{R}^2 \longrightarrow \left\{ \begin{bmatrix} y_1 \\ 0 \end{bmatrix} : y_1 \in \mathbb{R} \right\}.$$

Every two-dimensional region is flattened into a line segment, whose two-dimensional area is zero. This agrees with $\det(\mathbf{A}) = 0$.

Recall that **not injective** means that there exist at least two distinct inputs $\mathbf{x}^{(1)}, \mathbf{x}^{(2)} \in \mathbb{R}^n$ with the same output:

$$\mathbf{x}^{(1)} \neq \mathbf{x}^{(2)}, \quad \mathbf{A}\mathbf{x}^{(1)} = \mathbf{A}\mathbf{x}^{(2)}.$$

The statement does not mean that every pair of inputs has the same output; it means that at least one distinct pair cannot be distinguished from the output.

If $\mathbf{A}$ is singular, then there exists $\mathbf{z} \in \mathcal{N}(\mathbf{A})$ such that $\mathbf{A}\mathbf{z} = \mathbf{0}$. This implies $\mathbf{A}(\mathbf{x} + \mathbf{z}) = \mathbf{A}\mathbf{x}$ for all $\mathbf{x} \in \mathbb{R}^n$. Hence, a singular matrix is not injective and different inputs can produce the same output.

For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the general equivalences are

$\det(\mathbf{A}) = 0 \iff \mathbf{A} \text{ is singular}$ $\iff \mathcal{N}(\mathbf{A}) \neq \{\mathbf{0}\}$ $\iff \mathbf{A} \text{ is not injective}$ $\iff \text{different inputs can produce the same output}$ $\iff \text{the input cannot be uniquely recovered.}$

3.8.3 3.3 Why determinants multiply

Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$. The product $\mathbf{AB}$ represents the composition

$$\mathbf{x} \mapsto \mathbf{Bx} \mapsto \mathbf{ABx}.$$

Geometrically, $\mathbf{B}$ first scales signed volume by $\det(\mathbf{B})$, and $\mathbf{A}$ then scales the result by $\det(\mathbf{A})$. This suggests

$$\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B}).$$

A formal proof follows from the defining properties of the determinant. Hold $\mathbf{A}$ fixed and define

$$F : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}, \quad F(\mathbf{B}) := \det(\mathbf{AB}).$$

Write $\mathbf{B} = [\mathbf{b}_1 \ \cdots \ \mathbf{b}_n]$, where each $\mathbf{b}_j \in \mathbb{R}^n$. Then

$$\mathbf{AB} = [\mathbf{Ab}_1 \ \cdots \ \mathbf{Ab}_n].$$

To check linearity in column j , let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and $\alpha, \beta \in \mathbb{R}$. Because multiplication by $\mathbf{A}$ is linear,

$$\mathbf{A}(\alpha\mathbf{u} + \beta\mathbf{v}) = \alpha\mathbf{Au} + \beta\mathbf{Av}.$$

The determinant is linear in column j , so

$$\begin{aligned} F(\mathbf{b}_1, \dots, \alpha\mathbf{u} + \beta\mathbf{v}, \dots, \mathbf{b}_n) \\ = \alpha F(\mathbf{b}_1, \dots, \mathbf{u}, \dots, \mathbf{b}_n) + \beta F(\mathbf{b}_1, \dots, \mathbf{v}, \dots, \mathbf{b}_n). \end{aligned}$$

Thus, F is multilinear in the columns of $\mathbf{B}$. If two columns of $\mathbf{B}$ are equal, their images under $\mathbf{A}$ are equal, so $\mathbf{AB}$ has repeated columns and $F(\mathbf{B}) = 0$. Swapping two columns of $\mathbf{B}$ swaps the corresponding columns of $\mathbf{AB}$ and changes the determinant sign.

Hence, F is an alternating multilinear scalar-valued function of the columns of $\mathbf{B}$. We claim that

$$F(\mathbf{B}) = c \det(\mathbf{B}), \quad c := F(\mathbf{e}_1, \dots, \mathbf{e}_n).$$

The following expansion proves the claim.

Let $\mathbf{e}_1, \dots, \mathbf{e}_n$ be the standard basis of $\mathbb{R}^n$. Because it is a basis, every column $\mathbf{b}_j$ of $\mathbf{B}$ has the unique expansion

$$\mathbf{b}_j = \sum_{i=1}^n b_{ij} \mathbf{e}_i.$$

Using multilinearity once for each of the n columns gives

$$F(\mathbf{B}) = \sum_{i_1=1}^n \cdots \sum_{i_n=1}^n \left(\prod_{j=1}^n b_{i_j, j} \right) F(\mathbf{e}_{i_1}, \dots, \mathbf{e}_{i_n}).$$

This sum initially contains n^n terms. If two indices are equal, such as $i_p = i_q$, then the corresponding argument list contains the same basis vector twice. Alternation makes that term zero:

$$F(\dots, \mathbf{e}_{i_p}, \dots, \mathbf{e}_{i_p}, \dots) = 0.$$

A term can therefore survive only when $i_1, \dots, i_n$ are all different. Because every i_j belongs to $\{1, \dots, n\}$, the surviving index list must contain every index exactly once. Thus, it has the form

$$(i_1, \dots, i_n) = (\sigma(1), \dots, \sigma(n))$$

for some permutation $\sigma \in S_n$.

Alternation determines the value of every surviving basis term relative to the ordered basis term. Reordering $(\mathbf{e}_{\sigma(1)}, \dots, \mathbf{e}_{\sigma(n)})$ into $(\mathbf{e}_1, \dots, \mathbf{e}_n)$ requires an even or odd number of swaps, so

$$F(\mathbf{e}_{\sigma(1)}, \dots, \mathbf{e}_{\sigma(n)}) = \text{sgn}(\sigma) F(\mathbf{e}_1, \dots, \mathbf{e}_n).$$

Substituting this result into the expansion for $F(\mathbf{B})$ leaves only the permutation terms:

$$F(\mathbf{B}) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{j=1}^n b_{\sigma(j), j} F(\mathbf{e}_1, \dots, \mathbf{e}_n)$$

The scalar appearing in the claim is

$$c := F(\mathbf{e}_1, \dots, \mathbf{e}_n).$$

The columns of $\mathbf{I}_n$ are the ordered standard basis vectors, so the same reference value can be written as

$$c = F(\mathbf{e}_1, \dots, \mathbf{e}_n) = F(\mathbf{I}_n).$$

Substituting the surviving terms into the expansion gives

$$\begin{aligned} F(\mathbf{B}) &= c \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{j=1}^n b_{\sigma(j), j} \\ &= c \det(\mathbf{B}). \end{aligned}$$

A two-dimensional example makes the one-free-value idea explicit. In this example, $F(\mathbf{u}, \mathbf{v})$ is shorthand for $F([\mathbf{u} \ \mathbf{v}])$, where $\mathbf{u}$ and $\mathbf{v}$ are the two columns of the input matrix. Let $\mathbf{u} = u_1 \mathbf{e}_1 + u_2 \mathbf{e}_2$ and $\mathbf{v} = v_1 \mathbf{e}_1 + v_2 \mathbf{e}_2$. Using multilinearity in both columns gives

$$F(\mathbf{u}, \mathbf{v}) = u_1 v_1 F(\mathbf{e}_1, \mathbf{e}_1) + u_1 v_2 F(\mathbf{e}_1, \mathbf{e}_2) \\ + u_2 v_1 F(\mathbf{e}_2, \mathbf{e}_1) + u_2 v_2 F(\mathbf{e}_2, \mathbf{e}_2).$$

The repeated-basis terms are zero, $F(\mathbf{e}_1, \mathbf{e}_2) = c$, and $F(\mathbf{e}_2, \mathbf{e}_1) = -c$. Therefore,

$$F(\mathbf{u}, \mathbf{v}) = u_1 v_2 c - u_2 v_1 c \\ = c(u_1 v_2 - u_2 v_1) \\ = c \det[\mathbf{u} \ \mathbf{v}].$$

After c is known, this equation fixes $F(\mathbf{u}, \mathbf{v})$ for every pair $\mathbf{u}, \mathbf{v} \in \mathbb{R}^2$.

For the particular function $F(\mathbf{B}) = \det(\mathbf{AB})$,

$$c = F(\mathbf{I}_n) \\ = \det(\mathbf{A}\mathbf{I}_n) \\ = \det(\mathbf{A}).$$

Consequently,

$$\boxed{\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})}.$$

3.8.3.1 Why transposition does not change the determinant

The related identity

$$\boxed{\det(\mathbf{A}^\top) = \det(\mathbf{A})}$$

does not follow from multiplicativity alone. It follows directly from the permutation definition of the determinant.

Let $\mathbf{A} = [a_{ij}] \in \mathbb{R}^{n \times n}$. Because $(\mathbf{A}^\top)_{ij} = a_{ji}$,

$$\det(\mathbf{A}^\top) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{j=1}^n (\mathbf{A}^\top)_{\sigma(j), j} \\ = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{j=1}^n a_{j, \sigma(j)}.$$

A permutation $\sigma \in S_n$ is a bijection from the index set $\{1, \dots, n\}$ to itself. Because it is bijective, it has a unique **inverse permutation**, denoted by σ^{-1} , that reverses the mapping performed by σ . Define

$$\tau := \sigma^{-1}.$$

The defining identities are

$$\tau(\sigma(j)) = j, \quad \sigma(\tau(k)) = k.$$

Equivalently,

$$k = \sigma(j) \iff j = \tau(k).$$

For example, suppose $n = 3$ and

$$\sigma(1) = 2, \quad \sigma(2) = 3, \quad \sigma(3) = 1.$$

The inverse permutation reverses these assignments:

$$\tau(1) = 3, \quad \tau(2) = 1, \quad \tau(3) = 2.$$

For instance, $\sigma(1) = 2$ is reversed by $\tau(2) = 1$.

As σ runs through all permutations in S_n , $\tau = \sigma^{-1}$ also runs through all permutations in S_n . This is because inversion pairs every σ with exactly one τ , and the pairing is reversible: $\sigma = \tau^{-1}$. Therefore, replacing the summation index σ by τ neither loses nor duplicates a permutation.

The inverse permutation also has the same sign as the original permutation. Suppose σ can be produced by r pairwise swaps. To undo σ , perform those same r swaps in reverse order. Thus, σ^{-1} uses the same number of swaps as σ . They therefore have the same even-or-odd parity, so

$$\text{sgn}(\tau) = \text{sgn}(\sigma^{-1}) = \text{sgn}(\sigma).$$

Now consider the product

$$\prod_{j=1}^n a_{j, \sigma(j)}.$$

Introduce the new index

$$k := \sigma(j).$$

Because σ is a permutation, as j visits $1, \dots, n$, the index $k = \sigma(j)$ also visits every value $1, \dots, n$ exactly once. From $\tau = \sigma^{-1}$, the relation $k = \sigma(j)$ implies $j = \tau(k)$. Each factor can therefore be rewritten as

$$a_{j, \sigma(j)} = a_{\tau(k), k}.$$

Consequently,

$$\prod_{j=1}^n a_{j, \sigma(j)} = \prod_{k=1}^n a_{\tau(k), k}.$$

The two products contain the same scalar factors, possibly in a different order. Scalar multiplication is commutative, so their order does not affect the product. In the three-dimensional example above,

$$\prod_{j=1}^3 a_{j,\sigma(j)} = a_{1,2}a_{2,3}a_{3,1},$$

$$\prod_{k=1}^3 a_{\tau(k),k} = a_{3,1}a_{1,2}a_{2,3}.$$

These products are equal because they contain exactly the same three factors.

Therefore,

$$\begin{aligned}\det(\mathbf{A}^\top) &= \sum_{\tau \in S_n} \operatorname{sgn}(\tau) \prod_{k=1}^n a_{\tau(k),k} \\ &= \det(\mathbf{A}).\end{aligned}$$

Thus, exchanging the rows and columns changes how the same determinant terms are indexed but does not change their values or signs.

3.8.3.2 Determinant of an inverse

For nonsingular $\mathbf{A}$, multiplicativity does give the inverse identity. Taking determinants of $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n$ yields

$$\begin{aligned}1 &= \det(\mathbf{I}_n) \\ &= \det(\mathbf{A}^{-1}\mathbf{A}) \\ &= \det(\mathbf{A}^{-1})\det(\mathbf{A}).\end{aligned}$$

Since $\det(\mathbf{A}) \neq 0$,

$$\boxed{\det(\mathbf{A}^{-1}) = \frac{1}{\det(\mathbf{A})}}.$$

3.9 Quick test C: determinants

1. **Recall:** What does $|\det(\mathbf{A})|$ measure geometrically?
2. **Explanation:** What does the sign of a determinant record?
3. **Derivation:** Explain why repeated columns force the determinant to be zero.
4. **Application:** Compute $\det \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}$ and interpret its magnitude.
5. **Limitations:** Why is the determinant not defined in the same way for a rectangular matrix?
6. **Composition:** Explain geometrically why $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$.
7. **Reindexing:** If $\sigma(1) = 2$, $\sigma(2) = 3$, and $\sigma(3) = 1$, find $\tau = \sigma^{-1}$ and verify $\prod_{j=1}^3 a_{j,\sigma(j)} = \prod_{k=1}^3 a_{\tau(k),k}$.

3.9.1 Answers and hints

1. The factor by which the associated linear map scales n -dimensional volume.
2. A positive determinant preserves the ordered right-handed orientation; a negative determinant reverses it to a left-handed orientation and therefore indicates a reflection component; a zero determinant collapses the volume, so no full-dimensional orientation remains.
3. Swapping identical columns leaves the matrix unchanged but must negate the determinant, so the determinant equals its own negative and is zero.
4. The determinant is $2(4) - 1(3) = 5$, so areas are scaled by a factor of 5 and orientation is preserved.
5. A rectangular map changes dimension, so it does not map an n -dimensional volume to another volume in the same n -dimensional space.
6. $\mathbf{B}$ scales signed volume first and $\mathbf{A}$ scales the resulting signed volume second, so the factors multiply.
7. The inverse is $\tau(1) = 3$, $\tau(2) = 1$, and $\tau(3) = 2$. The left product is $a_{1,2}a_{2,3}a_{3,1}$, while the reindexed product is $a_{3,1}a_{1,2}a_{2,3}$; these are equal because scalar factors commute.

3.10 4. Orthogonal matrices

Let $\mathbf{Q} \in \mathbb{R}^{n \times n}$. The matrix $\mathbf{Q}$ is **orthogonal** if

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n.$$

The entries of $\mathbf{Q}^T \mathbf{Q}$ are pairwise dot products between columns of $\mathbf{Q}$. Therefore, this equation is equivalent to saying that the columns form an orthonormal basis of $\mathbb{R}^n$.

Because $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$, the transpose satisfies the defining equation for the inverse:

$$\boxed{\mathbf{Q}^{-1} = \mathbf{Q}^T}.$$

For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$,

$$(\mathbf{Q}\mathbf{x})^T (\mathbf{Q}\mathbf{y}) = \mathbf{x}^T \mathbf{Q}^T \mathbf{Q} \mathbf{y} = \mathbf{x}^T \mathbf{y}.$$

Thus, orthogonal matrices preserve dot products, lengths, and angles.

Taking determinants of $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$ gives

$$\det(\mathbf{Q})^2 = 1,$$

so

$$\det(\mathbf{Q}) = +1 \quad \text{or} \quad \det(\mathbf{Q}) = -1.$$

The **orthogonal group** is

$$O(n) := \{\mathbf{Q} \in \mathbb{R}^{n \times n} : \mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n\}.$$

The **special orthogonal group** is

$$SO(n) := \{\mathbf{Q} \in O(n) : \det(\mathbf{Q}) = 1\}.$$

Matrices in $SO(n)$ preserve lengths, angles, and orientation. Orthogonal matrices with determinant -1 preserve lengths and angles but reverse orientation through a reflection. Rotation matrices used for right-handed coordinate frames belong to $SO(2)$ or $SO(3)$.

3.11 Quick test D: orthogonal matrices

1. **Recall:** What equation defines an orthogonal matrix?
2. **Explanation:** Why is the inverse of an orthogonal matrix its transpose?
3. **Derivation:** Show that $\|\mathbf{Q}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$.
4. **Application:** Determine whether $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ belongs to $SO(2)$.
5. **Limitations:** Why is $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$ insufficient by itself to identify a proper rotation?

3.11.1 Answers and hints

1. $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$.
2. The transpose already satisfies the defining inverse equation.
3. Square both norms and use $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$.
4. It is orthogonal but has determinant -1 , so it is a reflection in $O(2)$ and does not belong to $SO(2)$.
5. Orthogonality permits determinant -1 reflections as well as determinant $+1$ rotations.

3.12 Cumulative chapter test

1. Define a basis and explain why coordinate columns are unique in a basis.
2. Given $\mathbf{A} \in \mathbb{R}^{3 \times 2}$ and $\mathbf{x} \in \mathbb{R}^2$, state the dimension of $\mathbf{A}\mathbf{x}$ and interpret it using the columns of $\mathbf{A}$.
3. Explain the relationship between $\mathcal{N}(\mathbf{A})$, independent columns, and full column rank.
4. Prove that a nonsingular square matrix cannot map a nonzero vector to zero.
5. Define the determinant using multilinearity, alternation, and normalisation.
6. Explain why $\det(\mathbf{A}) = 0$ indicates lost information.
7. Derive $\det(\mathbf{A}^{-1}) = 1/\det(\mathbf{A})$.
8. Distinguish $O(3)$ from $SO(3)$ and explain which set contains rigid-body rotation matrices.

3.12.1 Cumulative answers and hints

1. A basis is a linearly independent spanning set. Independence prevents multiple coefficient descriptions, while spanning guarantees that a description exists.
2. $\mathbf{Ax} \in \mathbb{R}^3$ and equals $x_1\mathbf{a}_1 + x_2\mathbf{a}_2$.
3. $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$ exactly when the columns are independent; for n columns this gives $\text{rank}(\mathbf{A}) = n$.
4. Nonsingularity is equivalent to $\mathbf{Ax} = \mathbf{0} \Rightarrow \mathbf{x} = \mathbf{0}$; the contrapositive is $\mathbf{x} \neq \mathbf{0} \Rightarrow \mathbf{Ax} \neq \mathbf{0}$.
5. The determinant is the unique scalar function that is linear in each column, changes sign under a column swap, and equals 1 on $\mathbf{I}_n$.
6. For $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\det(\mathbf{A}) = 0$ means that $\mathbf{A}$ is singular, so its null space contains a nonzero vector $\mathbf{z} \in \mathbb{R}^n$ satisfying $\mathbf{Az} = \mathbf{0}$. For any $\mathbf{x} \in \mathbb{R}^n$,

$$\mathbf{A}(\mathbf{x} + \mathbf{z}) = \mathbf{Ax} + \mathbf{Az} = \mathbf{Ax}.$$

Therefore, the distinct inputs $\mathbf{x}$ and $\mathbf{x} + \mathbf{z}$ produce the same output. The output cannot reveal the input component in the null-space direction $\mathbf{z}$, so the input cannot be uniquely recovered. This is the meaning of **lost information** here: an independent input direction or degree of freedom has disappeared, not that Shannon information has been calculated. Geometrically, the map collapses n -dimensional volume into a lower-dimensional set, which is why its volume-scaling factor is zero.

7. Take determinants of $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n$ and use multiplicativity.
8. $O(3)$ contains all orthogonal 3×3 matrices with determinant ± 1 ; $SO(3)$ contains the determinant-1 members and therefore the proper rigid-body rotations.

3.13 References

- Richard Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Appendix A, especially the discussion of orthonormal matrices, rank, singular values, and matrix factorisations.
- Kevin M. Lynch and Frank C. Park, *Modern Robotics*, Chapter 3, especially Section 3.2.1 on rotation matrices and the distinction between orthogonality and right-handed orientation.

4 Geometry and Coordinate Frames

4.1 Why coordinate frames matter

A robot represents positions, directions, velocities, measurements, and commands with numbers. Those numbers acquire physical meaning only when their coordinate frame and units are known. A controller that combines quantities expressed in incompatible frames can perform a mathematically valid calculation and produce a physically incorrect command.

Coordinate-frame methods allow us to:

- represent a robot's pose relative to a world frame;
- distinguish world-frame and body-frame quantities;
- transform sensor measurements into a shared frame;
- express target and tracking errors consistently;
- convert body-relative motion into world-relative motion; and
- detect invalid transformations before they reach a controller.

The central rule is simple: interpret every coordinate column together with the frame in which it is expressed.

4.2 Prerequisites and assumptions

4.2.1 Prerequisites

The reader should understand the material in the [linear algebra foundations chapter](#), especially:

- geometric vectors and coordinate columns;
- dot products, norms, and orthogonality;
- matrices as linear maps and compositions;
- linear independence, rank, and inverses;
- determinants and signed orientation; and
- orthogonal matrices.

The reader should also be comfortable with sine, cosine, and angles measured in radians.

4.2.2 Assumptions

- Physical space is modelled as Euclidean space.
- Robot links and sensor mounts are treated as rigid bodies.
- Coordinate frames are right-handed and use orthonormal basis vectors.
- Length is measured in metres, time in seconds, and angles in radians.
- Coordinate vectors are written as columns.
- Numerical examples ignore measurement noise unless stated otherwise.

These idealisations isolate the geometry. A physical robot also introduces measurement noise, calibration error, structural compliance, and model mismatch.

4.3 Dependency map

```
Linear algebra foundations
    |
    v
Cross products and rigid-body velocity
    |
    v
Coordinate frames and units
    |
    v
Rotation matrices
    |
    v
Point transformations
    |
    v
Differential-drive motion and feedback control
```

4.4 1. Cross product and rigid-body velocity

Let $\mathbf{u}$ and $\mathbf{v}$ be geometric vectors with coordinate columns ${}^a\mathbf{u}, {}^a\mathbf{v} \in \mathbb{R}^3$ expressed in the same right-handed orthonormal frame $\{a\}$. Their cross product, expressed in frame $\{a\}$, is defined by

$${}^a\mathbf{u} \times {}^a\mathbf{v} = \begin{bmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{bmatrix} \in \mathbb{R}^3.$$

Its direction follows the right-hand rule. If both vectors are nonzero, let $\theta \in [0, \pi]$ be the smaller angle between them. The cross-product magnitude is

$$\|{}^a\mathbf{u} \times {}^a\mathbf{v}\|_2 = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \sin \theta.$$

The cross product is perpendicular to both input vectors because

$${}^a\mathbf{u}^\top ({}^a\mathbf{u} \times {}^a\mathbf{v}) = 0, \quad {}^a\mathbf{v}^\top ({}^a\mathbf{u} \times {}^a\mathbf{v}) = 0.$$

4.4.1 1.1 Example: velocity of a point on a rotating rigid body

Let O and P be two points fixed to the same rigid body. Point O is the chosen reference point; it need not be stationary, and it need not be the body's centre. The vector ${}^a\mathbf{r}_{OP}$ runs from O to P . Although this vector is fixed relative to the body, its coordinates generally change as the body rotates. Express every quantity in the same orthonormal frame $\{a\}$:

- ${}^a\boldsymbol{\omega}$ as the body's angular-velocity vector, measured in rad s^{-1} ;
- ${}^a\mathbf{r}_{OP}$ as the position of P relative to O , measured in m;
- ${}^a\mathbf{v}_O$ as the linear velocity of O , measured in m s^{-1} ; and
- ${}^a\mathbf{v}_P$ as the linear velocity of P , measured in m s^{-1} .

At any instant, the motion of P can be understood as the sum of two simultaneous contributions:

1. **Translation with O .** If the body were not rotating, P would share the reference-point velocity ${}^a\mathbf{v}_O$.
2. **Motion relative to O caused by rotation.** Rotation adds a velocity whose direction and magnitude depend on ${}^a\boldsymbol{\omega}$ and ${}^a\mathbf{r}_{OP}$.

The angular-velocity vector points along the instantaneous rotation axis according to the right-hand rule, and its magnitude is the angular speed. Define the rotational contribution to the velocity of P by

$${}^a\mathbf{v}_{P/O} := {}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP}.$$

The cross product determines both the direction and the magnitude of this contribution. In particular,

$${}^a\boldsymbol{\omega}^\top {}^a\mathbf{v}_{P/O} = 0, \quad {}^a\mathbf{r}_{OP}^\top {}^a\mathbf{v}_{P/O} = 0.$$

Therefore, ${}^a\mathbf{v}_{P/O}$ is perpendicular to the rotation axis and to ${}^a\mathbf{r}_{OP}$. It points tangent to the instantaneous circular path generated by the rotation. Adding the translational and rotational contributions gives the rigid-body velocity relation

$$\boxed{{}^a\mathbf{v}_P = {}^a\mathbf{v}_O + {}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP}}.$$

4.4.1.1 Velocity-vector construction

The construction in Figure 4.1 shows this addition for one instant:

- The green arrow ${}^a\mathbf{r}_{OP}$ locates P relative to O .
- The magenta arrow is the reference-point velocity ${}^a\mathbf{v}_O$. It is deliberately not parallel to ${}^a\mathbf{r}_{OP}$.
- The orange arrow is the rotational contribution ${}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP} = {}^a\mathbf{v}_{P/O}$. The right-angle marker shows that it is perpendicular to ${}^a\mathbf{r}_{OP}$.
- The dashed arrows are translated copies of the two velocity contributions. Moving a vector without rotating or resizing it does not change the vector, so the copies can be arranged head to tail.
- The blue arrow joins the beginning of the first contribution to the end of the second. It is therefore the resultant velocity ${}^a\mathbf{v}_P$.

In this planar view, ${}^a\boldsymbol{\omega}$ points along $+z_a$, out of the page. The right-hand rule then gives the displayed tangential direction for ${}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP}$. Notice that only the rotational contribution must be perpendicular to ${}^a\mathbf{r}_{OP}$. The total velocity ${}^a\mathbf{v}_P$ generally is not, because ${}^a\mathbf{v}_O$ may point in any direction.

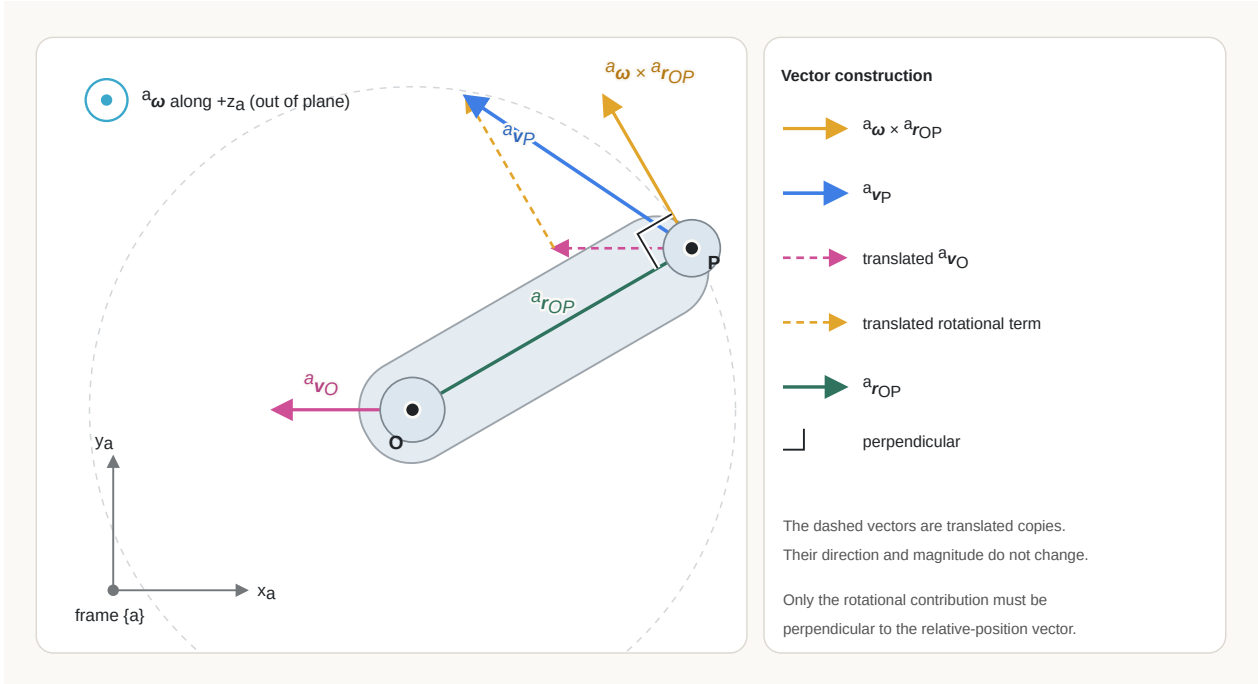


Figure 4.1: Velocity construction for two points on a rigid body. The rotational contribution is perpendicular to the position vector, while the reference-point velocity is independent of that perpendicular relationship.

For pure rotation about a fixed axis through O , the reference point is stationary, so ${}^a\mathbf{v}_O = \mathbf{0}$. Therefore,

$$\boxed{{}^a\mathbf{v}_P = {}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP}}.$$

If ϕ is the angle between ${}^a\boldsymbol{\omega}$ and ${}^a\mathbf{r}_{OP}$, then the relative speed caused by rotation is

$$\|{}^a\mathbf{v}_{P/O}\|_2 = \|{}^a\boldsymbol{\omega}\|_2 \|{}^a\mathbf{r}_{OP}\|_2 \sin \phi = \omega r_\perp,$$

where $r_\perp$ is the perpendicular distance from P to the rotation axis. A point on the axis has $r_\perp = 0$ and therefore has no relative linear velocity due to rotation. Under the pure-rotation assumption, ${}^a\mathbf{v}_P = {}^a\mathbf{v}_{P/O}$.

For planar counter-clockwise rotation about the positive z -axis,

$${}^a\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix}, \quad {}^a\mathbf{r}_{OP} = \begin{bmatrix} x \\ y \\ 0 \end{bmatrix}.$$

The point velocity is

$${}^a\mathbf{v}_P = {}^a\boldsymbol{\omega} \times {}^a\mathbf{r}_{OP} = \begin{bmatrix} -\omega y \\ \omega x \\ 0 \end{bmatrix}.$$

For example, if $\omega = 2 \text{ rad s}^{-1}$ and ${}^a\mathbf{r}_{OP} = [0.5 \ 0 \ 0]^\text{T} \text{ m}$, then

$${}^a\mathbf{v}_P = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

The point lies on the positive x -axis, while its instantaneous velocity points along the positive y -axis, tangent to its circular path.

4.5 Quick test A: cross products and rigid-body velocity

Complete this test without looking back at the section.

1. **Recall:** What rule determines the direction of $\mathbf{u} \times \mathbf{v}$?
2. **Explanation:** Why is $\mathbf{u} \times \mathbf{v}$ perpendicular to both input vectors?
3. **Derivation:** Substitute the component formula for ${}^a\mathbf{u} \times {}^a\mathbf{v}$ into ${}^a\mathbf{u}^\text{T}({}^a\mathbf{u} \times {}^a\mathbf{v})$ and show that the terms cancel.
4. **Application:** Compute $[1 \ 2 \ 0]^\text{T} \times [0 \ 1 \ 0]^\text{T}$.
5. **Limitations:** What happens to the cross product when the input vectors are parallel, and why does that prevent it from defining a unique normal direction?
6. **Transfer:** At one instant, a rigid body has ${}^a\boldsymbol{\omega} = [0 \ 0 \ 2]^\text{T} \text{ rad s}^{-1}$, ${}^a\mathbf{r}_{OP} = [1 \ 0 \ 0]^\text{T} \text{ m}$, and ${}^a\mathbf{v}_O = [0.5 \ 0.5 \ 0]^\text{T} \text{ m s}^{-1}$. Find ${}^a\mathbf{v}_{P/O}$ and ${}^a\mathbf{v}_P$. Which velocity is perpendicular to ${}^a\mathbf{r}_{OP}$?

4.5.1 Answers and hints

1. The right-hand rule, applied in the order from $\mathbf{u}$ to $\mathbf{v}$.
2. Its dot product with either input is zero.
3. Expanding gives $u_x(u_y v_z - u_z v_y) + u_y(u_z v_x - u_x v_z) + u_z(u_x v_y - u_y v_x) = 0$ after pairwise cancellation.
4. The result is $[0 \ 0 \ 1]^\top$.
5. Parallel vectors have $\sin \theta = 0$, so their cross product is the zero vector. A zero vector has no direction and therefore cannot specify a unique normal.
6. ${}^a\mathbf{v}_{P/O} = [0 \ 2 \ 0]^\top \text{ m s}^{-1}$ and ${}^a\mathbf{v}_P = [0.5 \ 2.5 \ 0]^\top \text{ m s}^{-1}$. The rotational contribution is perpendicular to ${}^a\mathbf{r}_{OP}$; the total velocity is not.

4.6 2. Coordinate frames

4.6.1 2.1 What defines a frame?

A three-dimensional coordinate frame $\{a\}$ consists of:

1. an origin O_a , which is the physical point from which positions in frame $\{a\}$ are measured; and
2. three right-handed orthonormal basis vectors $\mathbf{e}_{x,a}$, $\mathbf{e}_{y,a}$, and $\mathbf{e}_{z,a}$, which define the positive x_a -, y_a -, and z_a -axis directions.

The subscript a associates the origin and basis vectors with frame $\{a\}$. Orthonormal means that each basis vector has unit length and that distinct basis vectors are perpendicular. Right-handed means that $\mathbf{e}_{x,a} \times \mathbf{e}_{y,a} = \mathbf{e}_{z,a}$.

A free vector depends only on the basis orientation used to describe it. A point also depends on the frame origin. This distinction explains why changing the coordinate frame of a free vector requires only a rotation, whereas changing the coordinates of a point generally requires a rotation and a translation.

4.6.2 2.2 Orientation convention

Let $\{a\}$ and $\{b\}$ be three-dimensional right-handed orthonormal frames. Define $\mathbf{R}_{ab} \in \mathbb{R}^{3 \times 3}$ as the rotation matrix that maps coordinate columns from frame $\{b\}$ to frame $\{a\}$. Its columns are the basis vectors of $\{b\}$ expressed in frame $\{a\}$:

$$\mathbf{R}_{ab} = \begin{bmatrix} {}^a\mathbf{e}_{x,b} & {}^a\mathbf{e}_{y,b} & {}^a\mathbf{e}_{z,b} \end{bmatrix},$$

where each column ${}^a\mathbf{e}_{i,b} \in \mathbb{R}^3$ is an axis of frame $\{b\}$ written using the basis of frame $\{a\}$, for $i \in \{x, y, z\}$.

The following notation distinguishes a geometric vector from its coordinate representations:

Symbol	Meaning
$\mathbf{v}$	A geometric vector, independent of the coordinate frame used to describe it.

Symbol	Meaning
$\mathbf{e}_{x,b}$	The geometric unit vector along the positive x_b -axis of frame $\{b\}$.
$[\mathbf{v}]_b = {}^b\mathbf{v} \in \mathbb{R}^3$	The numerical coordinate column of $\mathbf{v}$ in frame $\{b\}$.
$[\mathbf{v}]_a = {}^a\mathbf{v} \in \mathbb{R}^3$	The numerical coordinate column of $\mathbf{v}$ in frame $\{a\}$.
$[\mathbf{e}_{x,b}]_a = {}^a\mathbf{e}_{x,b} \in \mathbb{R}^3$	The coordinate column of frame $\{b\}$'s geometric x -axis, expressed in frame $\{a\}$.

The bracket operator $[\cdot]_a$ means “take the coordinate column of the enclosed geometric vector in frame $\{a\}$.” The geometric vector $\mathbf{v}$ can be expanded using either frame’s basis:

$$\mathbf{v} = v_x^{(b)} \mathbf{e}_{x,b} + v_y^{(b)} \mathbf{e}_{y,b} + v_z^{(b)} \mathbf{e}_{z,b} = v_x^{(a)} \mathbf{e}_{x,a} + v_y^{(a)} \mathbf{e}_{y,a} + v_z^{(a)} \mathbf{e}_{z,a}.$$

Here, $v_i^{(b)} \in \mathbb{R}$ and $v_i^{(a)} \in \mathbb{R}$ are the scalar components of $\mathbf{v}$ along axis $i \in \{x, y, z\}$ of frames $\{b\}$ and $\{a\}$, respectively. The corresponding coordinate columns are

$${}^b\mathbf{v} = [\mathbf{v}]_b = \begin{bmatrix} v_x^{(b)} \\ v_y^{(b)} \\ v_z^{(b)} \end{bmatrix}, \quad {}^a\mathbf{v} = [\mathbf{v}]_a = \begin{bmatrix} v_x^{(a)} \\ v_y^{(a)} \\ v_z^{(a)} \end{bmatrix}.$$

Apply the coordinate operator $[\cdot]_a$ to the expansion of $\mathbf{v}$ in the frame- $\{b\}$ basis. Because taking coordinates is a linear operation,

$$[\mathbf{v}]_a = v_x^{(b)} [\mathbf{e}_{x,b}]_a + v_y^{(b)} [\mathbf{e}_{y,b}]_a + v_z^{(b)} [\mathbf{e}_{z,b}]_a.$$

Using the equivalent superscript notation gives

$${}^a\mathbf{v} = v_x^{(b)} {}^a\mathbf{e}_{x,b} + v_y^{(b)} {}^a\mathbf{e}_{y,b} + v_z^{(b)} {}^a\mathbf{e}_{z,b}.$$

Writing this linear combination as matrix-vector multiplication gives

$${}^a\mathbf{v} = \underbrace{\begin{bmatrix} {}^a\mathbf{e}_{x,b} & {}^a\mathbf{e}_{y,b} & {}^a\mathbf{e}_{z,b} \end{bmatrix}}_{\mathbf{R}_{ab}} \underbrace{\begin{bmatrix} v_x^{(b)} \\ v_y^{(b)} \\ v_z^{(b)} \end{bmatrix}}_{{}^b\mathbf{v}}.$$

Therefore,

$${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}.$$

The subscript order can be read as:

$\mathbf{R}_{ab}$ maps coordinates **from frame $\{b\}$ to frame $\{a\}$** .

The [linear algebra foundations chapter](#) develops orthogonal matrices, determinants, and inverses in general. Applying those results to the orthonormal, right-handed columns of $\mathbf{R}_{ab}$ gives:

$$\boxed{\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \mathbf{I}_3, \quad \det(\mathbf{R}_{ab}) = 1, \quad \mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^\top = \mathbf{R}_{ba}}.$$

The following derivations explain each property.

4.6.2.1 Orthonormal columns

Let $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3 \in \mathbb{R}^3$ denote the columns of $\mathbf{R}_{ab}$:

$$\mathbf{R}_{ab} = [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{r}_3],$$

where $\mathbf{r}_1 = {}^a\mathbf{e}_{x,b}$, $\mathbf{r}_2 = {}^a\mathbf{e}_{y,b}$, and $\mathbf{r}_3 = {}^a\mathbf{e}_{z,b}$. Multiplying the transpose by the original matrix forms every pairwise dot product between these columns:

$$\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \begin{bmatrix} \mathbf{r}_1^\top \mathbf{r}_1 & \mathbf{r}_1^\top \mathbf{r}_2 & \mathbf{r}_1^\top \mathbf{r}_3 \\ \mathbf{r}_2^\top \mathbf{r}_1 & \mathbf{r}_2^\top \mathbf{r}_2 & \mathbf{r}_2^\top \mathbf{r}_3 \\ \mathbf{r}_3^\top \mathbf{r}_1 & \mathbf{r}_3^\top \mathbf{r}_2 & \mathbf{r}_3^\top \mathbf{r}_3 \end{bmatrix}.$$

Because the frame axes are orthonormal, every column has unit length,

$$\mathbf{r}_i^\top \mathbf{r}_i = 1,$$

and distinct columns are perpendicular,

$$\mathbf{r}_i^\top \mathbf{r}_j = 0 \quad \text{for } i \neq j,$$

where $i, j \in \{1, 2, 3\}$ are column indices. Therefore,

$$\boxed{\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \mathbf{I}_3}.$$

This identity means that $\mathbf{R}_{ab}$ preserves dot products and therefore preserves vector lengths and angles.

4.6.2.2 Inverse and reverse coordinate mapping

The inverse $\mathbf{R}_{ab}^{-1} \in \mathbb{R}^{3 \times 3}$ is defined by

$$\mathbf{R}_{ab}^{-1} \mathbf{R}_{ab} = \mathbf{I}_3.$$

Because $\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \mathbf{I}_3$, the transpose satisfies the defining property of the inverse. Hence,

$$\boxed{\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^\top}.$$

The coordinate transformation from frame $\{b\}$ to frame $\{a\}$ is

$${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}.$$

Multiplying by the inverse gives

$${}^b\mathbf{v} = \mathbf{R}_{ab}^{-1} {}^a\mathbf{v}.$$

Define $\mathbf{R}_{ba} \in \mathbb{R}^{3 \times 3}$ as the matrix that maps coordinates in the reverse direction, from frame $\{a\}$ to frame $\{b\}$. By definition,

$${}^b\mathbf{v} = \mathbf{R}_{ba} {}^a\mathbf{v}.$$

The two reverse mappings must be the same, so

$$\boxed{\mathbf{R}_{ba} = \mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^\top}.$$

4.6.2.3 Determinant and orientation

The notation $\det(\mathbf{R})$ denotes the **determinant** of a square matrix $\mathbf{R}$. Geometrically, $|\det(\mathbf{R})|$ is the volume-scaling factor of the corresponding linear transformation. For an orthogonal matrix $\mathbf{R}$, taking determinants of $\mathbf{R}^\top \mathbf{R} = \mathbf{I}_3$ gives

$$\det(\mathbf{R}^\top \mathbf{R}) = \det(\mathbf{I}_3),$$

and therefore by $\det(\mathbf{R}^\top \mathbf{R}) = \det(\mathbf{R}^\top) \det(\mathbf{R})$ and $\det(\mathbf{R}^\top) = \det(\mathbf{R})$ gives

$$\det(\mathbf{R})^2 = 1.$$

Thus, an orthogonal matrix can have determinant $+1$ or -1 . Here, **orientation** means the handedness of the ordered coordinate axes. A right-handed frame satisfies

$$\mathbf{e}_x \times \mathbf{e}_y = \mathbf{e}_z.$$

The determinant sign has the following interpretation:

- $\det(\mathbf{R}) = +1$ preserves handedness, so a right-handed ordered basis remains right-handed.
- $\det(\mathbf{R}) = -1$ reverses handedness, so the transformation contains a reflection and produces a left-handed ordered basis from a right-handed one.

For example, consider

$$\mathbf{M} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The matrix $\mathbf{M} \in \mathbb{R}^{3 \times 3}$ reflects the x -coordinate. Its columns remain orthonormal, so $\mathbf{M}^\top \mathbf{M} = \mathbf{I}_3$, but $\det(\mathbf{M}) = -1$. It reverses handedness and is therefore a reflection rather than a rigid-body rotation.

Because frames $\{a\}$ and $\{b\}$ are both right-handed, their rotation matrix must satisfy

$$\boxed{\mathbf{R}_{ab}^\top \mathbf{R}_{ab} = \mathbf{I}_3 \quad \text{and} \quad \det(\mathbf{R}_{ab}) = 1}.$$

The determinant sign does not indicate whether a rotation is clockwise or counter-clockwise. It indicates whether the transformation preserves or reverses the handedness of space.

4.6.3 2.3 Points require translation

Let O_a and O_b be the origins of frames $\{a\}$ and $\{b\}$. Define ${}^a\mathbf{p}_b \in \mathbb{R}^3$ as the position of O_b relative to O_a , expressed in frame $\{a\}$ and measured in metres.

For a physical point P , let ${}^b\mathbf{p}_P \in \mathbb{R}^3$ be the position of P relative to O_b , expressed in frame $\{b\}$, and let ${}^a\mathbf{p}_P \in \mathbb{R}^3$ be the position of P relative to O_a , expressed in frame $\{a\}$. Both position columns are measured in metres. Their relationship is

$${}^a\mathbf{p}_P = \mathbf{R}_{ab} {}^b\mathbf{p}_P + {}^a\mathbf{p}_b.$$

The rotation $\mathbf{R}_{ab} {}^b\mathbf{p}_P$ changes the basis used to express the displacement from O_b to P . The translation ${}^a\mathbf{p}_b$ then accounts for the displacement from O_a to O_b . A homogeneous transformation combines these two operations into one matrix equation.

4.7 3. Worked frame-change example

Consider two planar frames $\{a\}$ and $\{b\}$ whose axes differ by an angle $\theta \in \mathbb{R}$, measured in radians. A positive θ means that frame $\{b\}$ is rotated counter-clockwise relative to frame $\{a\}$. Define $\mathbf{R}_{ab}(\theta) \in \mathbb{R}^{2 \times 2}$ as the rotation matrix that maps two-dimensional coordinate columns from frame $\{b\}$ to frame $\{a\}$:

$${}^a\mathbf{v} = \mathbf{R}_{ab}(\theta) {}^b\mathbf{v},$$

where $\mathbf{v}$ is a geometric free vector and ${}^a\mathbf{v}, {}^b\mathbf{v} \in \mathbb{R}^2$ are its coordinate columns in frames $\{a\}$ and $\{b\}$.

Let $\mathbf{e}_{x,a}$ and $\mathbf{e}_{y,a}$ be the geometric unit axes of frame $\{a\}$, and let $\mathbf{e}_{x,b}$ and $\mathbf{e}_{y,b}$ be the geometric unit axes of frame $\{b\}$. By definition, the columns of $\mathbf{R}_{ab}$ are the axes of frame $\{b\}$ expressed in frame $\{a\}$:

$$\mathbf{R}_{ab} = \begin{bmatrix} {}^a\mathbf{e}_{x,b} & {}^a\mathbf{e}_{y,b} \end{bmatrix}.$$

The x_b -axis makes angle θ with the x_a -axis. Its projection onto $\mathbf{e}_{x,a}$ is $\cos \theta$, and its projection onto $\mathbf{e}_{y,a}$ is $\sin \theta$. Therefore,

$${}^a\mathbf{e}_{x,b} = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}.$$

Because the planar frames are right-handed and orthonormal, the y_b -axis is obtained by rotating the x_b -axis counter-clockwise by $\pi/2$. Its angle measured from the x_a -axis is therefore $\theta + \pi/2$, so

$${}^a\mathbf{e}_{y,b} = \begin{bmatrix} \cos(\theta + \pi/2) \\ \sin(\theta + \pi/2) \end{bmatrix}.$$

Using

$$\cos(\theta + \pi/2) = -\sin \theta, \quad \sin(\theta + \pi/2) = \cos \theta,$$

gives

$${}^a\mathbf{e}_{y,b} = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}.$$

Placing these two coordinate columns side by side produces

$$\boxed{\mathbf{R}_{ab}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}}.$$

The column construction also explains why this matrix maps coordinates from frame $\{b\}$ to frame $\{a\}$. If

$${}^b\mathbf{v} = \begin{bmatrix} v_x^{(b)} \\ v_y^{(b)} \end{bmatrix},$$

where $v_x^{(b)}, v_y^{(b)} \in \mathbb{R}$ are the components of $\mathbf{v}$ along the x_b - and y_b -axes, then

$$\begin{aligned}
{}^a\mathbf{v} &= v_x^{(b)} {}^a\mathbf{e}_{x,b} + v_y^{(b)} {}^a\mathbf{e}_{y,b} \\
&= \begin{bmatrix} {}^a\mathbf{e}_{x,b} & {}^a\mathbf{e}_{y,b} \end{bmatrix} \begin{bmatrix} v_x^{(b)} \\ v_y^{(b)} \end{bmatrix} \\
&= \mathbf{R}_{ab}(\theta) {}^b\mathbf{v}.
\end{aligned}$$

For the worked example, $\theta = \pi/2$ rad. Substitution gives

$$\mathbf{R}_{ab}(\pi/2) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

The first column $[0 \ 1]^\top$ says that the positive x_b -axis points along the positive y_a -axis. The second column $[-1 \ 0]^\top$ says that the positive y_b -axis points along the negative x_a -axis.

Let ${}^b\mathbf{v} \in \mathbb{R}^2$ be the coordinate column of a free vector $\mathbf{v}$, expressed in frame $\{b\}$ and measured in metres:

$${}^b\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \text{ m.}$$

The coordinate column of the same geometric vector in frame $\{a\}$ is ${}^a\mathbf{v} \in \mathbb{R}^2$:

$${}^a\mathbf{v} = \mathbf{R}_{ab}(\theta) {}^b\mathbf{v} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix} \text{ m.}$$

The coordinates changed, but the physical length did not:

$$\|{}^b\mathbf{v}\|_2 = \sqrt{2^2 + 1^2} = \sqrt{5} = \sqrt{(-1)^2 + 2^2} = \|{}^a\mathbf{v}\|_2.$$

4.7.1 Interactive frame-rotation graph

Fix the frame- $\{b\}$ coordinate column

$${}^b\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

For each angle $\theta \in [-\pi, \pi]$, the corresponding frame- $\{a\}$ coordinate column is

$${}^a\mathbf{v}(\theta) = \mathbf{R}_{ab}(\theta) {}^b\mathbf{v} = \begin{bmatrix} 2 \cos \theta - \sin \theta \\ 2 \sin \theta + \cos \theta \end{bmatrix}.$$

In the HTML book, move the angle slider to rotate frame $\{b\}$ relative to frame $\{a\}$. The graph keeps ${}^b\mathbf{v} = [2 \ 1]^\top$ fixed, so the geometric vector rotates with frame $\{b\}$ and its coordinates ${}^a\mathbf{v}(\theta)$ update continuously.

The PDF shows the default slider state $\theta = \pi/2$ rad. Use the HTML book to vary θ interactively.

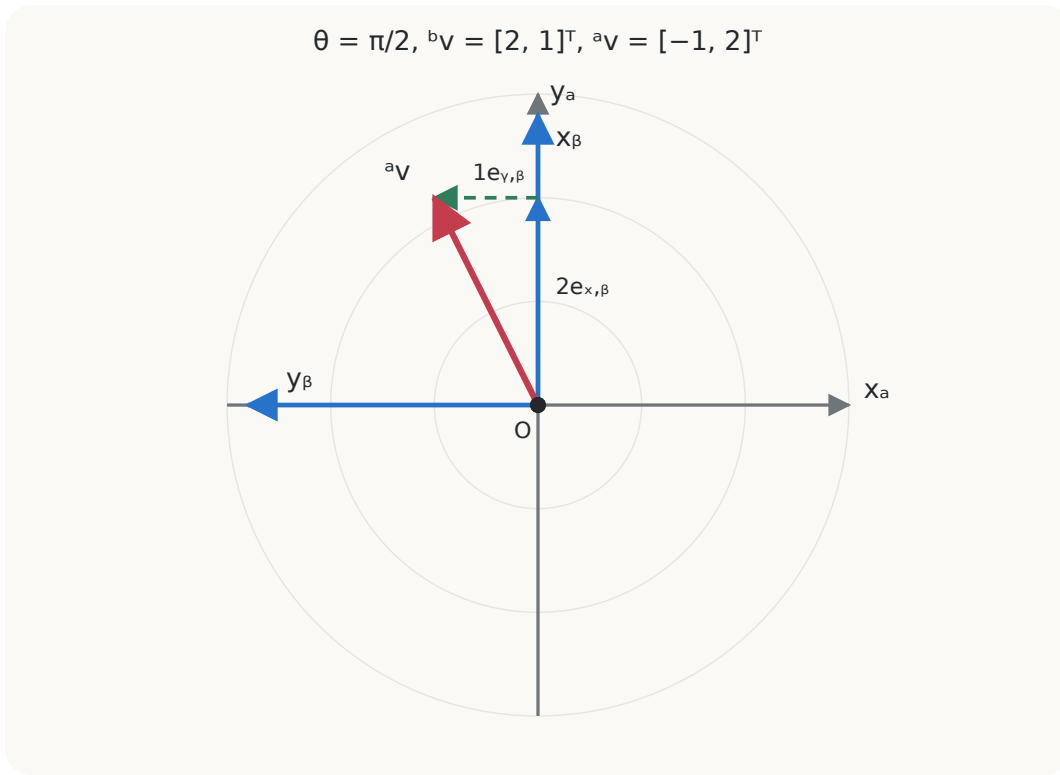


Figure 4.2: Static PDF view of the frame rotation at $\theta = \pi/2$ with ${}^b\mathbf{v} = [2 \ 1]^T$ and ${}^a\mathbf{v} = [-1 \ 2]^T$.

4.7.1.1 Equivalent NumPy calculation

The following NumPy snippet is the direct software translation of ${}^a\mathbf{v} = \mathbf{R}_{ab}(\theta){}^b\mathbf{v}$. The scalar `theta` stores θ in radians; `R_ab` is a NumPy array with shape $(2, 2)$ representing $\mathbf{R}_{ab}(\theta) \in \mathbb{R}^{2 \times 2}$; `v_b` is a NumPy array with shape $(2,)$ representing the fixed coordinate column ${}^b\mathbf{v} = [2 \ 1]^T$; and `v_a` is a NumPy array with shape $(2,)$ representing ${}^a\mathbf{v} \in \mathbb{R}^2$. The operator `@` performs matrix-vector multiplication.

```
import numpy as np

theta = np.pi / 2 # radians; replace this value with the selected angle

R_ab = np.array([
    [np.cos(theta), -np.sin(theta)],
    [np.sin(theta),  np.cos(theta)],
])

v_b = np.array([2.0, 1.0])
v_a = R_ab @ v_b
```

For $\theta = \pi/2$ rad, the calculation gives `v_a` approximately equal to `[-1.0, 2.0]`. Changing `theta` recomputes the same values displayed by the interactive graph while `v_b` remains fixed.

4.8 4. Differential-drive interpretation

For a planar mobile robot, let frame $\{w\}$ be fixed to the world and frame $\{b\}$ be attached to the robot. Let $\theta \in \mathbb{R}$ be the robot's heading angle, measured counter-clockwise in radians from the positive x_w -axis to the positive x_b -axis. Let $v \in \mathbb{R}$ be the signed forward speed, measured in m s^{-1} ; positive v means motion along the positive x_b -axis.

The body-frame velocity ${}^b\mathbf{v} \in \mathbb{R}^2$ is

$${}^b\mathbf{v} = \begin{bmatrix} v \\ 0 \end{bmatrix} \text{m s}^{-1}.$$

Define $\mathbf{R}_{wb}(\theta) \in \mathbb{R}^{2 \times 2}$ as the rotation matrix that maps planar coordinates from the robot frame $\{b\}$ to the world frame $\{w\}$. The world-frame velocity ${}^w\mathbf{v} \in \mathbb{R}^2$ is therefore

$${}^w\mathbf{v} = \mathbf{R}_{wb}(\theta) {}^b\mathbf{v} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} v \\ 0 \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \end{bmatrix} \text{m s}^{-1}.$$

This equation is the coordinate-frame foundation of the differential-drive kinematic model. It shows why a controller must distinguish a forward command in the robot frame from motion observed in the world frame.

4.9 5. Failure modes and engineering checks

4.9.1 Missing frame labels

Two identical columns can describe different physical directions when they are expressed in different frames. Interfaces should name the frame explicitly.

4.9.2 Reversed transformation

Using $\mathbf{R}_{ba}$ where $\mathbf{R}_{ab}$ is required rotates coordinates in the opposite direction. Check the subscript rule before multiplying.

4.9.3 Mixed units

Degrees passed to a function expecting radians produce valid-looking but wrong numbers. Convert at the system boundary and use radians internally.

4.9.4 Point-vector confusion

A point changes under translation; a free vector does not. Applying a translation to velocity or direction data is a modelling error.

4.9.5 Invalid numerical rotation

Let $\mathbf{R} \in \mathbb{R}^{n \times n}$ be a numerically stored matrix intended to represent a rotation, and let $\mathbf{I}_n \in \mathbb{R}^{n \times n}$ be the identity matrix. Rounding, integration drift, or corrupted data can make $\mathbf{R}$ non-orthonormal.

For a matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ with entries m_{ij} , define the Frobenius norm by

$$\|\mathbf{M}\|_F := \sqrt{\sum_{i=1}^n \sum_{j=1}^n m_{ij}^2}.$$

Let $\varepsilon_{\text{orth}} > 0$ and $\varepsilon_{\text{det}} > 0$ be declared numerical tolerances. A robust interface checks

$$\|\mathbf{R}^\top \mathbf{R} - \mathbf{I}_n\|_F \leq \varepsilon_{\text{orth}}$$

and

$$|\det(\mathbf{R}) - 1| \leq \varepsilon_{\text{det}}.$$

The first condition tests approximate orthonormality; the second tests approximate preservation of right-handed orientation.

4.10 Quick test B: coordinate frames and rotations

1. **Recall:** What do the columns of $\mathbf{R}_{ab}$ represent?
2. **Explanation:** Explain in words why $\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ba}$.
3. **Derivation:** Starting with ${}^a\mathbf{v} = \mathbf{R}_{ab} {}^b\mathbf{v}$, solve for ${}^b\mathbf{v}$.
4. **Application:** A robot has heading $\theta = \pi/2$ and forward speed $v = 0.5 \text{ m s}^{-1}$. Find its world-frame velocity.
5. **Limitations:** Give two ways a numerically stored matrix intended to be a rotation matrix can become invalid.
6. **Derivation:** Let $\{a\}$ and $\{b\}$ be right-handed orthonormal planar frames. Frame $\{b\}$ is rotated counter-clockwise by θ rad relative to frame $\{a\}$, and $\mathbf{R}_{ab}(\theta) \in \mathbb{R}^{2 \times 2}$ maps coordinate columns from frame $\{b\}$ to frame $\{a\}$. First express the x_b -axis and y_b -axis as coordinate columns ${}^a\mathbf{e}_{x,b}, {}^a\mathbf{e}_{y,b} \in \mathbb{R}^2$; then place those columns side by side to derive $\mathbf{R}_{ab}(\theta)$.

4.10.1 Answers and hints

1. The axes of frame $\{b\}$ expressed in coordinates of frame $\{a\}$.
2. Reversing the frame change undoes the original change; for an orthonormal rotation matrix, the inverse is its transpose.
3. Premultiply by $\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ba}$: ${}^b\mathbf{v} = \mathbf{R}_{ba} {}^a\mathbf{v}$.
4. $[0 \ 0.5]^\top \text{ m s}^{-1}$, up to floating-point rounding.

5. Examples include accumulated integration error, rounded entries, malformed input, unit mistakes in angle construction, and data corruption.
6. Since the x_b -axis makes angle θ with the x_a -axis, its frame- $\{a\}$ coordinate column is ${}^a\mathbf{e}_{x,b} = [\cos \theta \ \sin \theta]^\top$. Right-handed orthonormality places the y_b -axis at angle $\theta + \pi/2$, so ${}^a\mathbf{e}_{y,b} = [\cos(\theta + \pi/2) \ \sin(\theta + \pi/2)]^\top = [-\sin \theta \ \cos \theta]^\top$. Because these two coordinate columns are the columns of $\mathbf{R}_{ab}(\theta)$,

$$\mathbf{R}_{ab}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

4.11 References

- Kevin M. Lynch and Frank C. Park, *Modern Robotics*, Chapter 3, especially Sections 3.1 and 3.2.1 on free vectors, frames, rotation matrices, and rigid-body motions.
- Steven M. LaValle, *Planning Algorithms*, Chapters 3 and 4 on geometric representations, transformations, and configuration spaces.

5 Differential-Drive Kinematics and Numerical Integration: Model and Derivation

5.1 Why this chapter matters

Kinematics describes motion without asking which forces or torques produce it. For a differential-drive robot, kinematics connects wheel angular speeds to the time evolution of position and heading. Numerical integration converts this continuous-time model into sampled updates used by simulation, odometry, planning, and control.

This chapter develops that chain in four steps: define planar pose and its derivative, derive the no-sideways-motion model, derive body velocity from wheel speeds, and compare exact constant-input motion with Forward Euler integration.

5.2 Prerequisites and assumptions

The reader should understand vectors and matrices from the [linear algebra foundations chapter](#), coordinate frames and rotations from the [geometry chapter](#), and elementary derivatives, integrals, sine, and cosine.

The system considered here is a differential-drive ground robot: a rigid chassis propelled by left and right drive wheels that are mounted on a common axle and can be driven independently. The robot operates on a stationary, level, rigid floor, and the model describes its motion as viewed from above. Any additional support hardware does not affect the ideal kinematic model.

Frame $\{w\}$ is fixed to the floor and is called the world frame. Frame $\{b\}$ is rigidly attached to the robot and is called the body frame. Its origin O_b lies midway between the two drive-wheel ground-contact centre lines. The positive x_b -axis points forward, the positive y_b -axis points left, and the positive z_b -axis points upward, so frame $\{b\}$ is right-handed. As the robot moves, frame $\{b\}$ translates and rotates relative to the stationary frame $\{w\}$.

Both drive wheels have the same effective rolling radius $r > 0$, measured in metres. The track width $L > 0$, measured in metres, is the lateral distance between the left and right wheel ground-contact centre lines.

The kinematic model uses the following assumptions:

- The chassis remains rigid, and its pose changes only through horizontal position and rotation about the upward z_b -axis; its height, roll, and pitch remain unchanged.
- Each wheel maintains contact with the floor and rolls without longitudinal slip, so the distance travelled in the wheel's rolling direction equals the arc length swept out at its effective radius.

- The scalar body-frame lateral component ${}^b v_y$, also written v_y when the frame is clear, is measured in m s^{-1} and lies along the $+y_b$ -direction. The wheel constraints impose ${}^b v_y = 0$ at O_b . The robot can still turn because its forward direction changes continuously.
- The effective wheel radius r and track width L remain constant.
- Wheel deformation, tyre slip, backlash, encoder error, actuator dynamics, collisions, and uneven or moving terrain are ignored.
- Length is measured in metres, time in seconds, and angle in radians; corresponding velocities are measured in m s^{-1} and rad s^{-1} .

Together, these assumptions define an ideal planar kinematic model rather than a complete physical model. A real robot violates them to some degree, especially during rapid acceleration, braking, collision, wheel slip, or motion over compliant or uneven terrain.

Figure Figure 5.1 summarizes the frames, wheel dimensions, rolling condition, and lateral-velocity constraint used by the model.

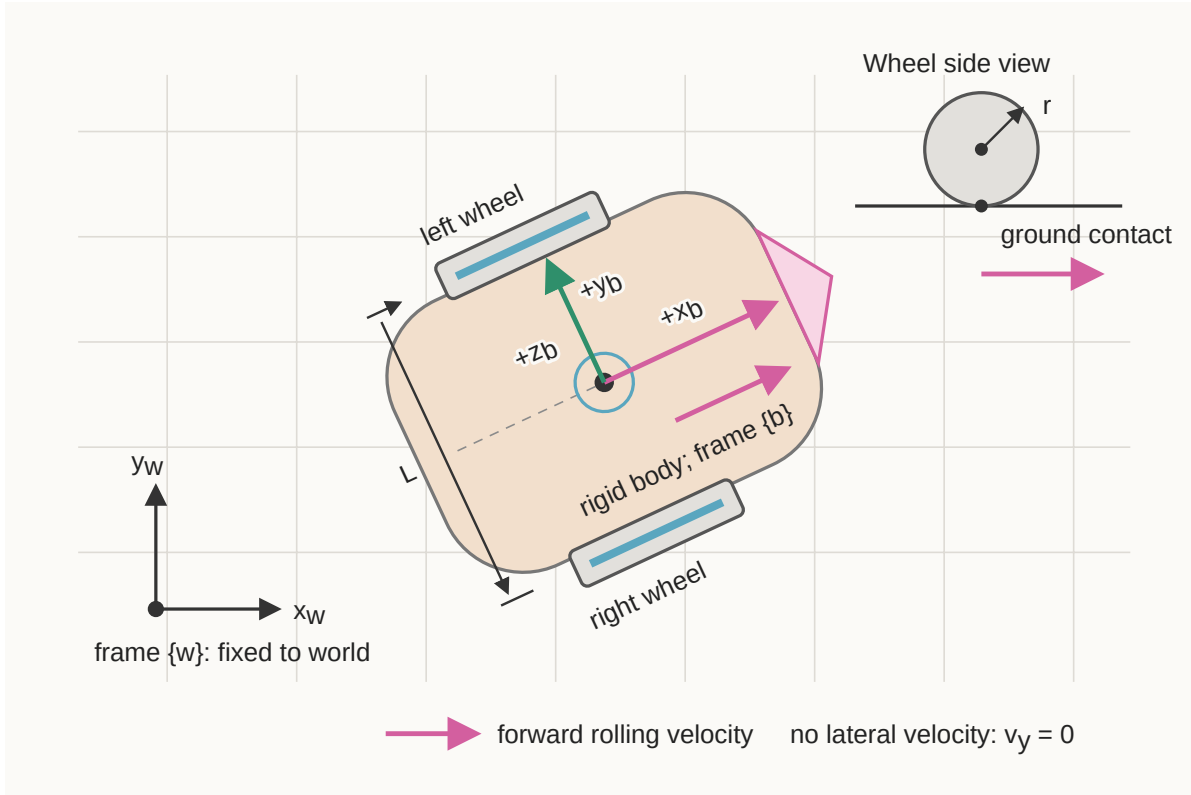


Figure 5.1: Differential-drive robot geometry and ideal rolling constraints.

As a preview of the pure-rolling relation, $v_i = r\dot{\phi}_i$ for $i \in \{L, R\}$, where v_i is the signed longitudinal ground speed of wheel i and $\dot{\phi}_i$ is its signed angular speed. Section 2.1 derives how the individual wheel speeds determine the body-forward speed v and yaw rate ω .

5.3 1. Planar pose and its derivative

Let $t \in \mathbb{R}_{\geq 0}$ denote continuous time, measured in seconds. Define the planar robot pose by

$$\mathbf{q}(t) := \begin{bmatrix} x(t) \\ y(t) \\ \theta(t) \end{bmatrix}.$$

Here, $x(t), y(t) \in \mathbb{R}$ are the world-frame coordinates of body-frame origin O_b , measured in metres, and $\theta(t) \in \mathbb{R}$ is the counter-clockwise heading of frame $\{b\}$ relative to frame $\{w\}$, measured in radians. Angles that differ by an integer multiple of 2π represent the same physical heading.

The pose derivative is

$$\dot{\mathbf{q}}(t) := \frac{d\mathbf{q}(t)}{dt} = \begin{bmatrix} \dot{x}(t) \\ \dot{y}(t) \\ \dot{\theta}(t) \end{bmatrix}.$$

The components $\dot{x}$ and $\dot{y}$ are measured in m s^{-1} , while $\dot{\theta}$ is measured in rad s^{-1} . Although they appear in one column, their units are not identical.

Define the body-velocity input by

$$\mathbf{u}(t) := \begin{bmatrix} v(t) \\ \omega(t) \end{bmatrix},$$

where $v(t) \in \mathbb{R}$ is signed forward speed along $+x_b$, measured in m s^{-1} , and $\omega(t) \in \mathbb{R}$ is signed yaw rate about $+z_b$, measured in rad s^{-1} . Positive ω produces counter-clockwise rotation when viewed from above.

5.3.1 1.1 Deriving the planar model

Because the ideal robot has no body-frame lateral velocity, its translational velocity expressed in frame $\{b\}$ is

$${}^b\mathbf{v} = \begin{bmatrix} v \\ 0 \end{bmatrix} \in \mathbb{R}^2.$$

The rotation matrix $\mathbf{R}_{wb}(\theta) \in \mathbb{R}^{2 \times 2}$ maps coordinates from frame $\{b\}$ to frame $\{w\}$. Therefore,

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \underbrace{\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}}_{\mathbf{R}_{wb}(\theta)} \begin{bmatrix} v \\ 0 \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \end{bmatrix}.$$

Together with $\dot{\theta} = \omega$, this gives

$$\dot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, \mathbf{u}) = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}.$$

Here, $\mathbf{f}$ is the kinematic state-derivative function: it maps current pose $\mathbf{q}$ and input $\mathbf{u}$ to instantaneous pose derivative $\dot{\mathbf{q}}$. The position rates depend on θ because the body forward direction changes relative to the world frame.

5.3.2 1.2 The no-sideways-motion constraint

Define the axle-midpoint linear velocity expressed in the world frame by

$${}^w\mathbf{v} := \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \in \mathbb{R}^2,$$

where $\dot{x}$ and $\dot{y}$ are its components along $+x_w$ and $+y_w$, respectively, and are measured in ms^{-1} .

The geometric unit vector along the body-frame $+y_b$ -axis, expressed using world-frame coordinates, is the second column of $\mathbf{R}_{wb}(\theta)$:

$${}^w\mathbf{e}_{y,b} = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix} \in \mathbb{R}^2.$$

Both ${}^w\mathbf{e}_{y,b}$ and ${}^w\mathbf{v}$ are expressed in frame $\{w\}$, and ${}^w\mathbf{e}_{y,b}$ has unit length. Their dot product therefore gives the signed component of the velocity along the robot's lateral $+y_b$ -direction:

$$\begin{aligned} {}^bv_y &= ({}^w\mathbf{e}_{y,b})^\top {}^w\mathbf{v} \\ &= \begin{bmatrix} -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \\ &= -\dot{x} \sin \theta + \dot{y} \cos \theta. \end{aligned}$$

The left superscript in bv_y makes explicit that this scalar is the y -component of velocity measured along a body-frame axis, even though the projection was calculated using world-frame coordinates.

The matrix $\mathbf{R}_{wb}(\theta)$ maps coordinate columns from frame $\{b\}$ to frame $\{w\}$. Its columns are orthonormal, so its inverse equals its transpose. Define the reverse coordinate map by

$$\begin{aligned} \mathbf{R}_{bw}(\theta) &:= \mathbf{R}_{wb}(\theta)^{-1} \\ &= \mathbf{R}_{wb}(\theta)^\top \\ &= \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}. \end{aligned}$$

Therefore, $\mathbf{R}_{bw}(\theta)$ maps coordinate columns from frame $\{w\}$ to frame $\{b\}$. The notation uses the same heading θ because both matrices depend on the robot's heading, but the reverse map applies the opposite rotation: $\mathbf{R}_{bw}(\theta) = \mathbf{R}_{wb}(-\theta)$.

The complete change from world-frame velocity components to body-frame velocity components is

$${}^b\mathbf{v} = \begin{bmatrix} {}^bv_x \\ {}^bv_y \end{bmatrix} = \mathbf{R}_{bw}(\theta) {}^w\mathbf{v} = \mathbf{R}_{wb}(\theta)^\top {}^w\mathbf{v} = \begin{bmatrix} \dot{x} \cos \theta + \dot{y} \sin \theta \\ -\dot{x} \sin \theta + \dot{y} \cos \theta \end{bmatrix}.$$

Here, ${}^b v_x$ and ${}^b v_y$ are the signed forward and lateral velocity components along $+x_b$ and $+y_b$, respectively, and both are measured in m s^{-1} . Thus, $\dot{x}$ and $\dot{y}$ are world-frame components, whereas ${}^b v_x$ and ${}^b v_y$ are body-frame components.

The ideal no-sideways-motion assumption is therefore written most clearly as

$$\boxed{{}^b v_y = -\dot{x} \sin \theta + \dot{y} \cos \theta = 0}.$$

The signed forward speed v defined earlier equals ${}^b v_x$, so the constrained body-frame velocity and its world-frame representation are

$${}^b \mathbf{v} = \begin{bmatrix} v \\ 0 \end{bmatrix}, \quad {}^w \mathbf{v} = \mathbf{R}_{wb}(\theta) \begin{bmatrix} v \\ 0 \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \end{bmatrix}.$$

Consequently, zero body-frame lateral velocity does not imply $\dot{y} = 0$. For example, when $\theta = \pi/4$, forward motion gives $\dot{x} = \dot{y} = v/\sqrt{2}$. The angle θ specifies orientation; it does not itself create translation. If $v = 0$ and $\omega \neq 0$, the robot rotates in place about O_b , so $\dot{x} = \dot{y} = 0$ while θ changes.

This equation is a nonholonomic velocity constraint. It restricts the directions in which the robot can move at the current pose, but it does not require the pose variables x , y , and θ to remain on a fixed lower-dimensional surface. In contrast, a holonomic condition such as $y = 0$ would confine every allowable pose to the world x_w -axis.

The distinction concerns instantaneous motion versus accumulated motion. At every instant, the axle midpoint translates only along the robot's current x_b -axis, but turning changes the direction of x_b relative to the world. For example, a robot initially aligned with $+x_w$ can rotate by $\pi/2$, drive forward a distance $d > 0$, and rotate back by $-\pi/2$. Its final heading equals its initial heading, yet it has acquired a world-frame $+y_w$ displacement of d . It never moved along its instantaneous y_b -axis; the net sideways displacement resulted from a sequence of allowed rotations and forward motions.

5.3.3 Quick test A: planar kinematics

1. Define every component of $\mathbf{q} = [x \ y \ \theta]^T$, including its frame and units.
2. Why do $\dot{x}$ and $\dot{y}$ depend on θ , while v does not?
3. Starting with ${}^b \mathbf{v} = [v \ 0]^T$, derive $\dot{y} = v \sin \theta$.
4. If $v = 1 \text{ m s}^{-1}$ and $\theta = \pi/2 \text{ rad}$, find $\dot{x}$ and $\dot{y}$.
5. Does the no-sideways-motion constraint prevent a net sideways displacement? Explain.

5.3.3.1 Answers and hints

1. x and y are the world-frame position coordinates of O_b in metres; θ is counter-clockwise body heading relative to the world frame in radians.
2. v is defined along the moving x_b -axis. Its world-frame components depend on the orientation of that axis.
3. The second row of $\mathbf{R}_{wb}(\theta)[v \ 0]^T$ is $v \sin \theta$.
4. $\dot{x} = 0 \text{ m s}^{-1}$ and $\dot{y} = 1 \text{ m s}^{-1}$.
5. No. The constraint forbids instantaneous sideways velocity, but combined forward and turning motions can produce a sideways displacement.

5.4 2. Differential-drive wheel kinematics

Let $\phi_L(t), \phi_R(t) \in \mathbb{R}$ be the rotational positions of the left and right wheels about their axles at time t , measured in radians. The subscripts L and R identify the left and right wheels. These are rolling angles, not steering angles: the wheel planes remain fixed relative to the chassis.

Choose $\phi_L = \phi_R = 0$ at an arbitrary reference position. A change of 2π radians is one complete wheel revolution. Although the marked orientation of a wheel repeats after each revolution, ϕ_L and ϕ_R are treated as unwrapped accumulated angles in $\mathbb{R}$, so they may increase beyond 2π or decrease below zero.

Their time derivatives $\dot{\phi}_L(t)$ and $\dot{\phi}_R(t)$ are the left and right wheel angular speeds, measured in rad s^{-1} . Positive angular speed is defined to roll the corresponding wheel and the robot forward.

For a wheel of effective radius r , let $\Delta\phi$ be a signed angular change and let Δs be the corresponding signed arc displacement swept out at the wheel rim. Both r and Δs are measured in metres. By the definition of radian measure,

$$\Delta\phi = \frac{\Delta s}{r}, \quad \frac{\text{m}}{\text{m}} = 1.$$

The metre units cancel, so a radian is dimensionless in the International System of Units (SI). Nevertheless, angles are labelled in rad and angular speeds in rad s^{-1} so their physical meanings remain clear. Rearranging the radian definition gives the swept arc displacement $\Delta s = r\Delta\phi$. Under rolling without slip, this swept arc displacement equals the signed longitudinal displacement along the ground.

For wheel $i \in \{L, R\}$, let $s_i(t)$ be its signed accumulated longitudinal ground displacement. Over a time interval $\Delta t > 0$, pure rolling gives $\Delta s_i = r\Delta\phi_i$. Because r is constant, dividing by Δt and taking the limit $\Delta t \rightarrow 0$ gives

$$v_i := \dot{s}_i = \lim_{\Delta t \rightarrow 0} \frac{\Delta s_i}{\Delta t} = r \lim_{\Delta t \rightarrow 0} \frac{\Delta\phi_i}{\Delta t} = r\dot{\phi}_i.$$

Applying this relation to the left and right wheels gives

$$v_L = r\dot{\phi}_L, \quad v_R = r\dot{\phi}_R,$$

where $v_L, v_R \in \mathbb{R}$ are measured in m s^{-1} . The unit calculation for either wheel is

$$\begin{aligned} \text{metres} \times \frac{\text{radians}}{\text{second}} &= \text{m} \cdot \frac{\text{rad}}{\text{s}} \\ &= \text{m} \cdot \frac{1}{\text{s}} \\ &= \text{m s}^{-1}. \end{aligned}$$

The symbol rad is retained to communicate that ϕ is an angle, even though it contributes no independent physical dimension in the unit calculation.

5.4.1 2.1 Relating wheel speeds to body motion

The rigid-body point-velocity relation derived in the [geometry and coordinate-frames chapter](#) is

$${}^b\mathbf{v}_P = {}^b\mathbf{v}_O + {}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{OP},$$

provided that every vector is expressed in the same orthonormal frame, here the body frame $\{b\}$. The vector ${}^b\mathbf{v}_P$ is the velocity of a rigid-body point P , ${}^b\mathbf{v}_O$ is the velocity of a reference point O , ${}^b\mathbf{r}_{OP}$ is the position of P relative to O , and ${}^b\boldsymbol{\omega}$ is the rigid body angular-velocity vector.

For the differential-drive chassis, choose the reference point as the axle midpoint $O = O_b$. Let C_L and C_R denote the left and right wheel centres on the axle. Both points are fixed relative to the chassis. Under ideal rolling without longitudinal slip, the $+x_b$ components of their translational velocities equal the longitudinal wheel speeds v_L and v_R defined above.

Because $+y_b$ points left, the wheel-centre position vectors are

$${}^b\mathbf{r}_{O_b C_L} = \begin{bmatrix} 0 \\ L/2 \\ 0 \end{bmatrix}, \quad {}^b\mathbf{r}_{O_b C_R} = \begin{bmatrix} 0 \\ -L/2 \\ 0 \end{bmatrix}, \quad {}^b\mathbf{r}_{O_b C_L}, {}^b\mathbf{r}_{O_b C_R} \in \mathbb{R}^3.$$

Their components have units of metres. The axle midpoint moves forward with scalar speed v , and the chassis yaws about $+z_b$ with scalar rate ω , so

$${}^b\mathbf{v}_{O_b} = \begin{bmatrix} v \\ 0 \\ 0 \end{bmatrix} \in \mathbb{R}^3, \quad {}^b\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix} \in \mathbb{R}^3.$$

The components of ${}^b\mathbf{v}_{O_b}$ have units of m s^{-1} , and the components of ${}^b\boldsymbol{\omega}$ have units of rad s^{-1} .

Applying the rigid-body relation to each wheel-centre point gives

$${}^b\mathbf{v}_{C_L} = {}^b\mathbf{v}_{O_b} + {}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{O_b C_L}, \quad {}^b\mathbf{v}_{C_R} = {}^b\mathbf{v}_{O_b} + {}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{O_b C_R}.$$

For the left wheel, the rotational contribution is

$${}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{O_b C_L} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix} \times \begin{bmatrix} 0 \\ L/2 \\ 0 \end{bmatrix} = \begin{bmatrix} -L\omega/2 \\ 0 \\ 0 \end{bmatrix}.$$

For the right wheel, it is

$${}^b\boldsymbol{\omega} \times {}^b\mathbf{r}_{O_b C_R} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix} \times \begin{bmatrix} 0 \\ -L/2 \\ 0 \end{bmatrix} = \begin{bmatrix} L\omega/2 \\ 0 \\ 0 \end{bmatrix}.$$

Therefore, the wheel-centre velocity vectors are

$${}^b\mathbf{v}_{C_L} = \begin{bmatrix} v - L\omega/2 \\ 0 \\ 0 \end{bmatrix}, \quad {}^b\mathbf{v}_{C_R} = \begin{bmatrix} v + L\omega/2 \\ 0 \\ 0 \end{bmatrix}.$$

The longitudinal wheel speeds v_L and v_R are the $+x_b$ components of these vectors. Consequently,

$$\boxed{v_L = v - \frac{L}{2}\omega, \quad v_R = v + \frac{L}{2}\omega.}$$

The signs follow from the right-hand-rule identities $\mathbf{e}_{z,b} \times \mathbf{e}_{y,b} = -\mathbf{e}_{x,b}$ and $\mathbf{e}_{z,b} \times (-\mathbf{e}_{y,b}) = \mathbf{e}_{x,b}$. For a positive yaw rate, the robot turns left: rotation reduces the left-wheel forward speed and increases the right-wheel forward speed. The product $L\omega$ has units m s^{-1} because radians are dimensionless in dimensional calculations.

Adding the two equations isolates forward speed, while subtracting the left equation from the right equation isolates yaw rate:

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{L}.$$

Substituting the pure-rolling relations produces

$$\boxed{v = \frac{r}{2}(\dot{\phi}_R + \dot{\phi}_L), \quad \omega = \frac{r}{L}(\dot{\phi}_R - \dot{\phi}_L).}$$

In matrix form,

$$\boxed{\begin{bmatrix} v \\ \omega \end{bmatrix} = \underbrace{\begin{bmatrix} r/2 & r/2 \\ -r/L & r/L \end{bmatrix}}_{\mathbf{G} \in \mathbb{R}^{2 \times 2}} \begin{bmatrix} \dot{\phi}_L \\ \dot{\phi}_R \end{bmatrix}.}$$

The matrix $\mathbf{G}$ maps the wheel-speed column $[\dot{\phi}_L \ \dot{\phi}_R]^\top$ to the body-velocity column $[v \ \omega]^\top$. In this chapter, this input-output direction is called the **wheel-to-body velocity mapping**: its inputs are the two wheel angular speeds, and its outputs are the resulting body-forward speed and yaw rate. It is also called the **forward differential-drive velocity kinematics**. Here, *velocity kinematics* means an algebraic relationship among instantaneous velocities without integrating the pose or modelling the forces that produce the motion. The word *forward* describes the calculation from wheel motion to body motion; it does not mean that the robot must travel in the positive forward direction, because the signed wheel speeds and v may be negative.

The two rows of $\mathbf{G}$ have different physical units because the two outputs have different units.

Because $r > 0$ and $L > 0$, the mapping can be inverted. Solving for the wheel speeds gives

$$\boxed{\dot{\phi}_L = \frac{v}{r} - \frac{L\omega}{2r}, \quad \dot{\phi}_R = \frac{v}{r} + \frac{L\omega}{2r}.}$$

This opposite input-output direction is called the **body-to-wheel velocity mapping**, or the **inverse differential-drive velocity kinematics**. Its inputs are a desired body-forward speed v and yaw rate ω , and its outputs are the ideal left and right wheel angular speeds $\dot{\phi}_L$ and $\dot{\phi}_R$ required by the no-slip model. The word *inverse* means that this calculation reverses the wheel-to-body mapping; it does not mean that the robot is commanded to drive backwards. In a controller, the body-to-wheel mapping converts a desired chassis motion into wheel-speed commands, whereas the wheel-to-body mapping can convert measured wheel speeds into an estimate of chassis motion.

5.4.2 2.2 Characteristic motions

- If $\dot{\phi}_L = \dot{\phi}_R$, then $\omega = 0$ and the robot moves straight.
- If $\dot{\phi}_L = -\dot{\phi}_R$, then $v = 0$ and the robot rotates about O_b .
- If one wheel is stationary, the ideal robot follows a circle about that wheel contact point.
- If the wheel speeds are unequal and not opposite, the robot follows an arc about an instantaneous centre; the signed turning radius is derived next.

5.4.2.1 2.2.1 Instantaneous centre of curvature and signed turning radius

Assume $\omega \neq 0$ at time t . At that instant, the planar rigid-body motion can be regarded as rotation about a point C in the plane whose instantaneous linear velocity is zero. This point is called the instantaneous centre of curvature (ICC). The body origin O_b therefore moves instantaneously along a circular arc centred at C .

The no-sideways-motion constraint makes the velocity of O_b tangent to the arc and parallel to the body x_b -axis. A circle radius is perpendicular to its tangent, so the line joining O_b and C lies along the lateral y_b -axis.

Let ${}^w\mathbf{p}_{O_b}, {}^w\mathbf{p}_C \in \mathbb{R}^2$ be the world-frame position columns of O_b and C , respectively. Define the ordinary nonnegative radius ρ , measured in metres, by

$$\rho := \text{distance}(O_b, C) = \|{}^w\mathbf{p}_C - {}^w\mathbf{p}_{O_b}\|_2 \geq 0.$$

The magnitude ρ records only the distance between the two points. Define the signed turning radius $R \in \mathbb{R}$ by using its sign to record which side of the robot contains the ICC:

$$R := \begin{cases} +\rho, & C \text{ is to the left of } O_b, \\ -\rho, & C \text{ is to the right of } O_b. \end{cases}$$

Consequently, $|R| = \rho$. The relative-position vector from O_b to C , expressed in world-frame coordinates, is

$${}^w\mathbf{r}_{O_b C} := {}^w\mathbf{p}_C - {}^w\mathbf{p}_{O_b} = R {}^w\mathbf{e}_{y,b}, \quad {}^w\mathbf{e}_{y,b} = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix} \in \mathbb{R}^2.$$

The unit vector ${}^w\mathbf{e}_{y,b}$ points along the robot lateral $+y_b$ -axis and is expressed in world coordinates. Multiplying it by $R > 0$ places C to the left of O_b , while multiplying it by $R < 0$ reverses the

direction and places C to the right. Figure 5.2 shows both cases. The forward velocity is tangent to the circular path and therefore perpendicular to the radius direction.

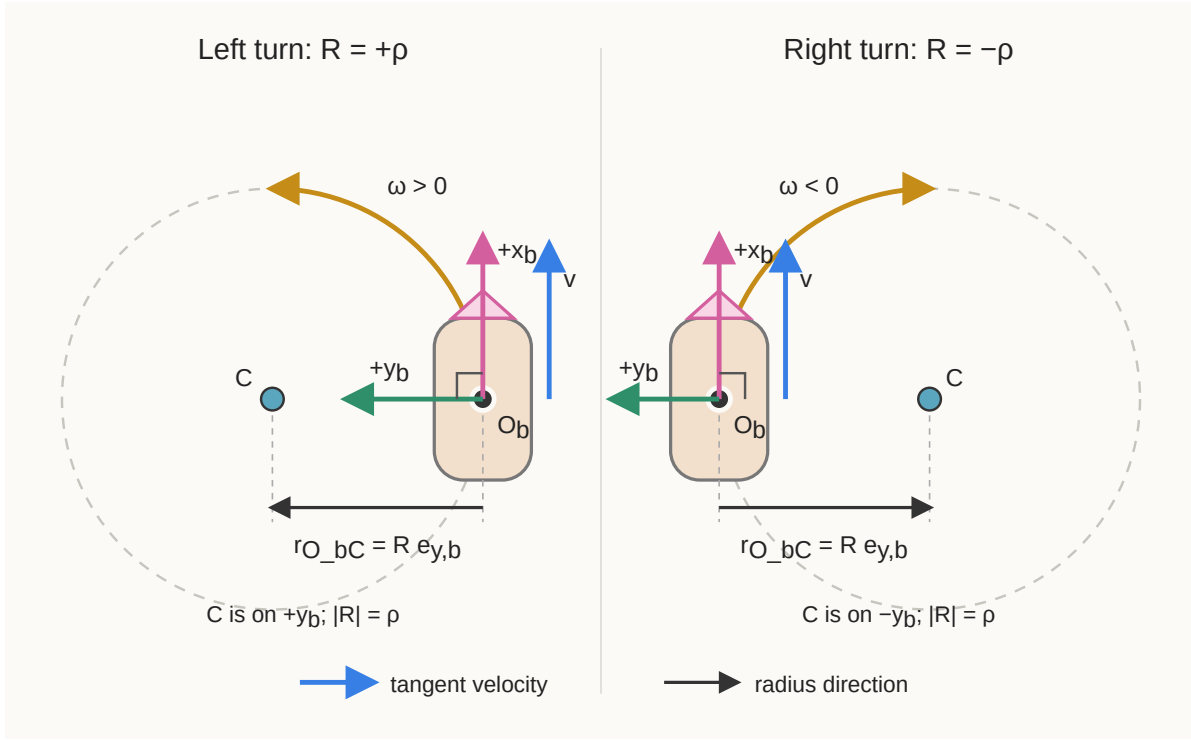


Figure 5.2: Relationship among the body origin, body axes, instantaneous centre of curvature, tangent velocity, and signed turning radius.

To derive the relationship among R , v , and ω , consider a short interval $\Delta t > 0$ during which the instantaneous values are treated as constant. Define the signed heading change by

$$\Delta\theta := \theta(t + \Delta t) - \theta(t),$$

where $\Delta\theta$ is measured in radians. Let $\Delta\ell \in \mathbb{R}$ be the signed distance travelled by O_b along the arc, measured in metres; $\Delta\ell > 0$ denotes travel in the $+x_b$ -direction and $\Delta\ell < 0$ denotes travel in the $-x_b$ -direction.

The radian definition gives arc length equal to radius times angular change. With the signed quantities just defined, it becomes

$$\boxed{\Delta\ell = R \Delta\theta}.$$

For a forward left turn, both R and $\Delta\theta$ are positive. For a forward right turn, both are negative. Their product is therefore the positive forward distance $\Delta\ell$ in either case; the same signed equation also remains valid for backward motion.

Divide the signed arc equation by Δt :

$$\frac{\Delta\ell}{\Delta t} = R \frac{\Delta\theta}{\Delta t}.$$

Taking the limit as $\Delta t \rightarrow 0$ gives

$$\frac{d\ell}{dt} = R \frac{d\theta}{dt}.$$

By definition, the signed forward speed is $v = d\ell/dt$, measured in m s^{-1} , and the signed yaw rate is $\omega = d\theta/dt$, measured in rad s^{-1} . Therefore,

$$\boxed{v = R\omega}.$$

When $\omega \neq 0$, division by ω yields

$$\boxed{R = \frac{v}{\omega}}, \quad \omega \neq 0.$$

The quotient has units of metres because radians are dimensionless in dimensional calculations. If $\omega = 0$ and $v \neq 0$, the robot moves straight and has no finite ICC; its turning radius is informally described as infinite. If $v = 0$ and $\omega \neq 0$, then $R = 0$ and the robot rotates in place about O_b .

The symbol R in this subsection is a scalar signed turning radius, not a rotation matrix.

5.4.3 Quick test B: wheel-to-body mapping

1. Define ϕ_L , ϕ_R , $\dot{\phi}_L$, $\dot{\phi}_R$, r , and L , including their units, and distinguish wheel angle from angular speed.
2. Why is a radian dimensionless in an SI unit calculation, and why does $r\dot{\phi}$ have units of metres per second?
3. Starting from ${}^b\mathbf{v}_P = {}^b\mathbf{v}_O + {}^b\omega \times {}^b\mathbf{r}_{OP}$, use the two wheel-centre position vectors to derive $v_L = v - L\omega/2$ and $v_R = v + L\omega/2$.
4. Why does equal wheel speed produce $\omega = 0$?
5. Derive $\omega = r(\dot{\phi}_R - \dot{\phi}_L)/L$.
6. Let $r = 0.10 \text{ m}$, $L = 0.50 \text{ m}$, $\dot{\phi}_L = 2 \text{ rad s}^{-1}$, and $\dot{\phi}_R = 4 \text{ rad s}^{-1}$. Find v and ω .
7. Define the ICC C , ordinary radius ρ , signed radius R , signed distance $\Delta\ell$, and heading change $\Delta\theta$. Starting from $\Delta\ell = R\Delta\theta$, derive $R = v/\omega$ and explain the signs for left and right turns.
8. Name two physical effects that can make predicted and real motion disagree.

5.4.3.1 Answers and hints

1. ϕ_L and ϕ_R are unwrapped left and right wheel rotational positions in radians; $\dot{\phi}_L$ and $\dot{\phi}_R$ are their angular speeds in rad s^{-1} ; r is effective wheel radius in metres; and L is track width in metres.
2. Radian measure is an arc-length-to-radius ratio, so its metre units cancel. Therefore, $r\dot{\phi}$ has units $\text{m} \cdot \text{rad s}^{-1} = \text{m s}^{-1}$.
3. In frame $\{b\}$, use ${}^b\mathbf{r}_{O_b C_L} = [0, L/2, 0]^T$, ${}^b\mathbf{r}_{O_b C_R} = [0, -L/2, 0]^T$, ${}^b\mathbf{v}_{O_b} = [v, 0, 0]^T$, and ${}^b\omega = [0, 0, \omega]^T$. The cross products contribute $[-L\omega/2, 0, 0]^T$ on the left and $[L\omega/2, 0, 0]^T$ on the right. Adding ${}^b\mathbf{v}_{O_b}$ and selecting the $+x_b$ components gives the two required equations.
4. Equal wheel speeds have zero difference, so the yaw-rate equation gives $\omega = 0$.

5. Subtract $v_L = v - L\omega/2$ from $v_R = v + L\omega/2$, then substitute $v_R = r\dot{\phi}_R$ and $v_L = r\dot{\phi}_L$.
6. $v = 0.30 \text{ m s}^{-1}$ and $\omega = 0.40 \text{ rad s}^{-1}$.
7. The ICC C is the point about which the robot is instantaneously rotating. The radius $\rho = \text{distance}(O_b, C) \geq 0$ is an ordinary distance, while $R = +\rho$ for an ICC on the left and $R = -\rho$ for one on the right. The quantity $\Delta\ell$ is signed forward distance and $\Delta\theta$ is signed heading change. Dividing $\Delta\ell = R\Delta\theta$ by Δt and taking the limit gives $v = R\omega$, hence $R = v/\omega$ for $\omega \neq 0$. A forward left turn has $R > 0$ and $\omega > 0$; a forward right turn has $R < 0$ and $\omega < 0$.
8. Examples include wheel slip, unequal wheel radii, tyre deformation, backlash, encoder error, an incorrect track width, and uneven ground.

5.5 3. From velocity to a finite pose change

The differential equation $\dot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, \mathbf{u})$ specifies instantaneous motion. A simulator or odometry system needs the pose after a finite interval.

Let t_k denote sample time k , where $k \in \{0, 1, 2, \dots\}$, and define

$$\Delta t := t_{k+1} - t_k > 0.$$

The sampling interval Δt is measured in seconds. Define $\mathbf{q}_k := \mathbf{q}(t_k)$ and assume that v and ω remain constant on $[t_k, t_{k+1}]$. This is a zero-order-hold input assumption.

5.5.1 3.1 Exact straight motion

If $\omega = 0$, the heading remains constant. Recall that

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix}.$$

Integrating the three rate equations gives

$$\begin{cases} x_{k+1} = x_k + v\Delta t \cos \theta_k, \\ y_{k+1} = y_k + v\Delta t \sin \theta_k, \\ \theta_{k+1} = \theta_k. \end{cases}$$

5.5.2 3.2 Exact constant-curvature motion

If $\omega \neq 0$, let $\tau \in [0, \Delta t]$ denote elapsed time within the interval. Then

$$\theta(t_k + \tau) = \theta_k + \omega\tau.$$

Substituting this heading into the position-rate equations and integrating gives

$$\begin{aligned} x_{k+1} - x_k &= \int_0^{\Delta t} v \cos(\theta_k + \omega\tau) d\tau \\ &= \frac{v}{\omega} [\sin(\theta_k + \omega\Delta t) - \sin \theta_k], \\ y_{k+1} - y_k &= \int_0^{\Delta t} v \sin(\theta_k + \omega\tau) d\tau \\ &= \frac{v}{\omega} [\cos \theta_k - \cos(\theta_k + \omega\Delta t)]. \end{aligned}$$

Therefore,

$$\boxed{\begin{aligned} x_{k+1} &= x_k + \frac{v}{\omega} [\sin(\theta_k + \omega\Delta t) - \sin \theta_k], \\ y_{k+1} &= y_k + \frac{v}{\omega} [\cos \theta_k - \cos(\theta_k + \omega\Delta t)], \\ \theta_{k+1} &= \theta_k + \omega\Delta t. \end{aligned}}$$

The signed turning radius is $R = v/\omega$, measured in metres, and the signed angular change is $\Delta\theta = \omega\Delta t$, measured in radians. The robot therefore follows a circular arc.

The formulas contain v/ω and therefore cannot be evaluated directly when $\omega = 0$. To express their continuous limit, define the unnormalised sinc function $\text{sinc} : \mathbb{R} \rightarrow \mathbb{R}$ by

$$\text{sinc}(z) := \begin{cases} \frac{\sin z}{z}, & z \neq 0, \\ 1, & z = 0. \end{cases}$$

Here, $z \in \mathbb{R}$ is a dimensionless scalar, and the value at zero follows from $\lim_{z \rightarrow 0} \sin z/z = 1$. This convention must be stated because some software libraries instead define a normalised sinc function containing πz .

Using the trigonometric identities

$$\sin(\theta_k + \Delta\theta) - \sin \theta_k = 2 \cos\left(\theta_k + \frac{\Delta\theta}{2}\right) \sin\left(\frac{\Delta\theta}{2}\right)$$

and

$$\cos \theta_k - \cos(\theta_k + \Delta\theta) = 2 \sin\left(\theta_k + \frac{\Delta\theta}{2}\right) \sin\left(\frac{\Delta\theta}{2}\right),$$

the common scale factor in both position increments can be rewritten, for $\omega \neq 0$, as

$$\begin{aligned}\frac{v}{\omega} 2 \sin\left(\frac{\Delta\theta}{2}\right) &= v\Delta t \frac{\sin(\Delta\theta/2)}{\Delta\theta/2} \\ &= v\Delta t \operatorname{sinc}\left(\frac{\Delta\theta}{2}\right),\end{aligned}$$

where $\Delta\theta = \omega\Delta t$ was used in the first equality. The exact update is consequently equivalent to

$$\boxed{\begin{aligned}x_{k+1} &= x_k + v\Delta t \operatorname{sinc}\left(\frac{\Delta\theta}{2}\right) \cos\left(\theta_k + \frac{\Delta\theta}{2}\right), \\ y_{k+1} &= y_k + v\Delta t \operatorname{sinc}\left(\frac{\Delta\theta}{2}\right) \sin\left(\theta_k + \frac{\Delta\theta}{2}\right), \\ \theta_{k+1} &= \theta_k + \Delta\theta.\end{aligned}}$$

As $\omega \rightarrow 0$, the angular change $\Delta\theta \rightarrow 0$, $\operatorname{sinc}(\Delta\theta/2) \rightarrow 1$, and the midpoint heading $\theta_k + \Delta\theta/2 \rightarrow \theta_k$. The position equations therefore reduce smoothly to the straight-motion update

$$x_{k+1} = x_k + v\Delta t \cos\theta_k, \quad y_{k+1} = y_k + v\Delta t \sin\theta_k.$$

Software may use an explicit straight-motion case when $|\omega|$ is below a declared tolerance or a sinc implementation designed to remain numerically stable near zero. Directly computing $\sin z/z$ at $z = 0$ would still divide by zero despite the function's well-defined limiting value.

5.5.3 Quick test C: exact finite motion

1. What does the zero-order-hold assumption say about v and ω ?
2. Why is $\theta(t_k + \tau) = \theta_k + \omega\tau$ when ω is constant?
3. Where does the factor $1/\omega$ in the position integral come from?
4. Starting from $\mathbf{q}_k = [0 \ 0 \ 0]^\top$, let $v = 1 \text{ m s}^{-1}$, $\omega = 1 \text{ rad s}^{-1}$, and $\Delta t = 1 \text{ s}$. Find the exact next pose.
5. Define the unnormalised sinc function and explain how it exposes the straight-motion limit.
6. Why should software not directly evaluate either v/ω at $\omega = 0$ or $\sin z/z$ at $z = 0$?

5.5.3.1 Answers and hints

1. Both inputs remain constant throughout the sample interval.
2. Integrating $\dot{\theta} = \omega$ for elapsed time τ gives heading change $\omega\tau$.
3. With $z = \theta_k + \omega\tau$, $dz = \omega d\tau$, so $d\tau = dz/\omega$.
4. $\mathbf{q}_{k+1} = [\sin 1 \ 1 - \cos 1 \ 1]^\top \approx [0.8415 \ 0.4597 \ 1.0000]^\top$, with position in metres and heading in radians.
5. The unnormalised function is $\operatorname{sinc}(z) = \sin z/z$ for $z \neq 0$ and $\operatorname{sinc}(0) = 1$. Rewriting the update with $\operatorname{sinc}(\Delta\theta/2)$ makes the limit visible: as $\omega \rightarrow 0$, $\Delta\theta \rightarrow 0$, the sinc factor tends to 1, and the midpoint heading tends to θ_k .

6. Both direct expressions would divide by zero even though their mathematical limits are well defined. Software must select the straight case or evaluate sinc with a numerically stable near-zero implementation.

5.6 4. Numerical integration with Forward Euler

For a general time-varying input, the exact solution may not have a convenient closed form. The integral form of the state equation over one interval is

$$\mathbf{q}(t_{k+1}) = \mathbf{q}(t_k) + \int_{t_k}^{t_{k+1}} \mathbf{f}(\mathbf{q}(t), \mathbf{u}(t)) dt.$$

Numerical integration approximates this integral using a finite number of evaluations of $\mathbf{f}$.

5.6.1 4.1 Derivation from a Taylor expansion

Assume $\mathbf{q}(t)$ is sufficiently differentiable near t_k . Its Taylor expansion is

$$\mathbf{q}(t_k + \Delta t) = \mathbf{q}(t_k) + \Delta t \dot{\mathbf{q}}(t_k) + \frac{\Delta t^2}{2} \ddot{\mathbf{q}}(\xi_k),$$

for some $\xi_k \in (t_k, t_{k+1})$. Substituting $\dot{\mathbf{q}}(t_k) = \mathbf{f}(\mathbf{q}_k, \mathbf{u}_k)$ and discarding the term proportional to Δt^2 gives the Forward Euler update

$$\boxed{\mathbf{q}_{k+1}^E = \mathbf{q}_k^E + \Delta t \mathbf{f}(\mathbf{q}_k^E, \mathbf{u}_k)}.$$

The superscript E identifies the Euler approximation, and $\mathbf{u}_k := \mathbf{u}(t_k)$ is the sampled input. Forward Euler uses the slope at the beginning of the interval and treats it as constant across the whole interval.

For the planar kinematic model,

$$\boxed{\begin{aligned} x_{k+1}^E &= x_k^E + \Delta t v_k \cos \theta_k^E, \\ y_{k+1}^E &= y_k^E + \Delta t v_k \sin \theta_k^E, \\ \theta_{k+1}^E &= \theta_k^E + \Delta t \omega_k. \end{aligned}}$$

Here, v_k and ω_k are the body-forward speed and yaw rate used for sample k .

5.6.2 4.2 Local and accumulated error

Let $E(\Delta t) \geq 0$ denote a chosen error magnitude produced using step size $\Delta t > 0$. For an exponent $p > 0$, the statement

$$E(\Delta t) = O(\Delta t^p) \quad \text{as} \quad \Delta t \rightarrow 0$$

means that there are constants $C > 0$ and $\Delta t_0 > 0$, independent of Δt , such that

$$E(\Delta t) \leq C\Delta t^p \quad \text{for every} \quad 0 < \Delta t \leq \Delta t_0.$$

The exponent p is the convergence order. The big- O statement is a bound on how the error scales for sufficiently small steps, not an equation claiming that the error equals Δt^p . When the leading error term is nonzero and Δt is sufficiently small, one commonly observes the approximate scaling $E(\Delta t) \approx K\Delta t^p$ for a constant $K > 0$ that does not depend on Δt .

The local truncation error is the error introduced in one step when that step begins from the exact state. Forward Euler has local truncation error $O(\Delta t^2)$. Over a fixed total time interval, reducing Δt creates more steps, and their errors accumulate. The resulting global error of Forward Euler is generally $O(\Delta t)$. In the sufficiently-small-step regime where the leading first-order term dominates, halving Δt therefore approximately halves the global error; it does not usually quarter it.

5.6.3 4.3 Worked comparison with the exact update

Use the initial pose and constant input

$$\mathbf{q}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad v = 1 \text{ m s}^{-1}, \quad \omega = 1 \text{ rad s}^{-1}, \quad \Delta t = 1 \text{ s}.$$

The exact update is

$$\mathbf{q}_1 \approx \begin{bmatrix} 0.8415 \text{ m} \\ 0.4597 \text{ m} \\ 1.0000 \text{ rad} \end{bmatrix}.$$

Forward Euler evaluates the direction only at initial heading $\theta_0 = 0$:

$$\mathbf{q}_1^E = \begin{bmatrix} 0 + 1 \cos 0 \\ 0 + 1 \sin 0 \\ 0 + 1 \end{bmatrix} = \begin{bmatrix} 1 \text{ m} \\ 0 \text{ m} \\ 1 \text{ rad} \end{bmatrix}.$$

Both methods produce the same heading because ω is constant and the heading equation is linear. The Euler position is wrong because the forward direction rotates substantially during the one-second interval. Define one-step position error by

$$e_p := \sqrt{(x_1^E - x_1)^2 + (y_1^E - y_1)^2}.$$

For this example, $e_p \approx 0.4863$ m. Smaller steps allow Euler to update the direction more often. When constant-input differential-drive motion is assumed, however, the exact arc update is available and more accurate.

5.6.4 4.4 Choosing and checking an integration method

- Use the exact straight-or-arc update when v and ω are held constant and the ideal planar model is appropriate.
- Use a numerical method when the input or model varies within the interval and no convenient exact solution is available.
- Reduce Δt when the state changes too much during one step, while recognising that smaller steps increase computation.
- Compare a numerical trajectory with a known exact case before trusting it on a harder case.
- Compute Δt from the intended time source; do not silently assume perfectly periodic execution.

5.6.5 Quick test D: Forward Euler

1. State the Forward Euler update and define every symbol in it.
2. In plain language, what approximation does Forward Euler make during one step?
3. Suppose one Forward Euler step begins from the exact pose and v and ω remain constant during the step. Why does Forward Euler compute the exact position when $\omega = 0$, but generally not when the robot turns with $\omega \neq 0$?
4. What does the global-error statement $E(\Delta t) = O(\Delta t)$ mean, and what approximate change in global error should occur when a sufficiently small Δt is halved over the same fixed total time interval?
5. Give one exact verification case and one physical limitation for an odometry integrator.

5.6.5.1 Answers and hints

1. $\mathbf{q}_{k+1}^E = \mathbf{q}_k^E + \Delta t \mathbf{f}(\mathbf{q}_k^E, \mathbf{u}_k)$. The terms are the next approximate state, current approximate state, step duration, state-derivative function, and sampled input.
2. It uses the derivative at the start as if that derivative remained constant across the interval.
3. When $\omega = 0$, the heading θ remains constant. Constant v then gives the constant world-frame velocity $[v \cos \theta_k \ v \sin \theta_k]^\top$, so multiplying the initial velocity by Δt evaluates the position integral exactly. When $\omega \neq 0$ and $v \neq 0$, the heading and therefore the world-frame velocity direction change continuously during the step. Forward Euler holds the initial direction constant and produces a straight tangent displacement instead of the exact arc displacement. The word “generally” allows special cases such as pure rotation with $v = 0$, for which the position remains unchanged and is therefore exact.
4. The statement $E(\Delta t) = O(\Delta t)$ means that constants $C > 0$ and $\Delta t_0 > 0$ exist such that $E(\Delta t) \leq C\Delta t$ whenever $0 < \Delta t \leq \Delta t_0$. It describes first-order scaling as $\Delta t \rightarrow 0$, not exact equality. In the regime where the leading first-order error dominates, $E(\Delta t) \approx K\Delta t$, so replacing Δt by $\Delta t/2$ approximately halves the global error over the same fixed total time interval.

5. Exact cases include constant straight motion, pure rotation, and constant-curvature motion. Physical limitations include slip, parameter error, encoder quantisation, and timing error.

5.7 5. Engineering implications and failure modes

5.7.1 Frame inconsistency

The scalar v is a body-frame forward speed, but $\dot{x}$ and $\dot{y}$ are world-frame components. Assigning $\dot{x} = v$ is valid only when $\theta = 0$ modulo 2π .

5.7.2 Sign-convention mismatch

Swapping left and right wheels or using a clockwise-positive yaw convention reverses the predicted turn. Under this chapter convention, $\dot{\phi}_L < \dot{\phi}_R$ must produce positive counter-clockwise yaw.

5.7.3 Unit mismatch

Wheel speed may arrive in revolutions per minute while the model expects radians per second. Heading may arrive in degrees while trigonometric functions expect radians. Convert units at explicit interface boundaries.

5.7.4 Incorrect or variable sampling time

Using an assumed Δt when measurements arrive irregularly causes distance and angle errors. An update should validate timestamp differences and define behaviour for zero, negative, missing, or unreasonably large intervals.

5.7.5 Angle representation

The equations allow θ to grow without bound, but an interface may require $\theta \in (-\pi, \pi]$. Wrapping changes the numerical representative, not the physical orientation. Heading differences require a wrap-aware calculation.

5.7.6 Model error versus integration error

Reducing Δt reduces integration error but cannot correct an inaccurate physical model. If a wheel slips, even exact integration of the ideal no-slip equations gives the wrong physical trajectory.

5.7.7 Encoder integration and drift

Wheel encoders measure incremental rotation rather than global position. Integrating these increments estimates pose relative to an initial condition. Small systematic and random errors accumulate, so wheel odometry alone generally drifts and requires external sensing for long-term correction.

5.8 Cumulative chapter test

1. Define $\mathbf{q}$, $\mathbf{u}$, $\mathbf{f}$, r , L , $\dot{\phi}_L$, $\dot{\phi}_R$, t_k , and Δt , including dimensions or units.
2. Starting with ${}^b\mathbf{v} = [v \ 0]^\top$, derive $\dot{x} = v \cos \theta$ and $\dot{y} = v \sin \theta$.
3. Derive the no-sideways-motion constraint from the world-frame coordinate column of the y_b -axis.
4. Derive the wheel-to-body equations for v and ω and the body-to-wheel equations for $\dot{\phi}_L$ and $\dot{\phi}_R$.
5. A robot has $r = 0.08$ m, $L = 0.40$ m, $\dot{\phi}_L = 5$ rad s⁻¹, and $\dot{\phi}_R = 7$ rad s⁻¹. Find v and ω and state the turn direction.
6. Derive the exact constant-curvature update by integrating the position rates over one interval.
7. Derive Forward Euler from the first two Taylor terms and explain the discarded term.
8. Explain why exact mathematical integration can still produce inaccurate physical odometry.
9. Design three verification cases that separately reveal a wheel-order error, a yaw-sign error, and excessive integration error.

5.8.1 Cumulative answers and hints

1. $\mathbf{q} = [x \ y \ \theta]^\top$ is planar pose with units m, m, rad; $\mathbf{u} = [v \ \omega]^\top$ contains forward speed and yaw rate in m s⁻¹ and rad s⁻¹; $\mathbf{f}$ maps pose and input to pose derivative; r is wheel radius in metres; L is track width in metres; $\dot{\phi}_L$ and $\dot{\phi}_R$ are wheel angular speeds in radians per second; t_k is sample time in seconds; and $\Delta t = t_{k+1} - t_k$ is the positive integration interval in seconds.
2. Premultiply ${}^b\mathbf{v}$ by $\mathbf{R}_{wb}(\theta)$ and read the two components.
3. Use ${}^w\mathbf{e}_{y,b} = [-\sin \theta \ \cos \theta]^\top$ and set its dot product with $[\dot{x} \ \dot{y}]^\top$ equal to zero.
4. Use $r\dot{\phi}_L = v - L\omega/2$ and $r\dot{\phi}_R = v + L\omega/2$. Addition isolates v and subtraction isolates ω ; solving the pair for wheel speeds gives the inverse equations.
5. $v = (0.08/2)(5 + 7) = 0.48$ m s⁻¹ and $\omega = (0.08/0.40)(7 - 5) = 0.40$ rad s⁻¹. The positive yaw rate is a counter-clockwise or left turn.
6. Write $\theta(t_k + \tau) = \theta_k + \omega\tau$, integrate over $\tau \in [0, \Delta t]$, and use antiderivatives $\sin(\theta_k + \omega\tau)/\omega$ and $-\cos(\theta_k + \omega\tau)/\omega$.
7. Taylor expansion gives $\mathbf{q}(t_k + \Delta t) = \mathbf{q}(t_k) + \Delta t \dot{\mathbf{q}}(t_k) + O(\Delta t^2)$. Replacing $\dot{\mathbf{q}}$ with $\mathbf{f}$ and omitting the second-order remainder gives Forward Euler. The discarded term represents within-step change of the derivative.
8. Exact integration eliminates approximation for the stated model and input assumption; it cannot eliminate slip, parameter error, encoder error, timing error, or other model violations.
9. For wheel order, command unequal known wheel speeds and check the turn direction. For yaw sign, command pure positive rotation and check that heading increases counter-clockwise. For integration error, compare with the exact constant-curvature solution at decreasing values of Δt and verify convergence.

5.9 References

- Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Chapter 13, especially the wheeled-mobile-robot kinematic models and odometry discussion.
- Steven M. LaValle, *Planning Algorithms*, Chapter 13, especially continuous-time differential models, velocity constraints, and wheeled-vehicle models.

6 Differential-Drive Kinematics and Numerical Integration: Implementation

6.1 Purpose and scope

This companion translates the differential-drive mathematics derived in [Differential-Drive Kinematics and Numerical Integration: Model and Derivation](#) into reusable software contracts, invariants, validation rules, and tests. It records how those equations map into dependable software without repeating the derivations.

6.2 1. Wheel-to-body velocity mapping

Let $r > 0$ be the common effective wheel radius in metres and let $L > 0$ be the track width in metres. Let $\dot{\phi}_L$ and $\dot{\phi}_R$ be the signed left and right wheel angular speeds in rad s^{-1} , where positive angular speed rolls the corresponding wheel forward.

The software implements

$$v = \frac{r}{2} (\dot{\phi}_R + \dot{\phi}_L), \quad \omega = \frac{r}{L} (\dot{\phi}_R - \dot{\phi}_L),$$

where v is signed body-forward speed in m s^{-1} and ω is signed counter-clockwise yaw rate in rad s^{-1} .

6.2.1 Software contract

The `wheel_to_body` operation accepts four finite scalar inputs: left wheel angular speed, right wheel angular speed, wheel radius, and track width. The wheel radius and track width must be strictly positive. Invalid inputs raise `ValueError`; the operation does not clamp values or substitute default geometry.

The operation returns an immutable `BodyVelocity` value containing `forward_speed_m_s` and `yaw_rate_rad_s`. It is deterministic, does not mutate its inputs, and does not depend on ROS state, timestamps, configuration files, or hidden global state.

6.2.2 Simplified reference snippet

The following equation-focused snippet shows how the mathematical factors map to Python expressions. It assumes that all inputs are finite and that `wheel_radius_m` and `track_width_m` are strictly positive; the production operation enforces those preconditions before performing the calculation.

```
@dataclass(frozen=True, slots=True)
class BodyVelocity:
    """Reduced planar body velocity in SI units."""

    forward_speed_m_s: float
    yaw_rate_rad_s: float

def wheel_to_body(
    *,
    left_angular_speed_rad_s: float,
    right_angular_speed_rad_s: float,
    wheel_radius_m: float,
    track_width_m: float,
) -> BodyVelocity:
    """Apply the wheel-to-body equations to validated inputs."""
    forward_speed_m_s = (
        wheel_radius_m / 2
        * (right_angular_speed_rad_s + left_angular_speed_rad_s)
    )
    yaw_rate_rad_s = (
        wheel_radius_m / track_width_m
        * (right_angular_speed_rad_s - left_angular_speed_rad_s)
    )

    return BodyVelocity(
        forward_speed_m_s=forward_speed_m_s,
        yaw_rate_rad_s=yaw_rate_rad_s,
    )
```

6.2.3 Invariants

- Equal wheel speeds produce zero yaw rate.
- Equal and opposite wheel speeds produce zero forward speed.
- Swapping the left and right wheel speeds preserves forward speed and reverses yaw rate.
- Two zero wheel speeds produce zero forward speed and zero yaw rate.
- Positive right-wheel speed relative to left-wheel speed produces positive counter-clockwise yaw.

6.3 2. Body-to-wheel velocity mapping

The `body_to_wheel` operation implements the inverse of the wheel-to-body mapping. Its inputs are signed body-forward speed v in m s^{-1} , signed counter-clockwise yaw rate ω in rad s^{-1} , wheel radius $r > 0$ in metres, and track width $L > 0$ in metres.

The required wheel angular speeds are

$$\dot{\phi}_L = \frac{v}{r} - \frac{L\omega}{2r}, \quad \dot{\phi}_R = \frac{v}{r} + \frac{L\omega}{2r},$$

where $\dot{\phi}_L$ and $\dot{\phi}_R$ are the signed left and right wheel angular speeds in rad s^{-1} .

6.3.1 Software contract

The operation accepts four finite scalar inputs. The wheel radius and track width must be strictly positive. Invalid inputs raise `ValueError`; the operation does not clamp values or substitute geometry.

The result is an immutable `WheelAngularSpeeds` value containing `left_angular_speed_rad_s` and `right_angular_speed_rad_s`.

6.3.2 Implementation mapping

The calculation separates forward and turning contributions:

```
@dataclass(frozen=True, slots=True)
class WheelAngularSpeeds:
    """Signed left and right wheel angular speeds in radians per second."""

    left_angular_speed_rad_s: float
    right_angular_speed_rad_s: float

def body_to_wheel(
    *,
    forward_speed_m_s: float,
    yaw_rate_rad_s: float,
    wheel_radius_m: float,
    track_width_m: float,
) -> WheelAngularSpeeds:
    """Apply the body-to-wheel equations to validated inputs."""
    forward_component_rad_s = forward_speed_m_s / wheel_radius_m
    turning_component_rad_s = (
        track_width_m * yaw_rate_rad_s / (2 * wheel_radius_m)
    )
    left_angular_speed_rad_s = (
        forward_component_rad_s - turning_component_rad_s
```



```

)
right_angular_speed_rad_s = (
    forward_component_rad_s + turning_component_rad_s
)

return WheelAngularSpeeds(
    left_angular_speed_rad_s=left_angular_speed_rad_s,
    right_angular_speed_rad_s=right_angular_speed_rad_s,
)

```

The forward component contributes equally to both wheels. The turning component is subtracted from the left wheel and added to the right wheel. Therefore, a positive yaw rate makes the right wheel turn faster than the left wheel.

6.3.3 Invariants and required tests

- Zero yaw rate produces equal wheel speeds of v/r .
- Zero forward speed produces equal-and-opposite wheel speeds.
- Reversing the yaw-rate sign swaps the left and right wheel speeds.
- Applying `body_to_wheel` and then `wheel_to_body` recovers the original v and ω within the declared floating-point tolerance.
- Non-finite inputs and non-positive geometry raise `ValueError`.

6.4 3. Exact constant-input pose integration

Let the initial planar pose be (x_k, y_k, θ_k) , where x_k and y_k are the world-frame coordinates of the axle midpoint in metres and θ_k is the unwrapped counter-clockwise heading in radians.

Assume that forward speed v in ms^{-1} and yaw rate ω in rads^{-1} remain constant over a time step $\Delta t > 0$ in seconds. Define the heading change

$$\Delta\theta = \omega\Delta t.$$

The exact update is

$$\begin{aligned}
 x_{k+1} &= x_k + v\Delta t \operatorname{sinc}\left(\frac{\Delta\theta}{2}\right) \cos\left(\theta_k + \frac{\Delta\theta}{2}\right), \\
 y_{k+1} &= y_k + v\Delta t \operatorname{sinc}\left(\frac{\Delta\theta}{2}\right) \sin\left(\theta_k + \frac{\Delta\theta}{2}\right), \\
 \theta_{k+1} &= \theta_k + \Delta\theta,
 \end{aligned}$$

where the unnormalised sinc function is

$$\operatorname{sinc}(z) = \begin{cases} \frac{\sin z}{z}, & z \neq 0, \\ 1, & z = 0. \end{cases}$$

The update is exact for the ideal continuous kinematic model only when v and ω remain constant during the time step. It does not account for wheel slip, changing commands, actuator dynamics, or model error.

6.4.1 Software contract

The `integrate_exact` operation accepts an immutable `PlanarPose`, finite forward speed, finite yaw rate, and a finite positive time step. A non-finite input or non-positive time step raises `ValueError`.

The operation returns a new `PlanarPose`. It does not mutate the initial pose or wrap the resulting heading into a fixed angular interval.

6.4.2 Numerically stable implementation mapping

The implementation avoids evaluating v/ω , which would be undefined when $\omega = 0$. Instead, it uses the sinc form:

```
@dataclass(frozen=True, slots=True)
class PlanarPose:
    """Pose of the robot's axle midpoint in the world frame.

    Attributes:
        position_x_m: World-frame x-coordinate in metres.
        position_y_m: World-frame y-coordinate in metres.
        heading_rad: Counter-clockwise heading from the world-frame
                     positive x-axis in radians.
    """

    position_x_m: float
    position_y_m: float
    heading_rad: float

def _sinc(value: float) -> float:
    """Return the unnormalised sinc function, including its value at zero."""
    if value == 0.0:
        return 1.0

    return sin(value) / value

heading_change_rad = yaw_rate_rad_s * time_step_s
half_heading_change_rad = heading_change_rad / 2
midpoint_heading_rad = pose.heading_rad + half_heading_change_rad

signed_displacement_m = (
    forward_speed_m_s
```

```

    * time_step_s
    * _sinc(half_heading_change_rad)
)

next_pose = PlanarPose(
    position_x_m=(
        pose.position_x_m
        + signed_displacement_m * cos(midpoint_heading_rad)
    ),
    position_y_m=(
        pose.position_y_m
        + signed_displacement_m * sin(midpoint_heading_rad)
    ),
    heading_rad=pose.heading_rad + heading_change_rad,
)

```

When $\omega = 0$, we have $\Delta\theta = 0$ and $\text{sinc}(0) = 1$. The same implementation therefore reduces smoothly to straight motion without a separate division by zero.

6.4.3 Invariants and required tests

- Zero yaw rate produces the exact straight-motion update.
- Zero forward speed changes only the heading.
- Constant nonzero forward speed and yaw rate produce constant-curvature motion.
- The input pose remains unchanged.
- The output heading remains unwrapped.
- Non-finite inputs raise `ValueError`.
- A zero or negative time step raises `ValueError`.

6.5 4. Forward Euler pose integration

Forward Euler approximates motion over a time step $\Delta t > 0$ by evaluating the pose derivative at the beginning of the step. For initial pose (x_k, y_k, θ_k) , constant forward speed v , and constant yaw rate ω , the update is

$$\begin{aligned}
 x_{k+1}^E &= x_k + \Delta t v \cos \theta_k, \\
 y_{k+1}^E &= y_k + \Delta t v \sin \theta_k, \\
 \theta_{k+1}^E &= \theta_k + \Delta t \omega.
 \end{aligned}$$

The superscript E identifies a Forward Euler approximation. Position is calculated entirely from the beginning-of-step heading θ_k ; the changing heading within the step does not affect that step's position increment.

6.5.1 Software contract

The `integrate_forward_euler` operation accepts the same inputs and validation rules as `integrate_exact`: an immutable `PlanarPose`, finite forward speed, finite yaw rate, and a finite positive time step.

The operation returns a new `PlanarPose`, does not mutate the input pose, and leaves the heading unwrapped. Invalid inputs raise `ValueError`.

6.5.2 Implementation mapping

```
next_pose = PlanarPose(
    position_x_m=(
        pose.position_x_m
        + time_step_s * forward_speed_m_s * cos(pose.heading_rad)
    ),
    position_y_m=(
        pose.position_y_m
        + time_step_s * forward_speed_m_s * sin(pose.heading_rad)
    ),
    heading_rad=(
        pose.heading_rad + time_step_s * yaw_rate_rad_s
    ),
)
```

6.5.3 Exact and approximate cases

When $\omega = 0$, the heading remains constant. The world-frame velocity is therefore constant, so one Forward Euler step produces the exact straight-motion position.

When $v = 0$, position remains unchanged and the heading update is exact because $\dot{\theta} = \omega$ is constant.

When both $v \neq 0$ and $\omega \neq 0$, the true path curves during the time step. Forward Euler instead follows the tangent defined by θ_k , so its one-step position is generally approximate.

For repeated integration over a fixed duration, Forward Euler has global error $O(\Delta t)$. In this regime, halving Δt should approximately halve the accumulated error.

6.5.4 Invariants and required tests

- Straight constant-speed motion agrees with the exact update.
- Pure rotation changes only the heading.
- A turning step uses the beginning-of-step heading.
- Smaller time steps reduce accumulated turning-position error.
- Halving the time step approximately halves global error in the first-order regime.
- Non-finite inputs and non-positive time steps raise `ValueError`.

7 ROS 2 Package Development Workflow

7.1 Goal

This workflow creates, builds, runs, and tests a small ROS 2 Python package from scratch. It uses `my_robot_package` as an example package name and `my_node` as an example executable name; replace them in a real project.

A **workspace** holds packages developed and built together. A **ROS 2 package** combines source code with the metadata needed for building, installing, and discovery. An **executable** is a program installed by a package and started with `ros2 run`.

The generated `my_node` executable only prints a message. It demonstrates the package workflow but does not yet communicate with other ROS 2 nodes.

7.2 Step 1 — Source the ROS 2 underlay

For Zsh, run:

```
source /opt/ros/jazzy/setup.zsh
```

For Bash, run:

```
source /opt/ros/jazzy/setup.bash
```

Use the script matching the current shell.

What happens: `/opt/ros/jazzy` is the ROS 2 Jazzy **underlay**, meaning the existing installation on which the new workspace depends. Its setup script adds ROS installation prefixes, Python packages, and executables to the current shell's search environment.

The command uses `source` because the environment of the current shell must change. If the script ran in a separate **child process**, meaning a process started by the shell, its environment changes would disappear when that child exited.

Verify the environment:

```
printf '%s\n' "$ROS_DISTRO"
command -v ros2
command -v colcon
```

The expected observations are `jazzy` and executable paths for `ros2` and `colcon`.

7.3 Step 2 — Create a workspace

For a new standalone workspace, run:

```
mkdir -p ~/ros_ws/src
cd ~/ros_ws
```

The workspace develops into:

```
ros_ws/
  src/      # Editable source
  build/    # Generated build workspace
  install/  # Generated runnable workspace image
  log/      # Generated build and test records
```

Only `src/` exists initially. `colcon` creates the other directories. Edit `src/`, not the generated files in `build/`, `install/`, or `log/`.

7.4 Step 3 — Create a Python package

From the workspace root, run:

```
cd src
ros2 pkg create \
  --build-type ament_python \
  --license Apache-2.0 \
  --node-name my_node \
  my_robot_package
cd ..
```

Run this command only when the package does not already exist.

What happens: `ros2 pkg create` generates `src/my_robot_package/`.

- `ament_python` selects Python packaging through `setuptools`.
- `Apache-2.0` is recorded in `package.xml`, and a matching `LICENSE` file is created.
- `--node-name my_node` creates a minimal Python executable and its `setup.py` entry point.

Omit `--node-name` when creating a library-only package.

For comparison, create a C++ package with:

```
ros2 pkg create \
  --build-type ament_cmake \
  --license Apache-2.0 \
  robot_controller
```

The common build types are `ament_python` for Python with `setuptools` and `ament_cmake` for C++ with CMake.

7.5 Step 4 — Inspect the generated package

```
src/my_robot_package/  
  LICENSE  
  package.xml  
  setup.py  
  setup.cfg  
  resource/  
    my_robot_package  
  my_robot_package/  
    __init__.py  
    my_node.py  
  test/  
    test_copyright.py  
    test_flake8.py  
    test_pep257.py
```

The outer `my_robot_package/` is the ROS package root. The inner directory is the Python import package used by `import my_robot_package`; the repeated name does not indicate nested repositories.

File or directory	Purpose
<code>package.xml</code>	Declares package identity, licence, dependencies, and build type
<code>setup.py</code>	Selects Python packages, data files, and executables to install
<code>setup.cfg</code>	Selects the ROS-compatible destination for Python executables
<code>resource/my_robot_package</code>	Registers the installed package in the ament resource index
<code>my_robot_package/</code>	Contains importable Python source
<code>test/</code>	Contains checks run by <code>pytest</code> and <code>colcon test</code>

The resource marker is intentionally empty. Its filename identifies the package.

7.6 Step 5 — Complete the package metadata

Replace placeholder descriptions and maintainer details in `package.xml` and `setup.py`. Keep their package name, version, and licence consistent. The `package.xml` export tells `colcon` which build integration to use:

```
<export>  
  <build_type>ament_python</build_type>  
</export>
```

A concise `setup.py` for the example is:

```
from setuptools import find_packages, setup

package_name = "my_robot_package"

setup(
    name=package_name,
    version="0.0.0",
    packages=find_packages(exclude=["test"]),
    data_files=[
        (
            "share/ament_index/resource_index/packages",
            [f"resource/{package_name}"],
        ),
        (f"share/{package_name}", ["package.xml"]),
    ],
    install_requires=["setuptools"],
    zip_safe=True,
    maintainer="Your Name",
    maintainer_email="you@example.com",
    description="Example ROS 2 Python package.",
    license="Apache-2.0",
    entry_points={
        "console_scripts": [
            "my_node = my_robot_package.my_node:main",
        ],
    },
)
```

The important connections are:

- `packages` installs importable Python packages.
- `data_files` installs the resource marker and `package.xml` where ROS expects them.
- `console_scripts` creates an executable that calls a Python function.

The entry point `my_node = my_robot_package.my_node:main` maps the executable name `my_node` to the function `main()` in the Python module `my_robot_package.my_node`.

The matching `setup.cfg` is:

```
[develop]
script_dir=$base/lib/my_robot_package

[install]
install_scripts=$base/lib/my_robot_package
```

Here, `$base` is the package installation prefix. `setup.cfg` specifies that executable scripts belong under `lib/my_robot_package`, where `ros2 run` looks for them. It does not create an executable by itself; the `console_scripts` entry does that.

7.7 Step 6 — Resolve dependencies and run fast tests

Resolve dependencies declared by packages under `src/`:

```
rosdep install \  
  --from-paths src \  
  --ignore-src \  
  --rosdistro jazzy \  
  -r \  
  -y
```

`rosdep` ignores dependencies supplied by source packages in the workspace, maps the remaining dependency keys to operating-system packages, and installs missing packages.

Run fast tests directly against the source tree:

```
python3 -m pytest \  
  src/my_robot_package/test -q
```

Source-level tests provide fast feedback but do not verify ROS metadata or the installed layout. Continue only when they pass; the count and duration depend on the package.

7.8 Step 7 — Build and install the package

From the workspace root, run:

```
colcon build \  
  --packages-select my_robot_package \  
  --symlink-install
```

What happens: `colcon` discovers packages under `src/`, reads their manifests, orders them by declared dependencies, and selects the appropriate build integration. `ament_python` then invokes `setuptools` using `setup.py` and `setup.cfg`.

The logical installed layout is:

```
install/my_robot_package/  
  lib/  
    python3.12/  
      site-packages/  
        my_robot_package/  
          __init__.py  
          my_node.py  
        my_robot_package/  
          my_node  
      share/
```

```
ament_index/  
  resource_index/  
    packages/  
      my_robot_package  
my_robot_package/  
  package.xml
```

The Python version in the path depends on the system.

- `site-packages/my_robot_package` supplies importable Python code.
- `lib/my_robot_package/my_node` supplies the executable used by `ros2 run`.
- The resource marker registers the package with ROS.
- The installed `package.xml` supplies package metadata.

ROS 2 needs this standard runtime layout to locate packages, executables, metadata, launch files, configuration, Python imports, and dependency environment hooks.

7.8.1 What `--symlink-install` changes

Supported files are linked back to editable source rather than copied:

```
install/.../my_node.py  
  ↓ symbolic link  
src/my_robot_package/my_robot_package/my_node.py
```

Python packages may use an `.egg-link` instead of the copied directory shown in the logical tree. Consequently, edits to existing linked Python files may appear without rebuilding, but `install/` is still needed for metadata, executables, and environment scripts.

Rebuild after changing `setup.py`, `setup.cfg`, `console_scripts`, installed data, dependencies, CMake configuration, or compiled C++ code. Rebuild after structural changes and before formal verification to avoid stale state.

7.9 Step 8 — Source the overlay and run the package

After a successful build, source the script matching the current shell.

```
source install/setup.zsh
```

```
source install/setup.bash
```

This command does not install anything. It adds the workspace's `install/` tree as an **overlay** on the ROS underlay by updating variables such as `AMENT_PREFIX_PATH`, `PYTHONPATH`, and `PATH`.

Sourcing affects only the current shell. A new terminal must source the ROS underlay and then the workspace overlay again.

Verify package discovery, Python import, and executable discovery:

```
ros2 pkg prefix my_robot_package
python3 -c "import my_robot_package"
ros2 run my_robot_package my_node
```

The final command should print:

```
Hi from my_robot_package.
```

`ros2 pkg prefix` queries the ament resource index. `ros2 run` uses the first argument as the package name and the second as the executable name, then starts `lib/my_robot_package/my_node`.

7.10 Step 9 — Run tests through colcon

```
colcon test --packages-select my_robot_package
colcon test-result --verbose
```

`colcon test` runs the package's configured tests and stores structured results under the build tree. `colcon test-result` aggregates them. A completed command is not necessarily successful; verification passes only when the summary reports no errors or failures.

7.11 Step 10 — Repeat the daily development loop

From the workspace root, repeat:

```
# 1. Edit source.
vim src/my_robot_package/my_robot_package/my_node.py

# 2. Run fast tests.
python3 -m pytest src/my_robot_package/test -q

# 3. Build and install.
colcon build \
  --packages-select my_robot_package \
  --symlink-install

# 4. Reload the overlay.
source install/setup.zsh

# 5. Run the executable.
ros2 run my_robot_package my_node

# 6. Run package-level verification.
colcon test --packages-select my_robot_package
colcon test-result --verbose
```

Use `source install/setup.bash` in Bash.

7.12 Common failure checks

Symptom	Check
<code>ros2: command not found</code>	Source the ROS underlay in the current shell
Package absent from <code>colcon list</code>	Check its location under <code>src/</code> and validate <code>package.xml</code>
Python package is not installed	Check <code>__init__.py</code> and the <code>packages</code> argument in <code>setup.py</code>
<code>ros2 pkg prefix</code> cannot find the package	Build and source the overlay
<code>ros2 run</code> cannot find the executable	Check <code>console_scripts</code> and <code>setup.cfg</code> , then rebuild and source
Test status is unclear	Run <code>colcon test-result --verbose</code>

7.13 References

- Open Robotics, [“Developing a ROS 2 package,”](#) *ROS 2 Jazzy Documentation*.
- Open Robotics, [“Using Python Packages with ROS 2,”](#) *ROS 2 Jazzy Documentation*.
- colcon, [“What is a Workspace?”](#) *colcon Documentation*.

8 Planar Rigid-Body Motion: $SE(2)$ and Twists

8.1 Why this chapter matters

The differential-drive pose $\mathbf{q} = [x \ y \ \theta]^\top$ is convenient for storing three coordinates, but it does not by itself expose how rigid motions compose, how the same motion is expressed in different frames, or why constant body velocity produces the exact curved-motion update derived previously. This chapter develops the planar rigid-motion structure needed to answer those questions. It begins by distinguishing a pose from its coordinates and representing a planar pose by a homogeneous transformation in the special Euclidean group $SE(2)$.

8.2 Prerequisites and assumptions

The reader should understand matrices and matrix multiplication from the [linear algebra foundations chapter](#), coordinate frames and rotation matrices from the [geometry and coordinate frames chapter](#), and planar pose notation from the [differential-drive kinematics chapter](#).

All frames in this learning block are right-handed planar frames embedded in three-dimensional physical space with a common vertical axis. Frame $\{w\}$ is the fixed world frame and frame $\{b\}$ is attached rigidly to the robot body. Distances are measured in metres and angles in radians.

8.3 1. Pose coordinates and rigid transformations

8.3.1 1.1 A pose is a relative geometric relationship

A planar rigid body's **pose** specifies both its position and its orientation relative to a reference frame. The word *relative* is essential: saying that a robot is at position $(2, 1)$ with heading $\pi/2$ is incomplete until the reference frame and sign convention are known.

Let O_w and O_b denote the origins of frames $\{w\}$ and $\{b\}$. The position of O_b relative to O_w , expressed in world-frame coordinates, is

$${}^w\mathbf{p}_b = \begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2,$$

where x and y are measured in metres. The orientation of frame $\{b\}$ relative to frame $\{w\}$ is represented by the rotation matrix $\mathbf{R}_{wb}(\theta) \in \mathbb{R}^{2 \times 2}$:

$$\mathbf{R}_{wb}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

The subscript order wb means that $\mathbf{R}_{wb}$ maps vector coordinates from frame $\{b\}$ to frame $\{w\}$. The pose of $\{b\}$ relative to $\{w\}$ can therefore be described by the pair

$$(\mathbf{R}_{wb}, {}^w\mathbf{p}_b).$$

This pair separates orientation from position. A homogeneous transformation will combine them into one matrix without confusing their different roles.

8.3.2 1.2 Planar rotations and $SO(2)$

The set of valid planar rotation matrices is called the **special orthogonal group in two dimensions**, written $SO(2)$ and defined by

$$SO(2) := \{\mathbf{R} \in \mathbb{R}^{2 \times 2} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{I}_2, \quad \det(\mathbf{R}) = 1\}.$$

The word *orthogonal* refers to $\mathbf{R}^\top \mathbf{R} = \mathbf{I}_2$, which means that the columns form an orthonormal basis and that $\mathbf{R}^{-1} = \mathbf{R}^\top$. The word *special* adds the condition $\det(\mathbf{R}) = 1$, excluding reflections whose determinant is -1 .

The word *group* means that the set is equipped with an operation satisfying four properties: closure, associativity, an identity element, and an inverse for every element. For $SO(2)$, the operation is matrix multiplication.

Closure means that multiplying any two elements of $SO(2)$ produces another element of $SO(2)$. If $\mathbf{R}(\alpha)$ and $\mathbf{R}(\beta)$ represent planar rotations through angles α and β , then

$$\mathbf{R}(\alpha)\mathbf{R}(\beta) = \mathbf{R}(\alpha + \beta) \in SO(2).$$

Geometrically, performing one planar rotation and then another still produces a valid planar rotation; it does not introduce scaling, shear, or reflection.

Associativity means that the placement of parentheses does not change the result of composing three rotations:

$$(\mathbf{R}_1 \mathbf{R}_2) \mathbf{R}_3 = \mathbf{R}_1 (\mathbf{R}_2 \mathbf{R}_3).$$

Associativity changes only the grouping, not the order, of the matrices. It should not be confused with commutativity, which would permit the order to be exchanged. The identity element is $\mathbf{I}_2$, and every $\mathbf{R} \in SO(2)$ has the inverse $\mathbf{R}^\top \in SO(2)$.

Every planar rotation matrix can be written as $\mathbf{R}(\theta)$ for some $\theta \in \mathbb{R}$. The angle is not unique because

$$\mathbf{R}(\theta + 2\pi n) = \mathbf{R}(\theta), \quad n \in \mathbb{Z},$$

where $\mathbb{Z}$ is the set of integers. Thus, θ is a **coordinate** used to describe an orientation, while $\mathbf{R}(\theta)$ represents the **orientation** itself. Angles that differ by an integer multiple of 2π describe the same element of $SO(2)$.

8.3.3 1.3 Transforming a point by rotation and translation

Let P be a geometric point fixed to the robot. Its position relative to O_b , expressed in body-frame coordinates, is ${}^b\mathbf{p}_P \in \mathbb{R}^2$ and is measured in metres. Its world-frame position is obtained by first rotating its body-frame displacement and then adding the world-frame position of the body origin:

$$\boxed{{}^w\mathbf{p}_P = \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b}.$$

This equation follows by tracing the geometric displacement from O_w to P . First travel from O_w to O_b by ${}^w\mathbf{p}_b$. Then travel from O_b to P by ${}^b\mathbf{p}_P$, but rotate that second displacement with $\mathbf{R}_{wb}$ so that both terms are expressed in world-frame coordinates before adding them.

A free vector, such as a displacement or velocity vector, has direction and magnitude but no fixed origin. It therefore changes coordinates using rotation alone:

$${}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v}.$$

Translation appears in the point equation because a point is located relative to a frame origin. It does not appear in the free-vector equation because a free vector is not attached to an origin.

8.3.4 1.4 Homogeneous transformations and $SE(2)$

The point mapping in Section 1.3 contains both a matrix multiplication and a vector addition. Homogeneous coordinates package these two operations into one matrix multiplication. Define the homogeneous coordinate column of point P by appending a 1:

$${}^b\overline{\mathbf{p}}_P := \begin{bmatrix} {}^b\mathbf{p}_P \\ 1 \end{bmatrix} \in \mathbb{R}^3.$$

The overline identifies a homogeneous coordinate representation, not a different geometric point. Define the **homogeneous transformation** from frame $\{b\}$ to frame $\{w\}$ by

$$\boxed{\mathbf{T}_{wb} := \begin{bmatrix} \mathbf{R}_{wb} & {}^w\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix}} \in \mathbb{R}^{3 \times 3},$$

where $\mathbf{0}_2 = [0 \ 0]^\top \in \mathbb{R}^2$. The subscript order means that $\mathbf{T}_{wb}$ maps homogeneous point coordinates from frame $\{b\}$ to frame $\{w\}$:

$$\begin{aligned} {}^w\overline{\mathbf{p}}_P &= \mathbf{T}_{wb} {}^b\overline{\mathbf{p}}_P \\ &= \begin{bmatrix} \mathbf{R}_{wb} & {}^w\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \begin{bmatrix} {}^b\mathbf{p}_P \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b \\ 1 \end{bmatrix}. \end{aligned}$$

The upper two entries reproduce the point-transformation equation. Appending 1 causes the translation column to contribute to the result. Thus, $\mathbf{T}_{wb}$ rotates the point's body-frame displacement into the world frame and then adds the world position of the body origin.

Using ${}^w\mathbf{p}_b = [x \ y]^\top$ and $\mathbf{R}_{wb} = \mathbf{R}_{wb}(\theta)$ gives the component form

$$\mathbf{T}_{wb} = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix}.$$

The rotation block is dimensionless, while the first two entries of the translation column have units of metres. The final row is an algebraic device that permits rotation and translation to be applied by one matrix multiplication; it does not represent another physical spatial dimension.

To preserve the rotation-only mapping for a free vector, define its homogeneous representation by appending a 0:

$${}^b\overline{\mathbf{v}} := \begin{bmatrix} {}^b\mathbf{v} \\ 0 \end{bmatrix}.$$

Then

$$\mathbf{T}_{wb} \begin{bmatrix} {}^b\mathbf{v} \\ 0 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{wb} {}^b\mathbf{v} \\ 0 \end{bmatrix}.$$

Appending 0 prevents the translation column from contributing. This algebraic distinction encodes the geometric fact that changing a point's reference frame requires rotation and translation, whereas changing the coordinates of a free vector requires only rotation.

The set of all planar homogeneous transformations is the **special Euclidean group in two dimensions**:

$$SE(2) := \left\{ \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \mid \mathbf{R} \in SO(2), \quad \mathbf{p} \in \mathbb{R}^2 \right\}.$$

Here, *Euclidean* indicates rigid motions that preserve Euclidean distances and angles. Matrix multiplication is the group operation, the 3×3 identity $\mathbf{I}_3$ is the identity motion, and every element has an inverse that reverses the coordinate mapping. Section 1.6 derives that inverse explicitly.

8.3.5 1.5 Composing transformations

Introduce a third frame $\{c\}$. Let $\mathbf{T}_{ab} \in SE(2)$ map coordinates from frame $\{b\}$ to frame $\{a\}$, and let $\mathbf{T}_{bc} \in SE(2)$ map coordinates from frame $\{c\}$ to frame $\{b\}$. Applying the two mappings to a point gives

$${}^a\overline{\mathbf{p}}_P = \mathbf{T}_{ab} (\mathbf{T}_{bc} {}^c\overline{\mathbf{p}}_P) = (\mathbf{T}_{ab} \mathbf{T}_{bc}) {}^c\overline{\mathbf{p}}_P.$$

Therefore, the transform that maps directly from frame $\{c\}$ to frame $\{a\}$ is

$$\boxed{\mathbf{T}_{ac} = \mathbf{T}_{ab} \mathbf{T}_{bc}}.$$

The adjacent frame label b identifies the intermediate frame. Expanding the block multiplication gives

$$\begin{aligned} \mathbf{T}_{ab} \mathbf{T}_{bc} &= \begin{bmatrix} \mathbf{R}_{ab} & {}^a\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_{bc} & {}^b\mathbf{p}_c \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}_{ab} \mathbf{R}_{bc} & \mathbf{R}_{ab} {}^b\mathbf{p}_c + {}^a\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix}. \end{aligned}$$

Consequently,

$$\boxed{\mathbf{R}_{ac} = \mathbf{R}_{ab} \mathbf{R}_{bc}, \quad {}^a\mathbf{p}_c = \mathbf{R}_{ab} {}^b\mathbf{p}_c + {}^a\mathbf{p}_b}.$$

The translation vectors cannot be added directly until they are expressed in the same frame. The product first rotates ${}^b\mathbf{p}_c$ into frame $\{a\}$ and only then adds ${}^a\mathbf{p}_b$.

8.3.6 1.6 Inverting a transformation

The inverse transform $\mathbf{T}_{ba} = \mathbf{T}_{ab}^{-1}$ must undo both the translation and the rotation. Starting from the point mapping

$${}^a\mathbf{p}_P = \mathbf{R}_{ab} {}^b\mathbf{p}_P + {}^a\mathbf{p}_b,$$

subtract ${}^a\mathbf{p}_b$ and premultiply by $\mathbf{R}_{ab}^{-1} = \mathbf{R}_{ab}^\top$:

$${}^b\mathbf{p}_P = \mathbf{R}_{ab}^\top ({}^a\mathbf{p}_P - {}^a\mathbf{p}_b).$$

Distributing the rotation gives

$${}^b\mathbf{p}_P = \mathbf{R}_{ab}^\top {}^a\mathbf{p}_P - \mathbf{R}_{ab}^\top {}^a\mathbf{p}_b.$$

Therefore,

$$\boxed{\mathbf{T}_{ab}^{-1} = \mathbf{T}_{ba} = \begin{bmatrix} \mathbf{R}_{ab}^\top & -\mathbf{R}_{ab}^\top {}^a\mathbf{p}_b \\ \mathbf{0}_2^\top & 1 \end{bmatrix}}.$$

The inverse translation is not generally $-{}^a\mathbf{p}_b$ because that column is expressed in frame $\{a\}$. It must first be negated and then expressed in frame $\{b\}$ by $\mathbf{R}_{ab}^\top$.

8.3.7 1.7 Pose coordinates versus the pose itself

The familiar column

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

has the following meaning:

- $x \in \mathbb{R}$ is the x_w -coordinate of the body origin O_b , expressed in the world frame and measured in metres;
- $y \in \mathbb{R}$ is the y_w -coordinate of O_b , expressed in the world frame and measured in metres; and
- $\theta \in \mathbb{R}$ is the counter-clockwise orientation of body frame $\{b\}$ relative to world frame $\{w\}$, measured in radians.

Therefore, the position of the body origin relative to the world origin, expressed in world-frame coordinates, is

$${}^w\mathbf{p}_b = \begin{bmatrix} x \\ y \end{bmatrix}.$$

The notation ${}^w\mathbf{p}_b$ means “the position of the body origin O_b , expressed in world-frame coordinates.” Thus, $\mathbf{q}$ is a coordinate description of the pose of frame $\{b\}$ relative to frame $\{w\}$. The same pose can be represented by $\mathbf{T}_{wb} \in SE(2)$:

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \mapsto \mathbf{T}_{wb}(\mathbf{q}) = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix}.$$

The coordinate column is useful for storage, plotting, and differential equations, but it is not an ordinary geometric vector. Its components represent different physical quantities, θ and $\theta + 2\pi n$ represent the same orientation, and componentwise addition does not in general reproduce rigid-motion composition. Transformations should be composed by matrix multiplication with explicit frame labels.

8.3.8 1.8 Worked example

Suppose the body origin is at

$${}^w\mathbf{p}_b = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \text{ m}$$

and the body heading is $\theta = \pi/2$ rad. A point P fixed one metre along the body $+x_b$ -axis has body-frame coordinates

$${}^b\mathbf{p}_P = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ m.}$$

The rotation and homogeneous transformation are

$$\mathbf{R}_{wb} \left(\frac{\pi}{2} \right) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{T}_{wb} = \begin{bmatrix} 0 & -1 & 2 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

Applying the transform gives

$$\begin{aligned} {}^w\bar{\mathbf{p}}_P &= \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P \\ &= \begin{bmatrix} 0 & -1 & 2 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}. \end{aligned}$$

Thus, ${}^w\mathbf{p}_P = [2 \ 2]^\top$ m. The one-metre body-frame displacement is rotated into the world $+y_w$ -direction and then added to the body origin at $[2 \ 1]^\top$ m.

8.4 Quick test A: pose and $SE(2)$

1. What geometric information is contained in the pair $(\mathbf{R}_{wb}, {}^w\mathbf{p}_b)$, and what does the subscript order wb mean?
2. Define $SO(2)$. What do its orthogonality and determinant conditions exclude or guarantee?
3. Why do θ and $\theta + 2\pi$ describe the same orientation?
4. Starting from the block definitions of $\mathbf{T}_{ab}$ and $\mathbf{T}_{bc}$, derive the translation column of $\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc}$.
5. Why is the inverse translation $-\mathbf{R}_{ab}^\top {}^a\mathbf{p}_b$ rather than simply $-{}^a\mathbf{p}_b$?
6. A body frame has world position $[3 \ 2]^\top$ m and heading $-\pi/2$ rad. Find the world coordinates of a point whose body coordinates are $[0.5 \ 0]^\top$ m.
7. Explain why $\mathbf{q}_1 + \mathbf{q}_2$ is not generally the composition of the two poses represented by $\mathbf{q}_1$ and $\mathbf{q}_2$.
8. Name two transformations that cannot be represented by an element of $SE(2)$ and explain why.

8.4.1 Answers and hints

1. $\mathbf{R}_{wb}$ describes the orientation of frame $\{b\}$ relative to frame $\{w\}$, while ${}^w\mathbf{p}_b$ locates O_b relative to O_w using world coordinates. The order wb means that the rotation maps coordinate columns from $\{b\}$ to $\{w\}$.
2. $SO(2) = \{\mathbf{R} \in \mathbb{R}^{2 \times 2} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{I}_2, \det \mathbf{R} = 1\}$. Orthogonality preserves lengths and angles and gives $\mathbf{R}^{-1} = \mathbf{R}^\top$; determinant +1 preserves orientation and excludes reflections.

3. Sine and cosine are 2π -periodic, so every entry of $\mathbf{R}(\theta + 2\pi)$ equals the corresponding entry of $\mathbf{R}(\theta)$.
4. Block multiplication gives ${}^a\mathbf{p}_c = \mathbf{R}_{ab} {}^b\mathbf{p}_c + {}^a\mathbf{p}_b$. The first term rotates the b -frame translation into frame $\{a\}$ before the two translations are added.
5. The reversed displacement must be expressed in frame $\{b\}$, whereas ${}^a\mathbf{p}_b$ is expressed in frame $\{a\}$. Premultiplication by $\mathbf{R}_{ab}^T$ performs the necessary coordinate change.
6. $\mathbf{R}_{wb}(-\pi/2)[0.5 \ 0]^T = [0 \ -0.5]^T$ m, so ${}^w\mathbf{p}_P = [3 \ 1.5]^T$ m.
7. Rigid-motion composition must account for the frame in which each translation is expressed and for the order of rotations and translations. Componentwise addition does neither, and angular coordinates are periodic.
8. Examples include nonuniform scaling, shear, and elastic deformation. They do not preserve every distance and angle, so they are not rigid motions and do not belong to $SE(2)$.

8.5 2. Passive coordinate changes and active rigid motions

The same numerical rotation or homogeneous-transformation matrix can appear in different geometric questions. Correct interpretation requires identifying what remains fixed, what changes, and the frame in which every coordinate column is expressed.

8.5.1 2.1 Passive coordinate change

A **passive coordinate change** keeps the geometric point, vector, or rigid body unchanged and changes only the coordinate frame used to describe it. The word *passive* indicates that nothing physically moves.

For example, let the geometric point P remain fixed while its coordinates are changed from frame $\{b\}$ to frame $\{w\}$. The passive point-coordinate transformation is

$${}^w\overline{\mathbf{p}}_P = \mathbf{T}_{wb} {}^b\overline{\mathbf{p}}_P.$$

The two coordinate columns ${}^b\overline{\mathbf{p}}_P$ and ${}^w\overline{\mathbf{p}}_P$ contain different numbers in general, but both describe the same geometric point P . The transform $\mathbf{T}_{wb}$ accounts for the position and orientation of frame $\{b\}$ relative to frame $\{w\}$.

For a free vector $\mathbf{v}$, translation is irrelevant and only the rotation block is used:

$${}^w\mathbf{v} = \mathbf{R}_{wb} {}^b\mathbf{v}.$$

Suppose frame $\{b\}$ is rotated counter-clockwise by $\pi/2$ relative to frame $\{w\}$ and the vector lies along $+x_b$. Its world-frame coordinates are

$$\begin{aligned} {}^w\mathbf{v} &= \mathbf{R}_{wb} {}^b\mathbf{v} \\ &= \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \end{aligned}$$

The vector has not rotated physically. It is the same geometric vector described once using the axes of $\{b\}$ and once using the axes of $\{w\}$.

8.5.2 2.2 Active rigid-body motion

An **active transformation** keeps the reference frame fixed and moves a geometric point, vector, or rigid body. The initial point P and final point P' are different geometric points, but both coordinate columns are expressed in the same frame.

Let $\mathbf{R}_\Delta \in SO(2)$ be an applied planar rotation and let $\mathbf{d}_\Delta \in \mathbb{R}^2$ be an applied translation expressed in world-frame coordinates and measured in metres. Define the active rigid-motion operator

$$\mathbf{T}_\Delta := \begin{bmatrix} \mathbf{R}_\Delta & \mathbf{d}_\Delta \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \in SE(2).$$

Applied to point P , it produces the new point P' according to

$$\boxed{{}^w\overline{\mathbf{p}}_{P'} = \mathbf{T}_\Delta {}^w\overline{\mathbf{p}}_P},$$

or, using ordinary two-dimensional coordinates,

$${}^w\mathbf{p}_{P'} = \mathbf{R}_\Delta {}^w\mathbf{p}_P + \mathbf{d}_\Delta.$$

Both sides are expressed in frame $\{w\}$. The point changes from P to P' , unlike a passive coordinate change, where the point remains P and the leading frame superscript changes.

For example, let $\mathbf{R}_\Delta = \mathbf{R}(\pi/2)$, $\mathbf{d}_\Delta = [2 \ 1]^\top$ m, and ${}^w\mathbf{p}_P = [1 \ 0]^\top$ m. Then

$${}^w\mathbf{p}_{P'} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} \text{ m}.$$

Here, the coordinates changed because the geometric point moved, not because a different coordinate frame was selected.

8.5.3 2.3 One matrix form, three interpretations

A homogeneous transformation may describe a relative pose, perform a passive coordinate change, or act as an active rigid-motion operator. The matrix arithmetic can be identical while the geometric meaning differs.

Interpretation	What changes?	Representative statement
Relative-pose description	Nothing is being operated on; the matrix records the pose of one frame relative to another.	$\mathbf{T}_{wb}$ describes frame $\{b\}$ relative to frame $\{w\}$.

Interpretation	What changes?	Representative statement
Passive coordinate change	The coordinate column and its frame label change; the geometric point remains fixed.	${}^w\bar{\mathbf{p}}_P = \mathbf{T}_{wb} {}^b\bar{\mathbf{p}}_P.$
Active rigid motion	The geometric point changes; the coordinate frame remains fixed.	${}^w\bar{\mathbf{p}}_{P'} = \mathbf{T}_\Delta {}^w\bar{\mathbf{p}}_P.$

The project notation reserves labelled transforms such as $\mathbf{T}_{wb}$ for relative poses and coordinate mappings. A symbol such as $\mathbf{T}_\Delta$ must be explicitly declared when the same matrix form is used as an active motion operator.

8.5.4 2.4 Composition order

Transformations act on column vectors from the left, so the rightmost matrix acts first. For a passive coordinate chain,

$${}^a\bar{\mathbf{p}}_P = \mathbf{T}_{ab}\mathbf{T}_{bc} {}^c\bar{\mathbf{p}}_P,$$

the rightmost transform $\mathbf{T}_{bc}$ first changes coordinates from frame $\{c\}$ to frame $\{b\}$, and $\mathbf{T}_{ab}$ then changes them from frame $\{b\}$ to frame $\{a\}$. Thus,

$$\mathbf{T}_{ac} = \mathbf{T}_{ab}\mathbf{T}_{bc}.$$

For two active motions $\mathbf{T}_1$ and $\mathbf{T}_2$,

$${}^w\bar{\mathbf{p}}_{P''} = \mathbf{T}_2\mathbf{T}_1 {}^w\bar{\mathbf{p}}_P,$$

motion $\mathbf{T}_1$ occurs first and motion $\mathbf{T}_2$ occurs second. Associativity permits the parentheses in a longer product to be regrouped, but it does not permit the matrix order to be reversed.

8.5.5 2.5 Why composition in $SE(2)$ is not commutative

Planar rotations alone commute, but planar rigid transformations generally do not because rotation changes the direction of a translation. Consider a translation of one metre along $+x_w$ and a counter-clockwise rotation of $\pi/2$ about O_w :

$$\mathbf{T}_D = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{T}_R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Translate first and rotate second:

$$\mathbf{T}_R \mathbf{T}_D = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

Rotate first and translate second:

$$\mathbf{T}_D \mathbf{T}_R = \begin{bmatrix} 0 & -1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The products differ. Applied to the world origin ${}^w\bar{\mathbf{p}}_O = [0 \ 0 \ 1]^T$, they give

$$\mathbf{T}_R \mathbf{T}_D {}^w\bar{\mathbf{p}}_O = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{T}_D \mathbf{T}_R {}^w\bar{\mathbf{p}}_O = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

Therefore,

$$\boxed{\mathbf{T}_R \mathbf{T}_D \neq \mathbf{T}_D \mathbf{T}_R}.$$

This noncommutativity is why a transformation sequence must state both the individual motions and their order.

8.5.6 2.6 Interpretation failures and engineering checks

- **Missing frame labels:** An equation such as $\mathbf{p}' = \mathbf{T}\mathbf{p}$ does not reveal whether $\mathbf{T}$ changes coordinates or moves the point. Declare the interpretation and frames.
- **Applying transformations in the wrong order:** For column-vector notation, transformations act from right to left. In $\mathbf{T}_2\mathbf{T}_1\mathbf{p}$, $\mathbf{T}_1$ acts first and $\mathbf{T}_2$ acts second. Verify an implementation using a point whose expected final location is easy to calculate.
- **Mixing coordinate frames:** Translation columns may be added only after they are expressed in the same frame.
- **Assuming wrong commutativity:** Reordering a rotation and translation generally changes the resulting pose.

8.6 Quick test B: interpretation and composition

1. Define a passive coordinate change. What changes, and what remains fixed?
2. Define an active rigid-body motion. What changes, and what remains fixed?
3. Explain the three interpretations of a homogeneous transformation used in this chapter.
4. In $\mathbf{T}_{ab}\mathbf{T}_{bc}{}^c\bar{\mathbf{p}}_P$, which matrix acts first, and what frame sequence does the product represent?
5. For active motions, what sequence is represented by $\mathbf{T}_2\mathbf{T}_1{}^w\bar{\mathbf{p}}_P$?
6. Using $\mathbf{T}_D$ and $\mathbf{T}_R$ from Section 2.5, explain geometrically why applying them to the origin in opposite orders gives different results.

7. Does the commutativity of planar rotations imply that all elements of $SE(2)$ commute? Explain.
8. A program receives an unlabelled matrix $\mathbf{T}$ and an unlabelled coordinate column $\mathbf{p}$. What information is missing before the multiplication can be interpreted safely?

8.6.1 Answers and hints

1. A passive coordinate change preserves the geometric object and changes only the frame used to express its coordinates. The coordinate values and frame label may change.
2. An active rigid-body motion preserves the reference frame and moves the geometric object. The initial and final coordinates describe different points or poses in the same frame.
3. A homogeneous transformation can describe the pose of one frame relative to another, map coordinates of an unchanged object between frames, or actively move an object while its coordinates remain expressed in one frame.
4. $\mathbf{T}_{bc}$ acts first, mapping from $\{c\}$ to $\{b\}$; $\mathbf{T}_{ab}$ then maps from $\{b\}$ to $\{a\}$. The complete product maps from $\{c\}$ to $\{a\}$.
5. $\mathbf{T}_1$ moves the point first, and $\mathbf{T}_2$ moves the resulting point second.
6. Translating the origin first places it at $[1\ 0]^T$, which the subsequent rotation moves to $[0\ 1]^T$. Rotating the origin first leaves it at the origin, after which translation places it at $[1\ 0]^T$.
7. No. Although $SO(2)$ rotations commute with each other, an $SE(2)$ transformation also contains translation. Rotation changes the direction of a translation, so general rotation-translation products do not commute.
8. The program needs to know whether $\mathbf{p}$ represents a point or free vector, the frame in which it is expressed, the source and destination frames or active-motion interpretation of $\mathbf{T}$, the units, and the intended multiplication order.

8.7 3. Time-varying rotations and rigid-body point velocity

A pose describes where a rigid body is at one instant. A time-varying pose describes motion. This section differentiates the position and orientation components of $\mathbf{T}_{wb}(t)$ and derives the velocity of any point fixed to the body.

8.7.1 3.1 Translational and angular velocity

Let the planar body pose vary continuously with time:

$$\mathbf{T}_{wb}(t) = \begin{bmatrix} \mathbf{R}_{wb}(t) & {}^w\mathbf{p}_b(t) \\ \mathbf{0}_2^T & 1 \end{bmatrix} \in SE(2),$$

where $t \in \mathbb{R}_{\geq 0}$ is time in seconds. The world-frame position of the body origin is

$${}^w\mathbf{p}_b(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} \in \mathbb{R}^2.$$

Its derivative, taken with respect to the fixed world frame and expressed in world-frame coordinates, is the translational velocity of O_b :

$$\boxed{{}^w\mathbf{v}_b(t) := \frac{d}{dt} {}^w\mathbf{p}_b(t) = \begin{bmatrix} \dot{x}(t) \\ \dot{y}(t) \end{bmatrix}} \in \mathbb{R}^2.$$

The entries of ${}^w\mathbf{v}_b$ are measured in m s^{-1} . The heading is $\theta(t) \in \mathbb{R}$, measured in radians, and its derivative is the signed yaw rate

$$\boxed{\omega(t) := \dot{\theta}(t)} \in \mathbb{R},$$

measured in rad s^{-1} . Positive ω denotes counter-clockwise rotation about the common $+z$ -axis of the planar frames.

8.7.2 3.2 Deriving the rotation-matrix derivative

The time-varying planar rotation matrix is

$$\mathbf{R}_{wb}(t) = \begin{bmatrix} \cos \theta(t) & -\sin \theta(t) \\ \sin \theta(t) & \cos \theta(t) \end{bmatrix}.$$

Differentiate every entry and apply the chain rule:

$$\begin{aligned} \dot{\mathbf{R}}_{wb}(t) &= \dot{\theta}(t) \begin{bmatrix} -\sin \theta(t) & -\cos \theta(t) \\ \cos \theta(t) & -\sin \theta(t) \end{bmatrix} \\ &= \omega(t) \begin{bmatrix} -\sin \theta(t) & -\cos \theta(t) \\ \cos \theta(t) & -\sin \theta(t) \end{bmatrix}. \end{aligned}$$

A square matrix $\mathbf{A}$ is **skew-symmetric** when its transpose equals its negative:

$$\boxed{\mathbf{A}^\top = -\mathbf{A}}.$$

Transposition reflects the entries across the main diagonal. The condition therefore means that entries mirrored across the diagonal have equal magnitudes and opposite signs; every diagonal entry must be zero. Define the dimensionless planar skew-symmetric generator

$$\boxed{\mathbf{S}_2 := \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}}.$$

The matrix $\mathbf{S}_2$ is skew-symmetric because $\mathbf{S}_2^\top = -\mathbf{S}_2$. Premultiplication by $\mathbf{S}_2$ rotates any planar coordinate column counter-clockwise by $\pi/2$:

$$\mathbf{S}_2 \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} -b \\ a \end{bmatrix}.$$

Direct multiplication gives

$$\mathbf{S}_2 \mathbf{R}_{wb}(\theta) = \mathbf{R}_{wb}(\theta) \mathbf{S}_2 = \begin{bmatrix} -\sin \theta & -\cos \theta \\ \cos \theta & -\sin \theta \end{bmatrix}.$$

Therefore,

$$\boxed{\dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \mathbf{R}_{wb} = \mathbf{R}_{wb} \omega \mathbf{S}_2}.$$

The two products are equal in the planar case because every element of $SO(2)$ commutes with $\mathbf{S}_2$. This equality is a special simplification of planar rotation; matrix order becomes important for three-dimensional angular velocity.

8.7.3 3.3 Why $\dot{\mathbf{R}}_{wb}$ is not a rotation matrix

Although $\mathbf{R}_{wb}(t) \in SO(2)$ at every time, its derivative $\dot{\mathbf{R}}_{wb}(t)$ is generally not an element of $SO(2)$. Its entries have units of s^{-1} , and it does not satisfy the rotation-matrix orthogonality condition.

Differentiate

$$\mathbf{R}_{wb}^\top \mathbf{R}_{wb} = \mathbf{I}_2$$

with respect to time:

$$\dot{\mathbf{R}}_{wb}^\top \mathbf{R}_{wb} + \mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb} = \mathbf{0}_{2 \times 2}.$$

Define

$$\Omega_b := \mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb}.$$

Its transpose is

$$\Omega_b^\top = \dot{\mathbf{R}}_{wb}^\top \mathbf{R}_{wb} = -\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb} = -\Omega_b,$$

so Ω_b is skew-symmetric. Substituting the planar rotation derivative gives

$$\boxed{\Omega_b = \mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2}.$$

The set of all 2×2 real skew-symmetric matrices is denoted

$$\mathfrak{so}(2) := \{\mathbf{A} \in \mathbb{R}^{2 \times 2} \mid \mathbf{A}^\top = -\mathbf{A}\}.$$

Why is this set associated with $SO(2)$? The group $SO(2)$ contains finite rotation matrices, while $\mathfrak{so}(2)$ will be shown below to contain their possible instantaneous directions. Differentiation provides the bridge between them.

Section 1.2 established that $SO(2)$ is a group under matrix multiplication. It is also **smooth**: $\mathbf{R}(\theta)$ varies differentiably with the continuous angle θ , and its multiplication and inverse operations vary differentiably with their input angles. A group with this compatible smooth structure is called a **Lie group**. Thus, $SO(2)$ is a Lie group, which allows its rotation trajectories to be differentiated.

To connect this smooth group to velocity, let a rotation change with time according to $\mathbf{R}(\theta(t))$. This time-indexed sequence of rotations is called a **curve in $SO(2)$** because every matrix on the trajectory belongs to $SO(2)$. If $\theta(0) = 0$, then the curve passes through the identity rotation $\mathbf{R}(0) = \mathbf{I}_2$ at time $t = 0$.

The derivative of this curve at $t = 0$ is its instantaneous direction of motion through the identity. This is analogous to a point moving around a circle: the point lies on the circle, but its instantaneous velocity lies along the straight tangent line touching the circle at that point. Similarly, $\mathbf{R}(\theta(t))$ lies in $SO(2)$, but its derivative generally does not; the derivative lies in a linear tangent space.

For a concrete example, consider the constant-rate curve $\mathbf{R}(at)$, where $t \in \mathbb{R}$ is time in seconds and $a \in \mathbb{R}$ is a constant angular rate measured in rad s^{-1} . Differentiating gives

$$\begin{aligned}\frac{d}{dt}\mathbf{R}(at) &= a \begin{bmatrix} -\sin(at) & -\cos(at) \\ \cos(at) & -\sin(at) \end{bmatrix} \\ &= a\mathbf{R}(at)\mathbf{S}_2.\end{aligned}$$

At $t = 0$, the rotation is $\mathbf{I}_2$ and the derivative is

$$\boxed{\left. \frac{d}{dt}\mathbf{R}(at) \right|_{t=0} = a\mathbf{S}_2}.$$

Every choice of a gives a possible instantaneous rotational direction at the identity. A **Lie algebra** is the linear space that describes all possible instantaneous motions of a Lie group, expressed at the group's identity element. For $SO(2)$, this Lie algebra is

$$\mathfrak{so}(2) = \{a\mathbf{S}_2 \mid a \in \mathbb{R}\}.$$

Why is the Lie algebra useful? The group $SO(2)$ is curved and is not a vector space: adding two rotation matrices does not generally produce another valid rotation matrix. In contrast, $\mathfrak{so}(2)$ is linear. For any $a, b \in \mathbb{R}$,

$$a\mathbf{S}_2 + b\mathbf{S}_2 = (a + b)\mathbf{S}_2 \in \mathfrak{so}(2).$$

Scalar multiplication likewise remains in $\mathfrak{so}(2)$. Instantaneous angular velocities can therefore be combined and scaled using ordinary linear-algebra operations even though finite rotations cannot.

Thus, $\mathfrak{so}(2)$ is the tangent space of $SO(2)$ at $\mathbf{I}_2$ and is a local linear model of the curved rotation group. For a sufficiently short time interval $\Delta t > 0$, measured in seconds, a constant angular rate a produces the approximation

$$\mathbf{R}(a\Delta t) \approx \mathbf{I}_2 + a\Delta t\mathbf{S}_2.$$

This approximation shows the different roles of the two objects: $\mathbf{R}(a\Delta t) \in SO(2)$ is a finite rotation, while $a\mathbf{S}_2 \in \mathfrak{so}(2)$ is its instantaneous generator. The tangent space is **defined at the identity** so that instantaneous directions from different orientations can be compared in one common linear space. For a rotation trajectory $\mathbf{R}_{wb}(t)$, premultiplication by $\mathbf{R}_{wb}^\top$ moves its derivative back to that common space:

$$\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \in \mathfrak{so}(2).$$

In addition to vector addition and scalar multiplication, a Lie algebra has a **Lie bracket**, which measures whether two infinitesimal motions commute. For matrices, the bracket is the **matrix commutator**

$$[\mathbf{A}, \mathbf{B}] := \mathbf{AB} - \mathbf{BA}, \quad \mathbf{A}, \mathbf{B} \in \mathfrak{so}(2).$$

For $\mathfrak{so}(2)$, write $\mathbf{A} = c\mathbf{S}_2$ and $\mathbf{B} = d\mathbf{S}_2$, where $c, d \in \mathbb{R}$. Then $\mathbf{AB} = cd\mathbf{S}_2^2 = \mathbf{BA}$, so $[\mathbf{A}, \mathbf{B}] = \mathbf{0}_{2 \times 2}$. The bracket is therefore trivial for planar rotations because planar rotations commute, but it becomes important for rigid motions in $SE(2)$ and for the three-dimensional rotation group $SO(3)$, where the order of motions generally matters.

8.7.4 3.4 Velocity of a point fixed to the body

Let P be a point rigidly fixed to the body. Its body-frame coordinates ${}^b\mathbf{p}_P \in \mathbb{R}^2$ are constant, so

$$\frac{d}{dt} {}^b\mathbf{p}_P = \mathbf{0}_2.$$

Its world-frame position is

$${}^w\mathbf{p}_P(t) = \mathbf{R}_{wb}(t) {}^b\mathbf{p}_P + {}^w\mathbf{p}_b(t).$$

Differentiate both sides:

$$\begin{aligned} {}^w\mathbf{v}_P &:= \frac{d}{dt} {}^w\mathbf{p}_P = \dot{\mathbf{R}}_{wb} {}^b\mathbf{p}_P + \mathbf{R}_{wb} \frac{d}{dt} {}^b\mathbf{p}_P + {}^w\mathbf{v}_b \\ &= \dot{\mathbf{R}}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{v}_b. \end{aligned}$$

Define the relative-position vector from O_b to P , expressed in world-frame coordinates, by

$${}^w\mathbf{r}_{O_b P} := {}^w\mathbf{p}_P - {}^w\mathbf{p}_b = \mathbf{R}_{wb} {}^b\mathbf{p}_P \in \mathbb{R}^2.$$

Using $\dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \mathbf{R}_{wb}$ gives

$$\boxed{{}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \omega \mathbf{S}_2 {}^w\mathbf{r}_{O_b P}}.$$

The first term is the translational velocity shared with the body origin. The second term is the additional velocity caused by rotation about the body origin. Because $\mathbf{S}_2$ rotates ${}^w\mathbf{r}_{O_bP}$ by $\pi/2$, the rotational contribution is perpendicular to the relative-position vector.

The two panels in Figure 8.1 separate the position-vector identity from the velocity-vector addition. In the left panel, the two solid vectors add head to tail to produce the dashed world-position vector of P . In the right panel, translated copies of the two velocity contributions form a parallelogram whose diagonal is ${}^w\mathbf{v}_P$; the right-angle marker applies specifically to the rotational contribution and ${}^w\mathbf{r}_{O_bP}$.

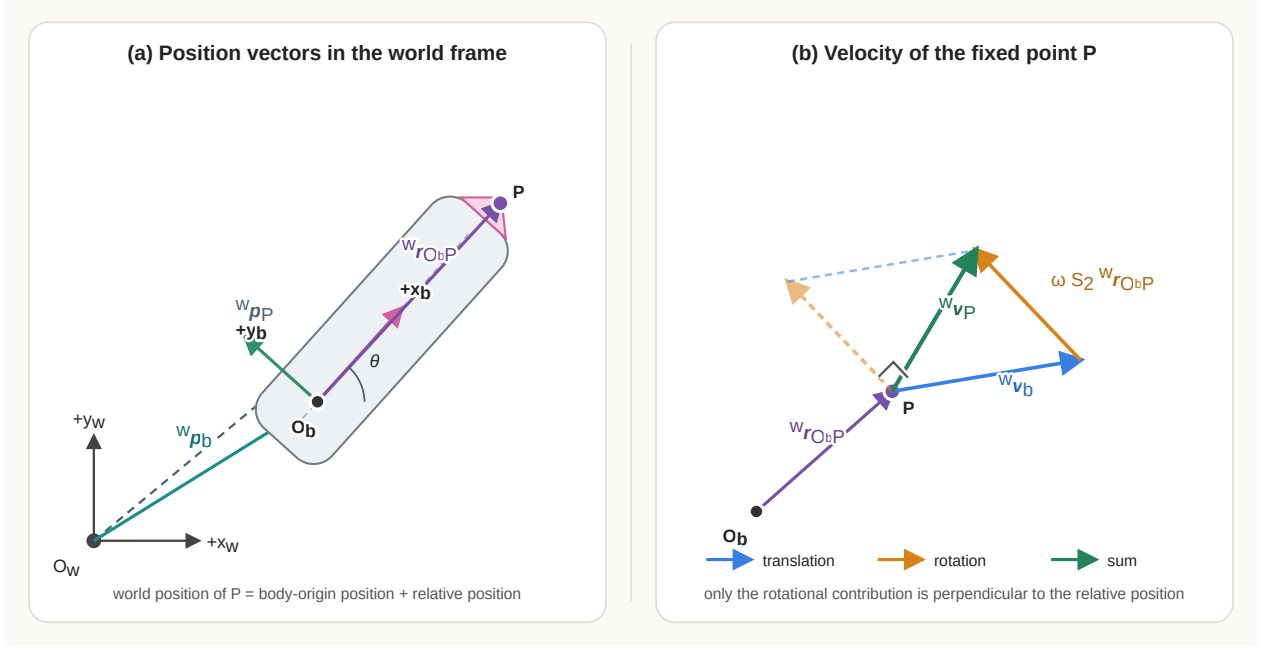


Figure 8.1: Position and velocity relations for a point fixed to a planar rigid body.

The same result can be written with the three-dimensional cross product by embedding the planar quantities in $\mathbb{R}^3$:

$${}^w\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix}, \quad {}^w\mathbf{r}_{O_bP}^{(3)} = \begin{bmatrix} r_x \\ r_y \\ 0 \end{bmatrix}.$$

Here, r_x and r_y , measured in metres, are the two entries of ${}^w\mathbf{r}_{O_bP}$, the superscript (3) marks its three-dimensional embedding, and $\times$ denotes the vector cross product.

Then

$${}^w\boldsymbol{\omega} \times {}^w\mathbf{r}_{O_bP}^{(3)} = \begin{bmatrix} -\omega r_y \\ \omega r_x \\ 0 \end{bmatrix},$$

whose first two entries equal $\omega \mathbf{S}_2 {}^w\mathbf{r}_{O_bP}$. Thus, the planar formula is the two-dimensional form of the rigid-body velocity relation

$$\mathbf{v}_P = \mathbf{v}_{O_b} + \omega \times \mathbf{r}_{O_b P}.$$

8.7.5 3.5 Worked example

At one instant, suppose the body heading is $\theta = \pi/2$ rad, the body-origin velocity is

$${}^w\mathbf{v}_b = \begin{bmatrix} 0.5 \\ 0 \end{bmatrix} \text{ m s}^{-1},$$

and the yaw rate is $\omega = 2 \text{ rad s}^{-1}$. Point P is fixed one metre along the body $+x_b$ -axis:

$${}^b\mathbf{p}_P = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ m}.$$

At $\theta = \pi/2$, its relative-position vector in the world frame is

$${}^w\mathbf{r}_{O_b P} = \mathbf{R}_{wb} \left(\frac{\pi}{2} \right) {}^b\mathbf{p}_P = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \text{ m}.$$

The rotational contribution is

$$\omega \mathbf{S}_2 {}^w\mathbf{r}_{O_b P} = 2 \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ 0 \end{bmatrix} \text{ m s}^{-1}.$$

Therefore,

$$\begin{aligned} {}^w\mathbf{v}_P &= {}^w\mathbf{v}_b + \omega \mathbf{S}_2 {}^w\mathbf{r}_{O_b P} \\ &= \begin{bmatrix} 0.5 \\ 0 \end{bmatrix} + \begin{bmatrix} -2 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} -1.5 \\ 0 \end{bmatrix} \text{ m s}^{-1}. \end{aligned}$$

The point lies one metre above the body origin in world coordinates. Counter-clockwise rotation therefore moves it instantaneously toward $-x_w$ at 2 m s^{-1} , while translation of the body origin contributes 0.5 m s^{-1} toward $+x_w$.

8.7.6 3.6 Assumptions, failure modes, and checks

- **Body-fixed point assumption:** The derivation sets $d({}^b\mathbf{p}_P)/dt = \mathbf{0}_2$. If P moves relative to the body, the additional term $\mathbf{R}_{wb} d({}^b\mathbf{p}_P)/dt$ must be retained.
- **Frame consistency:** ${}^w\mathbf{v}_b$ and $\omega \mathbf{S}_2 {}^w\mathbf{r}_{O_bP}$ must be expressed in the same frame before they are added.
- **Yaw sign:** With the stated convention, $\omega > 0$ produces counter-clockwise rotation. A sign error reverses the rotational velocity of every off-origin point.
- **Derivative versus rotation:** $\dot{\mathbf{R}}_{wb}$ is not a rotation matrix. Code must not validate or normalise it as though it belonged to $SO(2)$.
- **Numerical differentiation:** Estimating $\dot{\mathbf{R}}$ or velocity from sampled poses amplifies measurement noise and depends on an accurate sampling interval.

8.8 Quick test C: rotation derivatives and point velocity

1. Define ${}^w\mathbf{v}_b$ and ω , including their frames and units.
2. Differentiate $\mathbf{R}_{wb}(\theta)$ and show that $\dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \mathbf{R}_{wb}$.
3. What geometric operation does $\mathbf{S}_2$ perform on a planar vector?
4. Why is $\dot{\mathbf{R}}_{wb}$ generally not an element of $SO(2)$?
5. Differentiate $\mathbf{R}_{wb}^\top \mathbf{R}_{wb} = \mathbf{I}_2$ and use the result to show that $\mathbf{R}_{wb}^\top \dot{\mathbf{R}}_{wb}$ is skew-symmetric.
6. Starting from ${}^w\mathbf{p}_P = \mathbf{R}_{wb} {}^b\mathbf{p}_P + {}^w\mathbf{p}_b$, derive the velocity of a point fixed to the body.
7. Why is the rotational contribution to point velocity perpendicular to ${}^w\mathbf{r}_{O_bP}$?
8. How does the velocity equation change if P moves relative to frame $\{b\}$?

8.8.1 Answers and hints

1. ${}^w\mathbf{v}_b = d({}^w\mathbf{p}_b)/dt = [\dot{x} \ \dot{y}]^\top$ is the velocity of the body origin relative to the fixed world frame and expressed in world coordinates, measured in m s^{-1} . The scalar $\omega = \dot{\theta}$ is the signed yaw rate, measured in rad s^{-1} .
2. Entrywise differentiation and the chain rule give the matrix displayed in Section 3.2. Direct multiplication shows that its coefficient of ω equals $\mathbf{S}_2 \mathbf{R}_{wb}$, so $\dot{\mathbf{R}}_{wb} = \omega \mathbf{S}_2 \mathbf{R}_{wb}$.
3. $\mathbf{S}_2 [a \ b]^\top = [-b \ a]^\top$, so it rotates the coordinate column counter-clockwise by $\pi/2$ without changing its length.
4. Its entries have units of s^{-1} , and it does not generally satisfy $\dot{\mathbf{R}}_{wb}^\top \dot{\mathbf{R}}_{wb} = \mathbf{I}_2$ or have determinant 1.
5. Differentiation gives $\dot{\mathbf{R}}^\top \mathbf{R} + \mathbf{R}^\top \dot{\mathbf{R}} = \mathbf{0}$. Therefore, $(\mathbf{R}^\top \dot{\mathbf{R}})^\top = \dot{\mathbf{R}}^\top \mathbf{R} = -\mathbf{R}^\top \dot{\mathbf{R}}$.
6. Since P is body-fixed, $d({}^b\mathbf{p}_P)/dt = \mathbf{0}_2$. Therefore, ${}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \dot{\mathbf{R}}_{wb} {}^b\mathbf{p}_P = {}^w\mathbf{v}_b + \omega \mathbf{S}_2 {}^w\mathbf{r}_{O_bP}$.
7. $\mathbf{S}_2$ rotates the relative-position vector by $\pi/2$, so their dot product is zero. This is the tangential direction of instantaneous circular motion about O_b .
8. Retain the relative-motion term: ${}^w\mathbf{v}_P = {}^w\mathbf{v}_b + \dot{\mathbf{R}}_{wb} {}^b\mathbf{p}_P + \mathbf{R}_{wb} d({}^b\mathbf{p}_P)/dt$.

8.9 4. Planar twists and $\mathfrak{se}(2)$

The previous section described translational and angular velocity separately. A **planar twist** combines them into one representation of the instantaneous motion of a rigid body. This representation

will connect the pose derivative $\dot{\mathbf{T}}_{wb}$ to the differential-drive inputs v and ω .

8.9.1 4.1 Twist coordinates

In this chapter, a planar twist is represented by the coordinate column

$$\xi := \begin{bmatrix} \nu_x \\ \nu_y \\ \omega \end{bmatrix} \in \mathbb{R}^3.$$

The first two entries form a linear-velocity column $\nu = [\nu_x \ \nu_y]^\top \in \mathbb{R}^2$, whose entries are measured in m s^{-1} . The third entry $\omega \in \mathbb{R}$ is angular velocity, measured in rad s^{-1} . The symbol ξ is the Greek letter *xi*. The word *twist* refers to an instantaneous rigid-body velocity, not to a pose or a finite displacement.

This chapter uses the component order $[\nu_x \ \nu_y \ \omega]^\top$. Some textbooks and software libraries place angular velocity first, so the component order must always be declared. Although ξ is mathematically a three-entry real column, its entries have different physical units. A norm such as $\|\xi\|_2$ therefore has no direct physical meaning unless a length scale is introduced to compare translation with rotation.

The numerical values of ν depend on both the coordinate frame and the point chosen as the velocity reference. Consequently, the phrases *body twist* and *spatial twist* identify more than two different coordinate descriptions of an ordinary free vector.

8.9.2 4.2 The hat operator and $\mathfrak{se}(2)$

The **hat operator** maps the twist coordinate column to a 3×3 matrix:

$$\hat{\xi} := \begin{bmatrix} 0 & -\omega & \nu_x \\ \omega & 0 & \nu_y \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} \omega \mathbf{S}_2 & \nu \\ \mathbf{0}_2^\top & 0 \end{bmatrix}.$$

The wide hat in $\hat{\xi}$ denotes this mapping; it does not denote an estimate. The inverse operation, which recovers ξ from its matrix representation, is commonly called the **vee operator** and is written $(\hat{\xi})^\vee = \xi$.

The set of all matrices with this structure is

$$\mathfrak{se}(2) := \left\{ \begin{bmatrix} a \mathbf{S}_2 & \mathbf{u} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \mid a \in \mathbb{R}, \mathbf{u} \in \mathbb{R}^2 \right\}.$$

The notation $\mathfrak{se}(2)$ is read “little s e two.” It is the Lie algebra of $SE(2)$: geometrically, it is the three-dimensional vector space tangent to $SE(2)$ at the identity transformation. Elements of $SE(2)$ represent finite poses, whereas elements of $\mathfrak{se}(2)$ represent infinitesimal rigid motions or instantaneous rigid-body velocities.

When $\hat{\xi}$ represents physical velocity, the entries of its 2×2 block have units of s^{-1} and the entries of its right column have units of m s^{-1} . The matrix is therefore a structured representation of a physical rate, not an ordinary dimensionless homogeneous transformation.

8.9.3 4.3 Body twist

For compactness in this derivation, write

$$\mathbf{T} := \mathbf{T}_{wb} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix}, \quad \dot{\mathbf{T}} = \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_2^\top & 0 \end{bmatrix},$$

where $\mathbf{R} = \mathbf{R}_{wb} \in SO(2)$ and $\mathbf{p} = {}^w\mathbf{p}_b \in \mathbb{R}^2$. The entries of $\mathbf{p}$ are measured in metres, and $\dot{\mathbf{p}} = {}^w\mathbf{v}_b$ is the world-frame velocity of the body origin.

Premultiply $\dot{\mathbf{T}}$ by $\mathbf{T}^{-1}$:

$$\begin{aligned} \mathbf{T}^{-1}\dot{\mathbf{T}} &= \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top\mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}^\top\dot{\mathbf{R}} & \mathbf{R}^\top\dot{\mathbf{p}} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} \omega\mathbf{S}_2 & \nu_b \\ \mathbf{0}_2^\top & 0 \end{bmatrix}. \end{aligned}$$

Here,

$$\nu_b := \mathbf{R}_{wb}^\top {}^w\mathbf{v}_b = \begin{bmatrix} \nu_{x_b} \\ \nu_{y_b} \end{bmatrix}$$

is the velocity of the body origin expressed along the body axes x_b and y_b . Its entries are measured in m s^{-1} . The **body twist** and its matrix representation are therefore

$$\xi_b := \begin{bmatrix} \nu_{x_b} \\ \nu_{y_b} \\ \omega \end{bmatrix}, \quad \hat{\xi}_b = \mathbf{T}_{wb}^{-1}\dot{\mathbf{T}}_{wb}.$$

Equivalently,

$$\dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb}\hat{\xi}_b.$$

The subscript b states that the twist is resolved in the instantaneous body frame. In planar motion, ω has the same scalar value in the world and body descriptions because both frames share the same rotation axis; this simplification does not extend to the three-component angular-velocity coordinates used in spatial motion.

8.9.4 4.4 Spatial twist expressed in the world frame

Postmultiplying $\dot{\mathbf{T}}$ by $\mathbf{T}^{-1}$ produces a different element of $\mathfrak{se}(2)$:

$$\begin{aligned}\dot{\mathbf{T}}\mathbf{T}^{-1} &= \begin{bmatrix} \dot{\mathbf{R}} & \dot{\mathbf{p}} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \\ &= \begin{bmatrix} \dot{\mathbf{R}}\mathbf{R}^\top & \dot{\mathbf{p}} - \dot{\mathbf{R}}\mathbf{R}^\top \mathbf{p} \\ \mathbf{0}_2^\top & 0 \end{bmatrix} \\ &= \begin{bmatrix} \omega \mathbf{S}_2 & \nu_w \\ \mathbf{0}_2^\top & 0 \end{bmatrix},\end{aligned}$$

where

$$\nu_w := \dot{\mathbf{p}} - \omega \mathbf{S}_2 \mathbf{p}.$$

Write $\nu_w = [\nu_{x_w} \ \nu_{y_w}]^\top \in \mathbb{R}^2$, where ν_{x_w} and ν_{y_w} are its components along the world axes and are measured in m s^{-1} .

The **spatial twist expressed in the world frame** is

$$\xi_w := \begin{bmatrix} \nu_{x_w} \\ \nu_{y_w} \\ \omega \end{bmatrix}, \quad \hat{\xi}_w = \dot{\mathbf{T}}_{wb} \mathbf{T}_{wb}^{-1}.$$

Equivalently,

$$\dot{\mathbf{T}}_{wb} = \hat{\xi}_w \mathbf{T}_{wb}.$$

A crucial distinction is

$$\nu_w \neq \dot{\mathbf{p}} = {}^w \mathbf{v}_b$$

in general. The column ν_w is the linear part of the spatial twist, not simply the body-origin velocity expressed in world coordinates. To see its meaning, substitute its definition into the velocity field from Section 3.4. The velocity of a body-fixed point P can be written

$${}^w \mathbf{v}_P = \nu_w + \omega \mathbf{S}_2 {}^w \mathbf{p}_P.$$

Thus, ν_w is the velocity, expressed in world coordinates, assigned by the instantaneous rigid-motion field to the location O_w where ${}^w \mathbf{p}_P = \mathbf{0}_2$. This is why changing the reference origin changes the linear part of a twist.

8.9.5 4.5 Relating body and spatial twists

Using the two definitions gives

$$\hat{\xi}_w = \dot{\mathbf{T}}\mathbf{T}^{-1} = \mathbf{T}(\mathbf{T}^{-1}\dot{\mathbf{T}})\mathbf{T}^{-1} = \mathbf{T}\hat{\xi}_b\mathbf{T}^{-1}.$$

The corresponding mapping between the coordinate columns is

$$\boxed{\xi_w = \text{Ad}_{\mathbf{T}_{wb}} \xi_b},$$

where $\text{Ad}_{\mathbf{T}}$ is called the **adjoint representation** of $\mathbf{T}$. For the component order used here,

$$\boxed{\text{Ad}_{\mathbf{T}} = \begin{bmatrix} \mathbf{R} & -\mathbf{S}_2\mathbf{p} \\ \mathbf{0}_2^\top & 1 \end{bmatrix} \in \mathbb{R}^{3 \times 3}}.$$

Consequently,

$$\nu_w = \mathbf{R}\nu_b - \omega\mathbf{S}_2\mathbf{p}.$$

The term $-\omega\mathbf{S}_2\mathbf{p}$ accounts for the change of reference origin. A twist therefore does not transform like a free velocity vector, for which rotation alone would be sufficient.

8.9.6 4.6 Differential-drive body twist

Under the ideal rolling constraints, a differential-drive robot has no lateral velocity at the body origin. Its body-frame linear velocity is

$$\nu_b = \begin{bmatrix} v \\ 0 \end{bmatrix},$$

where v is the signed forward speed in ms^{-1} . Its body twist is therefore

$$\boxed{\xi_b = \begin{bmatrix} v \\ 0 \\ \omega \end{bmatrix}, \quad \hat{\xi}_b = \begin{bmatrix} 0 & -\omega & v \\ \omega & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}}.$$

Substitute this matrix into $\dot{\mathbf{T}}_{wb} = \mathbf{T}_{wb}\hat{\xi}_b$. The rotation and translation blocks give

$$\dot{\mathbf{R}}_{wb} = \mathbf{R}_{wb}\omega\mathbf{S}_2, \quad {}^w\mathbf{v}_b = \mathbf{R}_{wb} \begin{bmatrix} v \\ 0 \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \end{bmatrix}.$$

Therefore, the familiar differential-drive state equation is recovered:

$$\dot{\mathbf{q}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix}.$$

The twist formulation has not introduced a different motion model. It has expressed the same model in a form that respects rigid-motion composition and generalises naturally to three-dimensional robotics.

8.9.7 4.7 Worked example: the two twist representations

Suppose

$${}^w\mathbf{p}_b = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \text{ m}, \quad \theta = \frac{\pi}{2} \text{ rad},$$

and the differential-drive body twist has $v = 0.5 \text{ m s}^{-1}$ and $\omega = 2 \text{ rad s}^{-1}$. The body-twist column is

$$\xi_b = \begin{bmatrix} 0.5 \\ 0 \\ 2 \end{bmatrix},$$

where the first two entries have units of m s^{-1} and the last has units of rad s^{-1} . At $\theta = \pi/2$, the velocity of the body origin expressed in the world frame is

$$\dot{\mathbf{p}} = \mathbf{R}_{wb} \begin{bmatrix} 0.5 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \text{ m s}^{-1}.$$

The spatial twist's linear part is

$$\begin{aligned} \nu_w &= \dot{\mathbf{p}} - \omega \mathbf{S}_2 \mathbf{p} \\ &= \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} - 2 \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 \\ -3.5 \end{bmatrix} \text{ m s}^{-1}. \end{aligned}$$

This does not contradict ${}^w\mathbf{v}_b = [0 \ 0.5]^\top \text{ m s}^{-1}$. The two columns refer to different points in the same instantaneous velocity field. Substituting the body-origin position into the spatial velocity field recovers its correct velocity:

$$\nu_w + \omega \mathbf{S}_2 {}^w\mathbf{p}_b = \begin{bmatrix} 2 \\ -3.5 \end{bmatrix} + \begin{bmatrix} -2 \\ 4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \text{ m s}^{-1}.$$

8.9.8 4.8 Assumptions, failure modes, and checks

- **Undeclared component order:** A column containing the same three numbers can represent different twists if one convention uses linear velocity first and another uses angular velocity first.
- **Confusing a twist with a pose:** ξ and $\hat{\xi}$ describe instantaneous motion; neither is an element of $SE(2)$ or a finite pose.
- **Confusing ν_w with ${}^w\mathbf{v}_b$:** They are equal only when $\omega = 0$, $\mathbf{p} = \mathbf{0}_2$, or their correction term happens to vanish.
- **Rotating a twist like a free vector:** Changing the reference origin introduces the translation-dependent term in Ad_T .
- **Assuming zero lateral velocity for every rigid body:** The condition $\nu_{y_b} = 0$ comes from the ideal differential-drive rolling constraint, not from the definition of a planar twist.
- **Mixing finite and infinitesimal quantities:** $\mathbf{T}_{wb}$ is dimensionless apart from its translation column, whereas $\hat{\xi}$ contains rates. They cannot be added directly.

8.10 Quick test D: planar twists

1. What physical quantities are stored in $\xi = [\nu_x \ \nu_y \ \omega]^\top$, and what are their units?
2. Define the hat operator for the component order used in this chapter.
3. What is $\mathfrak{se}(2)$, and how does it differ from $SE(2)$?
4. Starting from $\mathbf{T}_{wb}$ and $\dot{\mathbf{T}}_{wb}$, derive $\hat{\xi}_b = \mathbf{T}_{wb}^{-1} \dot{\mathbf{T}}_{wb}$.
5. Why is the linear part ν_w of the spatial twist generally different from ${}^w\mathbf{v}_b$?
6. State the two equivalent pose-rate equations involving the body and spatial twists.
7. Derive the differential-drive state equation from $\xi_b = [v \ 0 \ \omega]^\top$.
8. Why is it unsafe to compare twist magnitudes using an ordinary Euclidean norm without first choosing a length scale?

8.10.1 Answers and hints

1. ν_x and ν_y are the two components of a linear velocity, measured in m s^{-1} ; ω is angular velocity, measured in rad s^{-1} . Their frame and reference point must also be declared.
2. The hat matrix has $\omega \mathbf{S}_2$ as its upper-left 2×2 block, ν as its upper-right 2×1 block, and a zero bottom row, as displayed in Section 4.2.
3. $SE(2)$ is the group of finite planar rigid transformations. Its Lie algebra $\mathfrak{se}(2)$ is the vector space of structured matrices representing infinitesimal rigid motions and instantaneous velocities.
4. Block multiplication gives upper-left block $\mathbf{R}^\top \dot{\mathbf{R}}$, upper-right block $\mathbf{R}^\top \dot{\mathbf{p}}$, and a zero bottom row. Identify $\mathbf{R}^\top \dot{\mathbf{R}} = \omega \mathbf{S}_2$ and $\nu_b = \mathbf{R}^\top \dot{\mathbf{p}}$.
5. A spatial twist uses the fixed world origin as its velocity reference. Therefore, its linear part is $\nu_w = \dot{\mathbf{p}} - \omega \mathbf{S}_2 \mathbf{p}$; the second term accounts for the offset between the world and body origins.
6. $\mathbf{T}_{wb} = \mathbf{T}_{wb} \hat{\xi}_b$ and $\dot{\mathbf{T}}_{wb} = \hat{\xi}_w \mathbf{T}_{wb}$.
7. Matrix multiplication gives ${}^w\mathbf{v}_b = \mathbf{R}_{wb} [v \ 0]^\top = [v \cos \theta \ v \sin \theta]^\top$ and $\dot{\theta} = \omega$.
8. The entries combine linear and angular velocities with different physical units. A length scale is required to convert angular speed into a comparable linear-speed scale.

8.11 References

- John J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed., Chapters 2 and 5, especially frame mappings, transformation interpretations, angular velocity, and rigid-body point velocity.
- Peter Corke, *Robotics, Vision and Control: Fundamental Algorithms in Python*, 3rd ed., Chapters 2-3, especially time-varying pose, angular velocity, and derivatives of orientation and pose.
- Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Chapters 2-3, especially configuration spaces, angular velocity, skew-symmetric representations, body and spatial twists, and rigid-body motions. The $SE(3)$ twist definitions are specialised here to planar motion in $SE(2)$.

9 Dynamics, Forces, Torque, Friction, and Actuator Limits

9.1 Why this chapter matters

Kinematics predicts how a robot's pose changes when its velocity is known, but it does not explain how that velocity is produced or how quickly it can change. Dynamics connects motion to mass, rotational inertia, external forces, and external torques. This connection is necessary for physically meaningful simulation, actuator selection, acceleration limits, braking analysis, and feedback control.

This chapter begins with straight-line motion because it exposes the central force-acceleration relation without rotational notation. Later sections extend the same reasoning to wheel torque, planar rotation, traction, friction, and actuator limits.

9.2 Prerequisites and assumptions

The reader should understand vectors and units from the [linear algebra foundations chapter](#), coordinate frames from the [geometry chapter](#), and derivatives and numerical integration from the [kinematics and numerical integration chapter](#).

The first model treats the robot as a rigid body constrained to translate along a fixed straight line on level ground. The line is fixed in the world frame, so it is an inertial reference for the speeds considered here. The robot's mass is constant, vertical motion is absent, and all forces are represented by their signed components along the line. Rotational motion, wheel inertia, slip, and changing actuator force are excluded from this first model.

9.3 1. From kinematics to dynamics

9.3.1 1.1 Position, velocity, and acceleration

Let $t \in \mathbb{R}_{\geq 0}$ denote continuous time in seconds. Let $s(t) \in \mathbb{R}$ be the robot's signed position along the fixed line, measured in metres. Increasing s defines the positive direction.

The signed longitudinal velocity is

$$v(t) := \dot{s}(t) = \frac{ds(t)}{dt},$$

where $v(t)$ is measured in m s^{-1} . The signed longitudinal acceleration is

$$a(t) := \dot{v}(t) = \ddot{s}(t) = \frac{d^2 s(t)}{dt^2},$$

where $a(t)$ is measured in m s^{-2} . Positive acceleration means that velocity changes in the positive direction; it does not necessarily mean that the robot is moving forward. For example, a robot with $v < 0$ and $a > 0$ is moving in the negative direction while its speed decreases.

Kinematics relates the quantities through $\dot{s} = v$. A dynamic model adds an equation that determines $\dot{v} = a$ from physical causes.

9.3.2 1.2 Net external force

A force is an interaction that can change an object's translational motion. Let $n \in \mathbb{N}$ be the number of external forces acting on the chosen robot system, and let $F_j(t) \in \mathbb{R}$ be the signed component of external force $j \in \{1, \dots, n\}$ along the positive line direction. Each F_j is measured in newtons, abbreviated N.

The net external force is the algebraic sum

$$F_{\text{net}}(t) := \sum_{j=1}^n F_j(t),$$

where $F_{\text{net}}(t) \in \mathbb{R}$ is also measured in newtons. Forces pointing in the positive direction have positive components, while forces pointing in the negative direction have negative components. Internal forces between parts included inside the chosen robot system do not appear separately in this whole-system balance because they occur in equal-and-opposite pairs.

For a driven ground robot, the forward force on the robot is the traction force exerted by the ground at the wheel contacts. Motor torque helps create this contact force, but motor torque and longitudinal force are different physical quantities and have different units.

9.3.3 1.3 Newton's second law and the forward-dynamics equation

Let $m \in \mathbb{R}_{>0}$ be the robot's constant mass in kilograms. Newton's second law for translation along the line is

$$F_{\text{net}}(t) = ma(t).$$

Substituting $a = \dot{v} = \ddot{s}$ gives

$$F_{\text{net}}(t) = m\dot{v}(t) = m\ddot{s}(t).$$

Because $m > 0$, division by mass gives the forward-dynamics equation

$$\boxed{a(t) = \dot{v}(t) = \frac{F_{\text{net}}(t)}{m}}.$$

Here, *forward dynamics* means calculating acceleration from the current physical model and the applied forces. In this simple constant-mass model, the acceleration depends only on net force and mass. The dimensional check is

$$\frac{\text{N}}{\text{kg}} = \frac{\text{kg m s}^{-2}}{\text{kg}} = \text{m s}^{-2},$$

which matches the unit of acceleration.

If a nonnegative drive-force magnitude F_{drive} acts forward and a nonnegative resistance-force magnitude F_{resist} acts backward, then their signed sum is

$$F_{\text{net}} = F_{\text{drive}} - F_{\text{resist}},$$

so

$$a = \frac{F_{\text{drive}} - F_{\text{resist}}}{m}.$$

This equation shows why a velocity command cannot change the physical velocity instantaneously: finite forces acting on positive mass produce finite acceleration.

9.3.4 1.4 Constant-force motion over one interval

Suppose F_{net} and m remain constant during the interval from sample time t_k to $t_{k+1} = t_k + \Delta t$, where $\Delta t > 0$ is measured in seconds. Define $s_k := s(t_k)$ and $v_k := v(t_k)$. The acceleration is constant and equals

$$a = \frac{F_{\text{net}}}{m}.$$

For elapsed time $\tau \in [0, \Delta t]$, integrating $\dot{v} = a$ from 0 to τ gives

$$v(t_k + \tau) = v_k + a\tau.$$

Setting $\tau = \Delta t$ gives the exact constant-force velocity update

$$v_{k+1} = v_k + \frac{F_{\text{net}}}{m} \Delta t.$$

Integrating $\dot{s}(t_k + \tau) = v_k + a\tau$ over the same interval gives

$$\begin{aligned} s_{k+1} &= s_k + \int_0^{\Delta t} (v_k + a\tau) d\tau \\ &= s_k + v_k \Delta t + \frac{1}{2} a \Delta t^2 \\ &= s_k + v_k \Delta t + \frac{F_{\text{net}}}{2m} \Delta t^2. \end{aligned}$$

These updates are exact only under the stated one-dimensional, constant-mass, and constant-net-force assumptions.

9.3.5 1.5 Worked example

Consider a robot with mass $m = 20 \text{ kg}$, forward traction magnitude $F_{\text{drive}} = 30 \text{ N}$, and backward resistance magnitude $F_{\text{resist}} = 10 \text{ N}$. Its net force and acceleration are

$$F_{\text{net}} = 30 - 10 = 20 \text{ N},$$

$$a = \frac{20}{20} = 1 \text{ m s}^{-2}.$$

If $s_k = 0 \text{ m}$, $v_k = 0 \text{ m s}^{-1}$, and $\Delta t = 2 \text{ s}$, then

$$v_{k+1} = 0 + (1)(2) = 2 \text{ m s}^{-1},$$

and

$$s_{k+1} = 0 + (0)(2) + \frac{1}{2}(1)(2^2) = 2 \text{ m}.$$

The example predicts motion only along the fixed line. It does not yet explain how motor torque becomes traction, how much traction the ground can support, or how wheel and chassis rotation affect the result.

9.3.6 Quick test A

1. In one sentence each, distinguish kinematics from dynamics.
2. Define s , v , a , m , and F_{net} , including units.
3. Starting from $F_{\text{net}} = ma$, derive $\dot{v} = F_{\text{net}}/m$ and state why division by m is valid.
4. A 10 kg robot experiences 18 N forward and 3 N backward. Find its acceleration.
5. With the same net force, what happens to acceleration if mass doubles?
6. Name two assumptions that must hold for the constant-force position update to be exact.

9.3.7 Quick test A answers and hints

1. Kinematics relates motion variables without modelling their physical causes; dynamics relates forces and torques to acceleration and motion.
2. s is signed position in metres, $v = \dot{s}$ is signed velocity in m s^{-1} , $a = \dot{v}$ is signed acceleration in m s^{-2} , $m > 0$ is mass in kilograms, and F_{net} is the signed sum of external force components in newtons.
3. Substitute $a = \dot{v}$ into $F_{\text{net}} = ma$ and divide both sides by the strictly positive mass m .
4. $F_{\text{net}} = 18 - 3 = 15 \text{ N}$, so $a = 15/10 = 1.5 \text{ m s}^{-2}$.
5. Acceleration halves because $a = F_{\text{net}}/m$.

6. Examples include one-dimensional translation, constant mass, constant net force during the interval, and validity of the rigid-body model in an inertial frame.

9.4 2. From motor torque to wheel traction

9.4.1 2.1 Torque and its units

Torque, also called the moment of a force, is a vector quantity that represents the tendency of a force to rotate a body about a chosen point or axis. Torque is not a separate kind of force: it depends on the applied force and where that force acts.

Choose a point O on the rotation axis. Let $\rho \in \mathbb{R}^3$ be the position vector from O to the point where the force is applied, measured in metres, and let $\mathbf{F} \in \mathbb{R}^3$ be the applied force expressed in the same coordinate frame, measured in newtons. The torque about O is

$$\tau_O := \rho \times \mathbf{F},$$

where $\times$ denotes the three-dimensional vector cross product and $\tau_O \in \mathbb{R}^3$ is measured in newton-metres, written N m. If $\rho = [\rho_x \ \rho_y \ \rho_z]^\top$ and $\mathbf{F} = [F_x \ F_y \ F_z]^\top$, then the cross product is calculated as

$$\rho \times \mathbf{F} = \begin{bmatrix} \rho_y F_z - \rho_z F_y \\ \rho_z F_x - \rho_x F_z \\ \rho_x F_y - \rho_y F_x \end{bmatrix}.$$

The order of the cross product matters: $\mathbf{F} \times \rho = -\tau_O$.

The torque vector is perpendicular to the plane containing ρ and $\mathbf{F}$. Its direction follows the right-hand rule: curl the fingers of the right hand from ρ toward $\mathbf{F}$ through the smaller angle, and the thumb points in the direction of τ_O .

Figure 9.1 shows the position vector, applied force, angle, perpendicular moment arm, and out-of-page torque direction in one geometric construction.

Let $\rho := \|\rho\|_2$ be the distance from O to the force application point in metres, let $F := \|\mathbf{F}\|_2$ be the force magnitude in newtons, and let $\theta \in [0, \pi]$ be the angle from ρ to $\mathbf{F}$. The cross-product magnitude satisfies

$$\begin{aligned} \|\rho \times \mathbf{F}\|_2^2 &= \|\rho\|_2^2 \|\mathbf{F}\|_2^2 - (\rho^\top \mathbf{F})^2 \\ &= \rho^2 F^2 - \rho^2 F^2 \cos^2 \theta \\ &= \rho^2 F^2 \sin^2 \theta. \end{aligned}$$

Taking the nonnegative square root gives the torque magnitude

$$\tau := \|\tau_O\|_2 = \rho F \sin \theta.$$

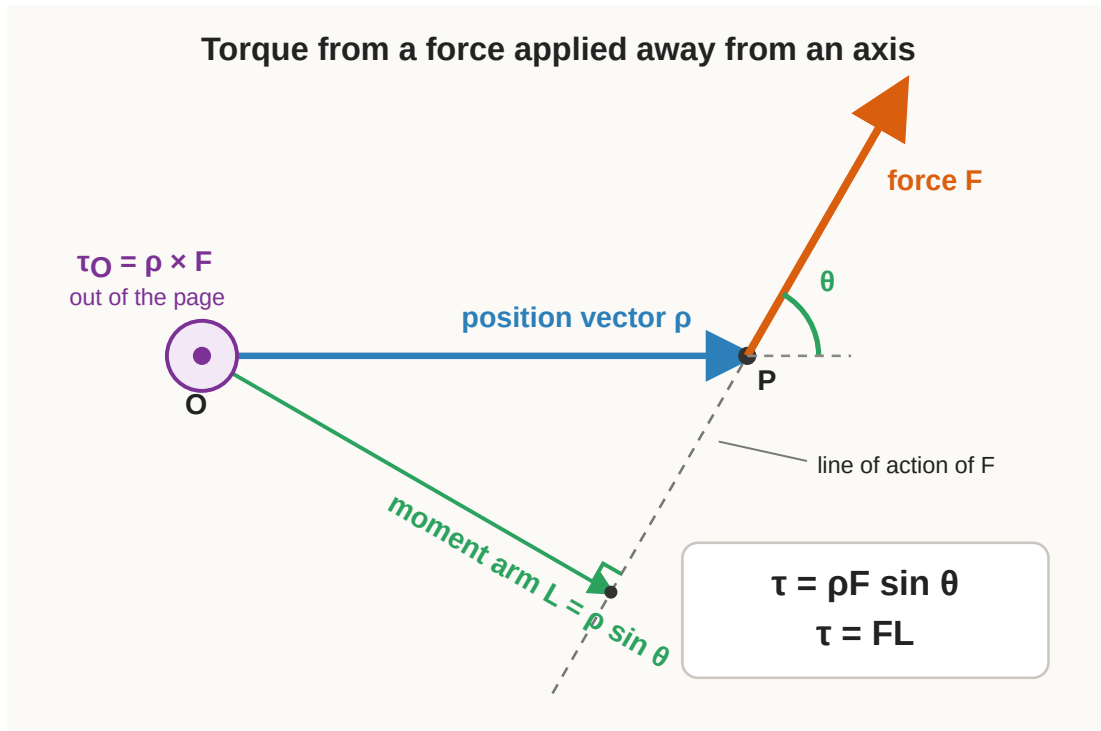


Figure 9.1: Vector geometry of torque about point O .

Only the component of force perpendicular to ρ contributes to the torque magnitude. Equivalently, define the perpendicular moment arm

$$L := \rho \sin \theta,$$

where $L \geq 0$ is the shortest distance in metres from O to the force's line of action. The line of action is the infinite line through the application point in the direction of $\mathbf{F}$. Substitution gives the scalar formula

$$\boxed{\tau = FL}.$$

Thus $\tau = FL$ is not a different torque law; it is the magnitude of the cross-product law written using the perpendicular moment arm. If ρ and $\mathbf{F}$ are perpendicular, then $\theta = \pi/2$, $L = \rho$, and $\tau = \rho F$. If they are parallel or antiparallel, then $\sin \theta = 0$ and the torque is zero.

Figure 9.2 holds radius and force magnitude constant while changing only θ . The angled force produces less torque because only its component perpendicular to the radius contributes.

If $\mathbf{e}_a \in \mathbb{R}^3$ is a unit vector along a chosen axis, the signed scalar torque about that axis is the projection

$$\tau_a := \mathbf{e}_a^T \tau_O.$$

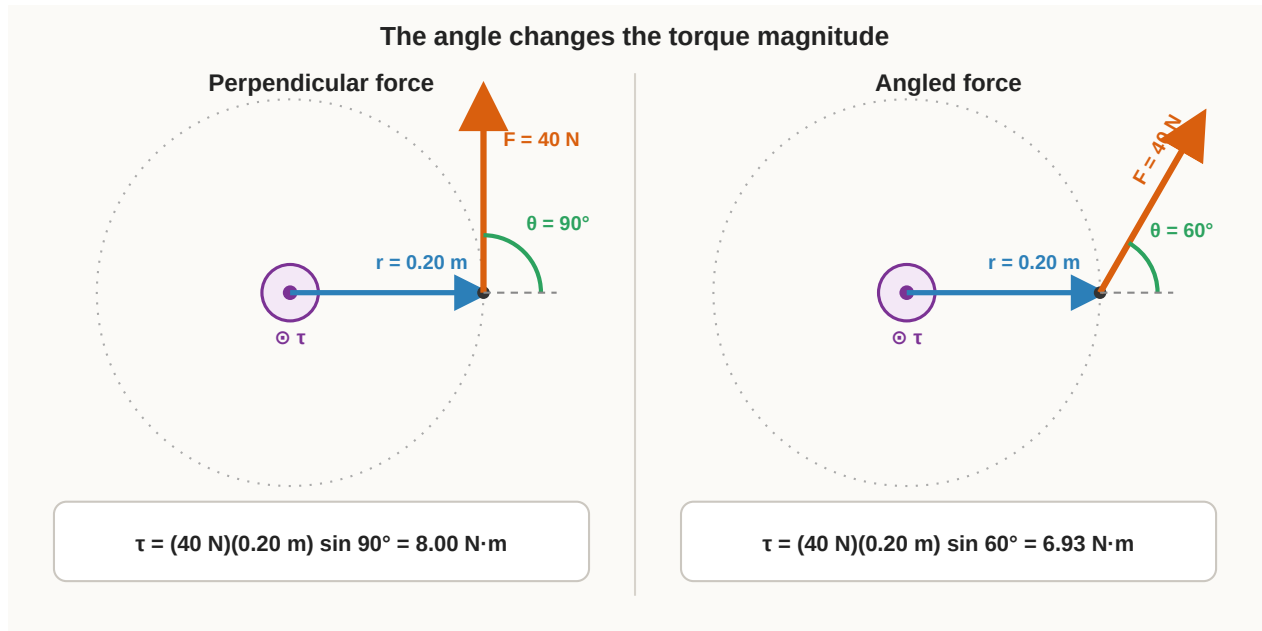


Figure 9.2: Torque magnitude for equal forces applied at 90° and 60° .

Positive or negative τ_a indicates which rotational direction the torque acts about the selected axis. Although torque and energy both have dimensions of force times distance, torque is a vector rotational action, whereas energy is a scalar quantity.

For a wheel of radius r , the position vector from the axle to the ground contact is perpendicular to the longitudinal traction force, so $L = r$ and the contact-torque magnitude is rF . This geometry is the origin of the commonly used wheel relation $F = \tau/r$, but applying it to a driven wheel requires the signed rotational balance derived next.

9.4.2 2.2 The wheel rotational balance

Let $i \in \{L, R\}$ identify the left or right drive wheel. Use the same sign convention as the kinematics chapter: $\dot{\phi}_i > 0$ means that wheel i rotates in the direction that rolls the robot forward. Its angular acceleration is $\ddot{\phi}_i = d\dot{\phi}_i/dt$, measured in rad s^{-2} .

Let $\tau_i \in \mathbb{R}$ be the signed drive torque applied at the axle of wheel i , measured in N m , with positive torque acting in the positive wheel-rotation direction. Let $F_i \in \mathbb{R}$ be the signed longitudinal force exerted by the ground on wheel i , measured in newtons, with $F_i > 0$ pointing forward. This ground-contact force is called the wheel's traction force.

Let $I_i \in \mathbb{R}_{>0}$ be the wheel's mass moment of inertia about its axle, measured in kg m^2 . A mass moment of inertia quantifies resistance to angular acceleration about a specified axis; it is the rotational counterpart of mass in translational dynamics.

The positive axle torque tends to increase $\dot{\phi}_i$. The moment of a positive traction force about the axle acts in the opposite rotational direction, so the signed net torque about the axle is

$$\tau_{\text{net},i} = \tau_i - rF_i.$$

Newton's second law for rotation states that net torque equals mass moment of inertia times angular acceleration. Therefore,

$$\tau_i - rF_i = I_i \ddot{\phi}_i.$$

Solving this balance for traction gives

$$F_i = \frac{\tau_i - I_i \ddot{\phi}_i}{r}.$$

The wheel must use part of the applied torque to accelerate its own rotational inertia. If wheel inertia is negligible or $\ddot{\phi}_i$ is sufficiently small, the balance reduces to the quasi-static approximation

$$F_i \approx \frac{\tau_i}{r}.$$

Here, *quasi-static* means that the wheel's rotational-inertia term is small enough to neglect in the torque balance; it does not mean that the robot's translational acceleration must be zero. The dimensional check is

$$\frac{\text{N m}}{\text{m}} = \text{N},$$

which matches the unit of traction force.

9.4.3 2.3 Combining the two drive wheels

The motor torques are internal interactions when the robot, motors, and wheels are treated as one system. The longitudinal forces exerted by the ground are external forces, so they accelerate the whole robot. During straight forward driving, assume $F_L \geq 0$ and $F_R \geq 0$. Their total nonnegative drive-force magnitude is

$$F_{\text{drive}} = F_L + F_R.$$

Under the quasi-static wheel approximation,

$$F_{\text{drive}} \approx \frac{\tau_L + \tau_R}{r}.$$

If $F_{\text{resist}} \geq 0$ is the magnitude of all backward longitudinal resistance included in the model, the chassis acceleration from Section 1 becomes

$$a \approx \frac{1}{m} \left(\frac{\tau_L + \tau_R}{r} - F_{\text{resist}} \right).$$

This equation assumes straight motion and does not describe yaw acceleration caused by unequal wheel torques. It also assumes that the wheel-ground contacts can supply the calculated traction without slipping.

9.4.4 2.4 Ideal gearing

An electric motor often rotates faster and produces less torque than the wheel requires, so a reduction gearbox is placed between the motor and wheel. Let $G > 1$ be the dimensionless reduction ratio, let $\dot{\phi}_m$ and τ_m be the motor-shaft angular speed and torque, and let $\dot{\phi}_w$ and τ_w be the wheel-side angular speed and torque. Choose the positive motor and wheel shaft directions so that positive motor rotation produces positive wheel rotation through the transmission.

For an ideal lossless gearbox,

$$\dot{\phi}_w = \frac{\dot{\phi}_m}{G}.$$

Mechanical rotational power is $P = \tau\dot{\phi}$, measured in watts, where $1 \text{ W} = 1 \text{ N m s}^{-1}$. Conservation of power for the ideal gearbox gives

$$\tau_m \dot{\phi}_m = \tau_w \dot{\phi}_w.$$

Substituting $\dot{\phi}_m = G\dot{\phi}_w$ and cancelling the nonzero wheel speed gives

$$\tau_w = G\tau_m.$$

Thus an ideal reduction gearbox divides angular speed by G and multiplies torque by G . If $\eta \in (0, 1]$ is a dimensionless efficiency representing power losses for forward motor-to-wheel power flow, a simple non-ideal approximation is

$$\tau_w = \eta G \tau_m.$$

Combining this approximation with negligible wheel rotational inertia gives

$$F \approx \frac{\eta G \tau_m}{r}.$$

This expression is a demanded traction force, not a guarantee that the ground contact can produce it. Gear losses, bearing friction, motor-current limits, wheel inertia, and the wheel-ground friction limit can all reduce the realised motion.

9.4.5 2.5 Worked example

Consider two identical drive wheels with radius $r = 0.10$ m. Suppose each wheel receives axle torque $\tau_L = \tau_R = 1.2$ N m, and neglect wheel rotational inertia and drivetrain losses. Each wheel's predicted traction is

$$F_L = F_R = \frac{1.2}{0.10} = 12 \text{ N}.$$

The total drive force is therefore

$$F_{\text{drive}} = 12 + 12 = 24 \text{ N}.$$

For robot mass $m = 20$ kg and backward resistance magnitude $F_{\text{resist}} = 4$ N, the predicted acceleration is

$$a = \frac{24 - 4}{20} = 1 \text{ m s}^{-2}.$$

This result remains valid only while the assumed torques are available and both wheel contacts roll without longitudinal slip.

9.4.6 Quick test B

1. Define torque, moment arm, mass moment of inertia, and traction force, including units.
2. Starting from $\tau_i - rF_i = I_i\ddot{\phi}_i$, solve for F_i .
3. State the approximation required to use $F_i \approx \tau_i/r$.
4. Why does the ground-contact force, rather than the internal motor torque by itself, appear in the whole-robot translational force balance?
5. An ideal gearbox has $G = 20$, motor torque $\tau_m = 0.15$ N m, and wheel radius $r = 0.06$ m. Neglecting inertia and losses, calculate the wheel torque and demanded traction force.
6. Why can the demanded force τ_w/r differ from the traction force realised on the ground?

9.4.7 Quick test B answers and hints

1. Torque is a rotational action about an axis in N m; the moment arm is the perpendicular axis-to-line-of-action distance in metres; mass moment of inertia measures resistance to angular acceleration about an axis in kg m^2 ; and traction is the longitudinal ground-contact force on the wheel in newtons.
2. Move $I_i\ddot{\phi}_i$ to the other side and divide by $r > 0$ to obtain $F_i = (\tau_i - I_i\ddot{\phi}_i)/r$.
3. The wheel rotational-inertia term $I_i\ddot{\phi}_i$ must be negligible relative to the applied torque, and omitted drivetrain losses must also be acceptably small.
4. Motor torque transfers forces internally among the motor, wheel, and chassis; the ground exerts the external longitudinal force that changes the momentum of the whole robot.
5. $\tau_w = G\tau_m = (20)(0.15) = 3.0$ N m and $F = \tau_w/r = 3.0/0.06 = 50$ N.
6. The wheel may need torque to accelerate its own inertia, the drivetrain has losses and limits, and the contact may slip when the demanded traction exceeds the available friction.

9.5 References

- Kevin M. Lynch and Frank C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Chapter 8, especially Sections 8.1, 8.2, and 8.9.
- Karl J. Åström and Richard M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, Chapter 3.